



The **AiEdge** ACADEMY

How to Parallelize Training



CPU vs GPU vs TPU

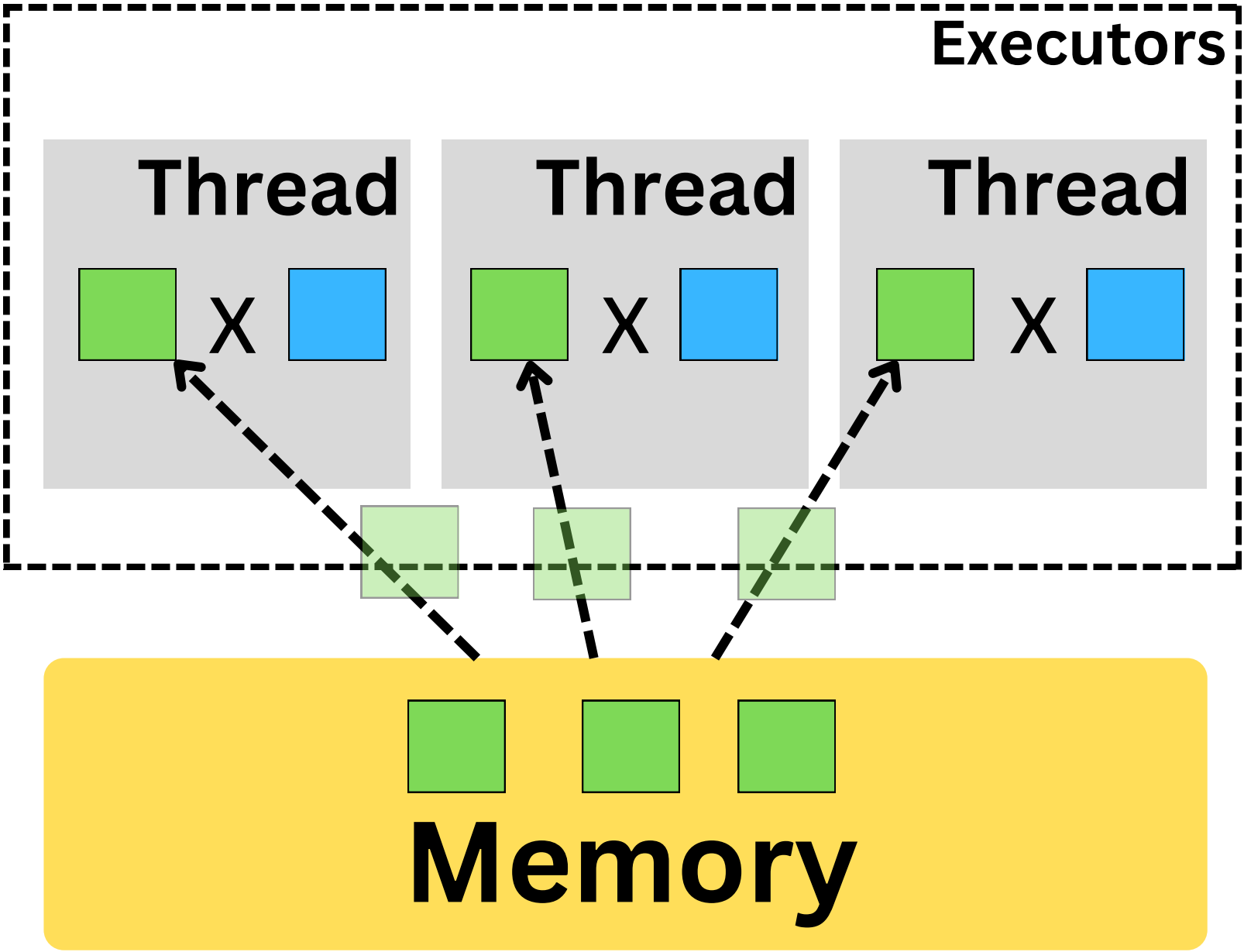
- **CPU:** complex instructions between scalars
- **GPU:** simple computations between vector data
- **TPU:** simple computations between matrix data



CPU

Data available
for processing

Data in memory

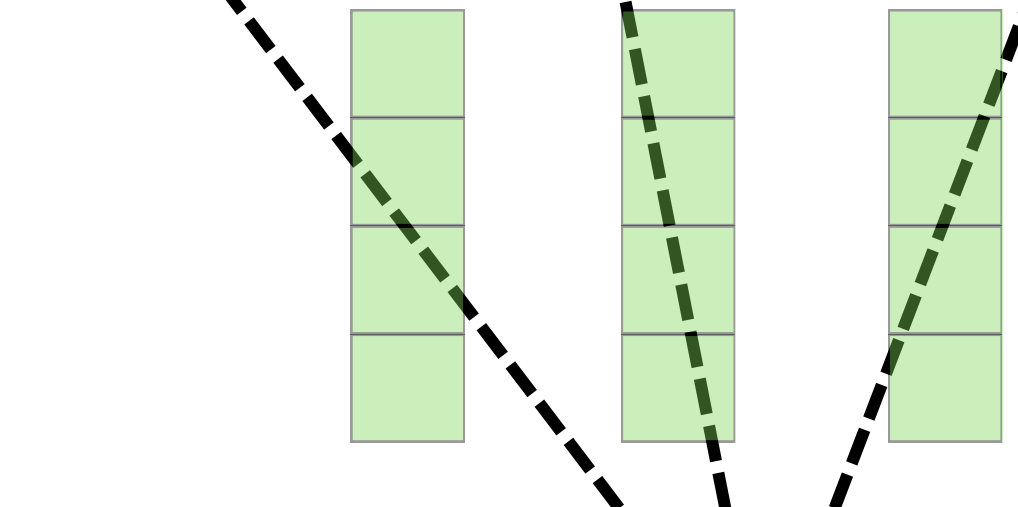
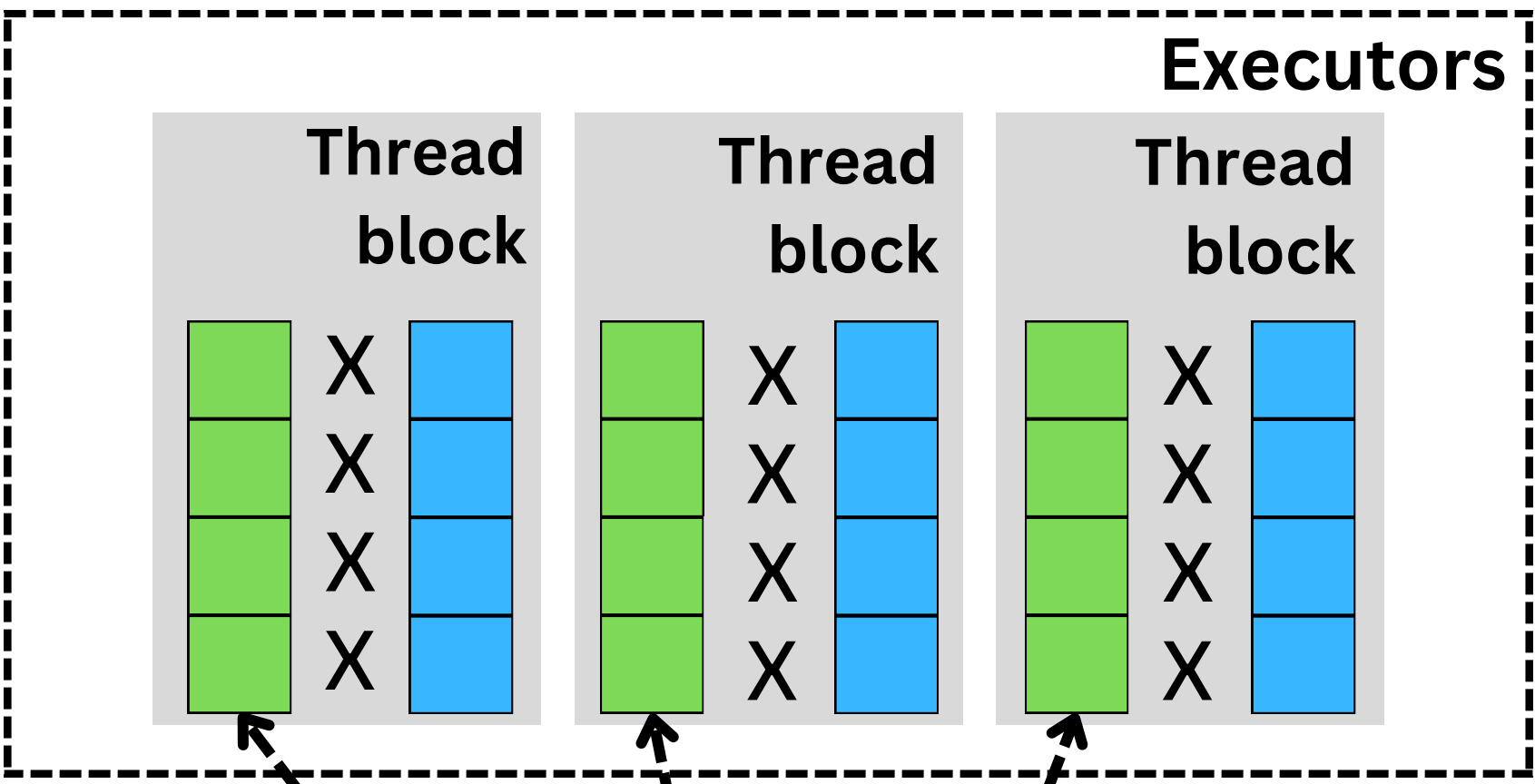


Parallelized scalar multiplications



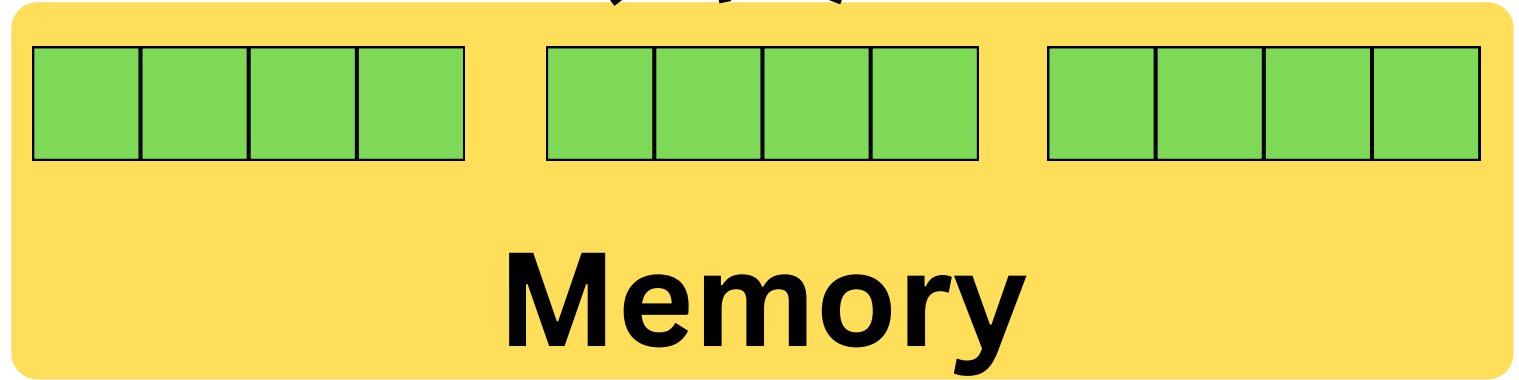
GPU

Vector data
available
for processing



Parallelized
vector
multiplications

Vector data in
memory



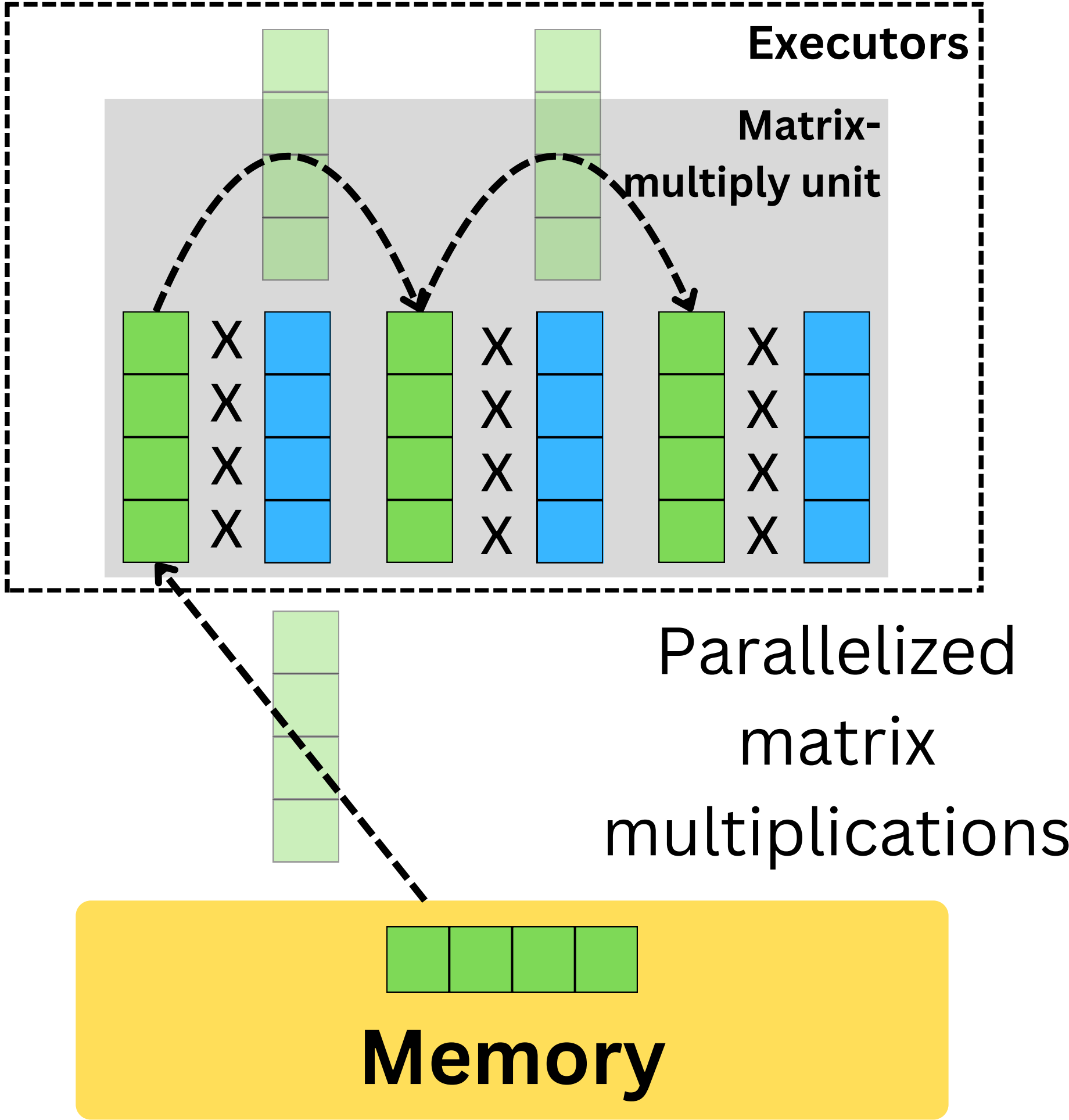
Memory



TPU

Matrix data
available
for processing

Vector data in
memory

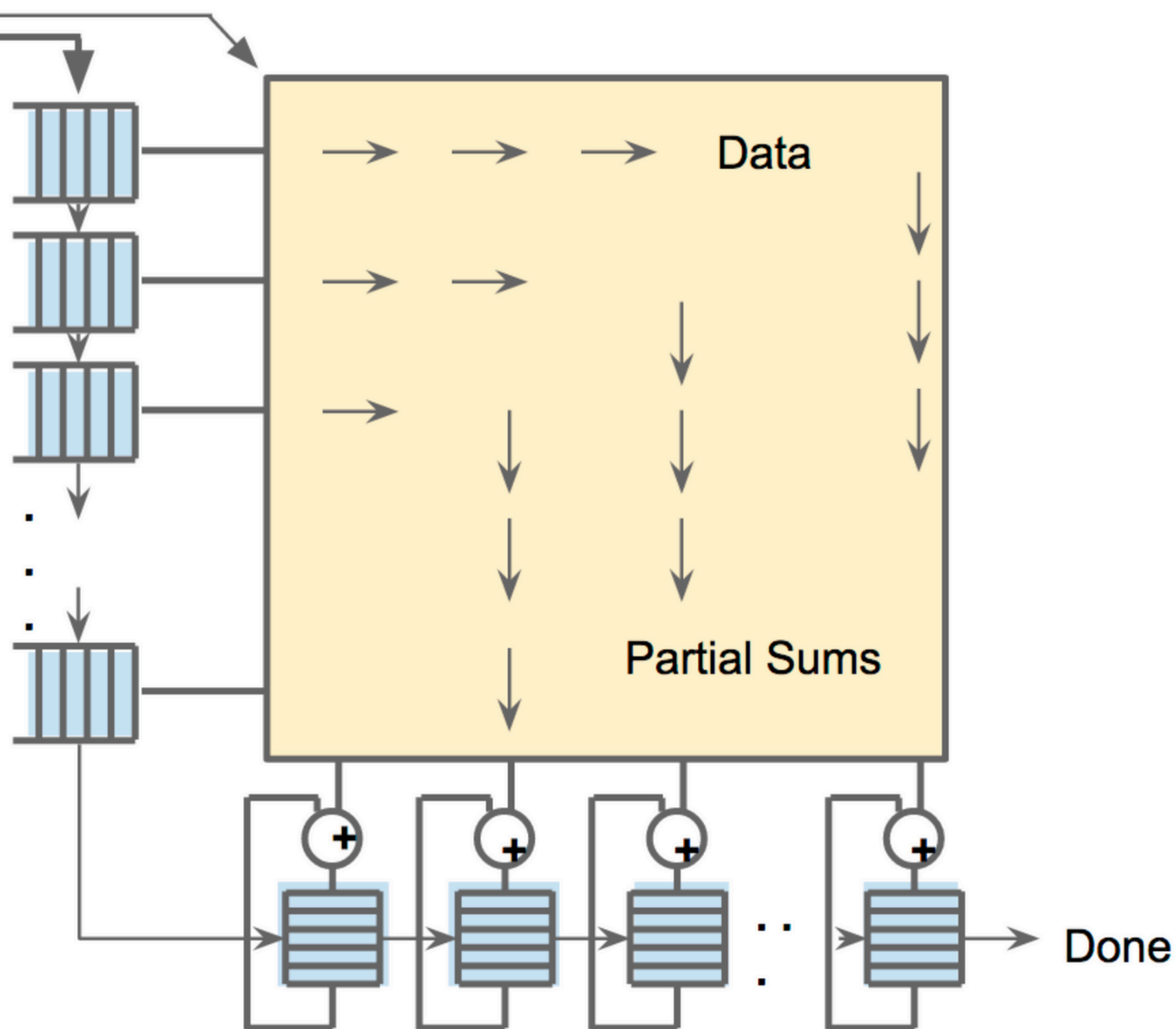




TPU



Systolic data flow of the Matrix Multiply Unit

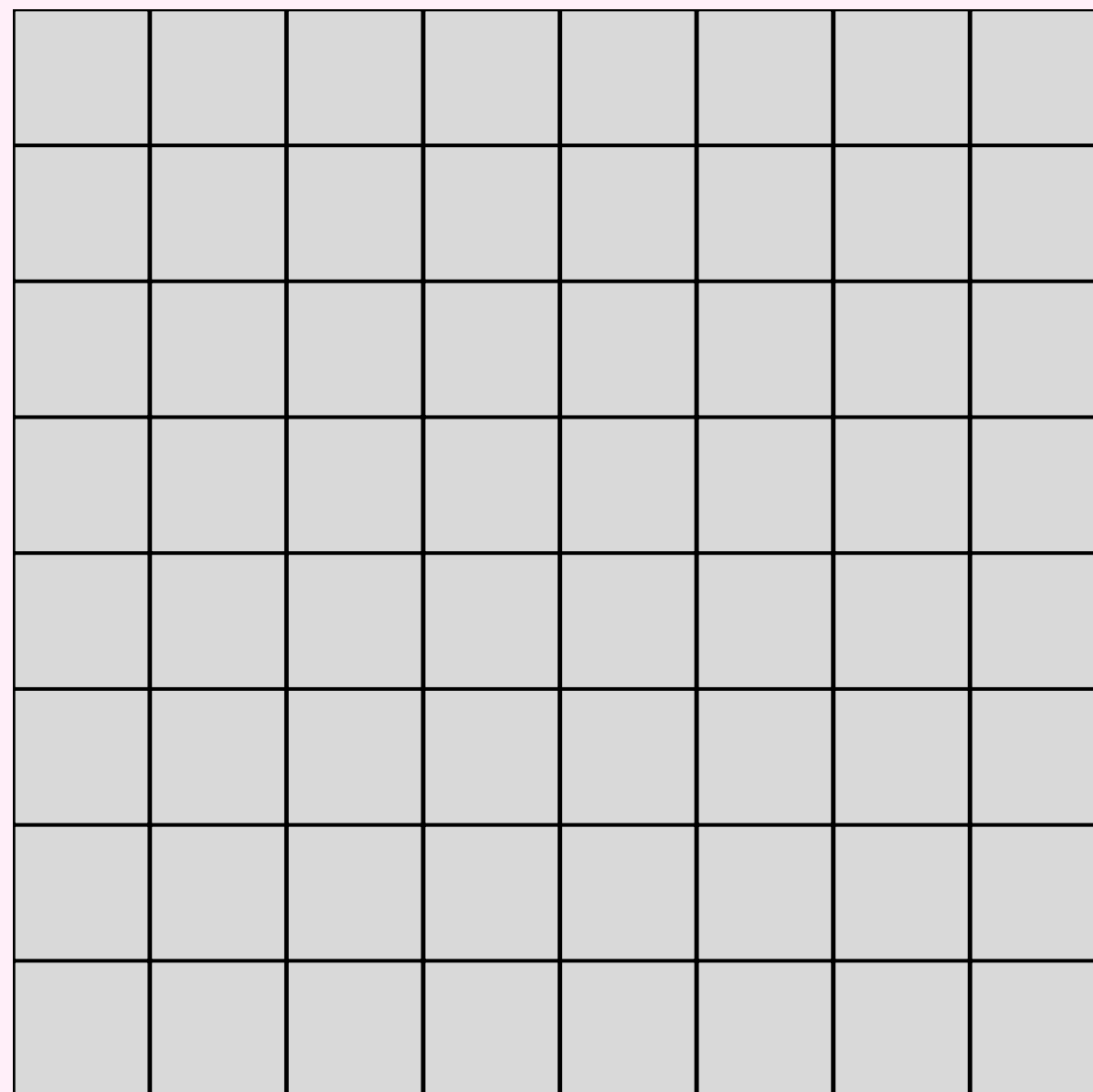




The GPU Architecture

smallest unit of processing

- addition
- subtraction
- multiplication
- and division

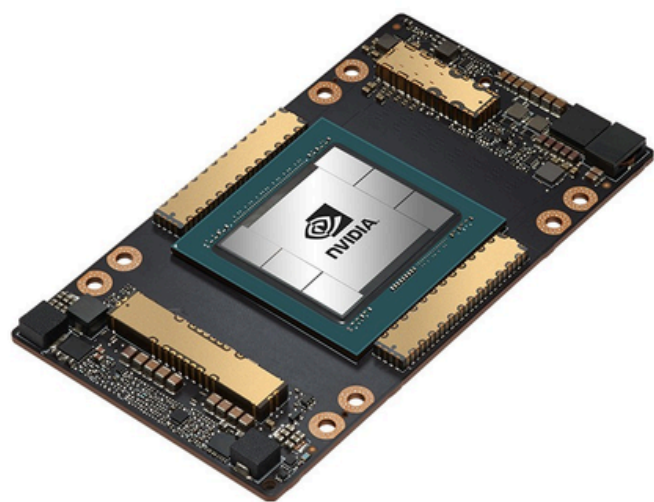


Multiple threads



The GPU Architecture

NVIDIA A100 – Ampere (2020)



- 6,912 CUDA cores
- 432 Tensor Cores

NVIDIA H100 – Hopper (2022)



- 16,896 CUDA cores
- 528 Tensor Cores

GeForce RTX 4090 Ada Lovelace (2022)

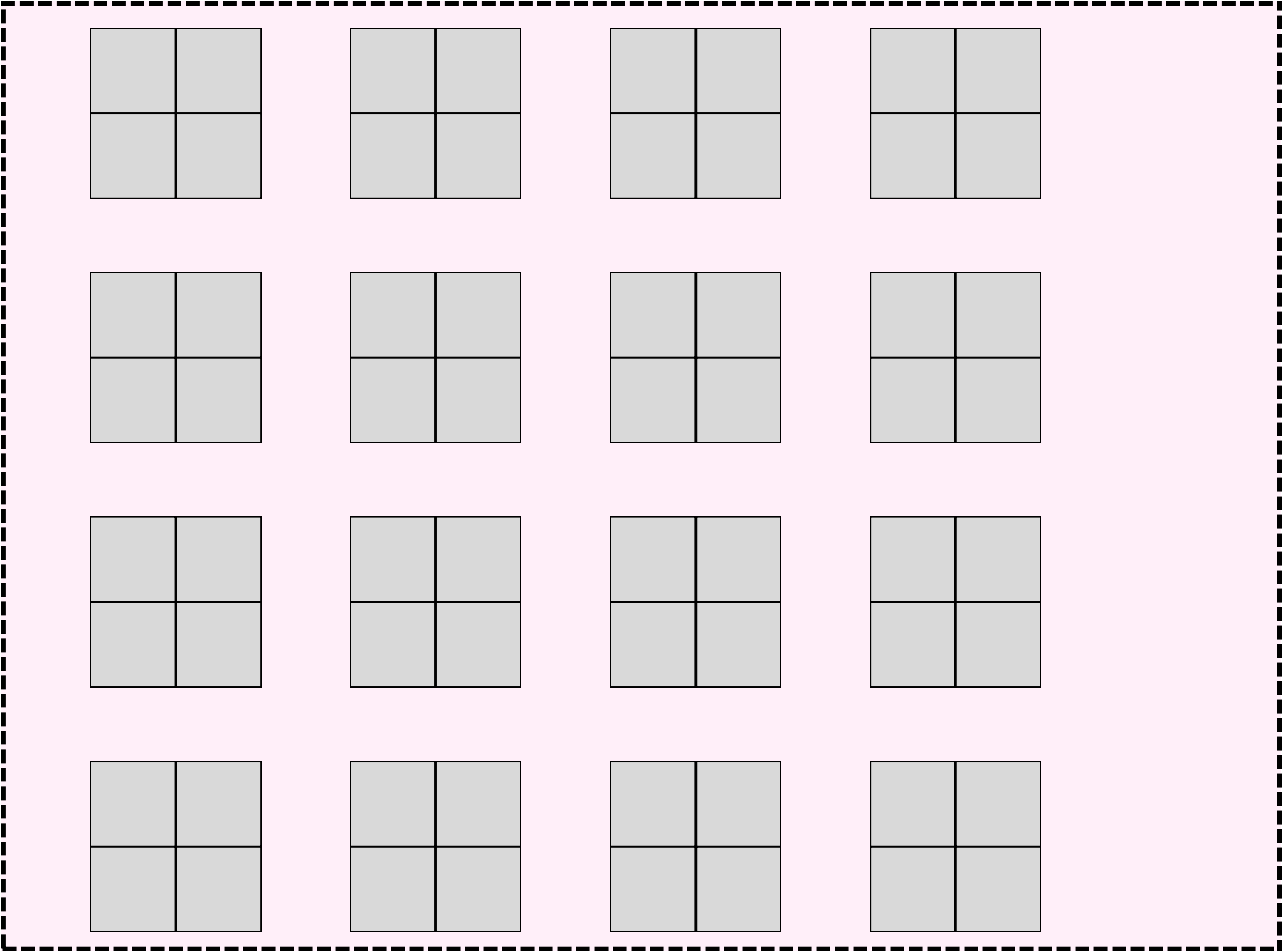


- 16,384 CUDA cores
- 512 Tensor Cores



The GPU Architecture

Thread blocks





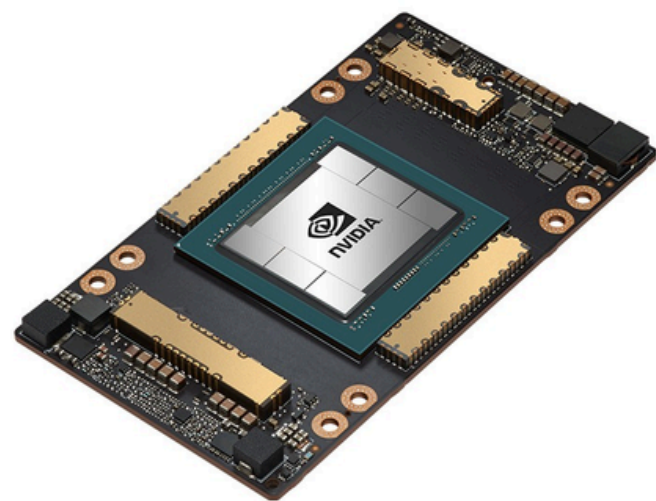
The GPU Architecture

NVIDIA A100 – Ampere (2020)

NVIDIA H100 – Hopper (2022)

GeForce RTX 4090 Ada Lovelace (2022)

- Up to 1024 threads per block

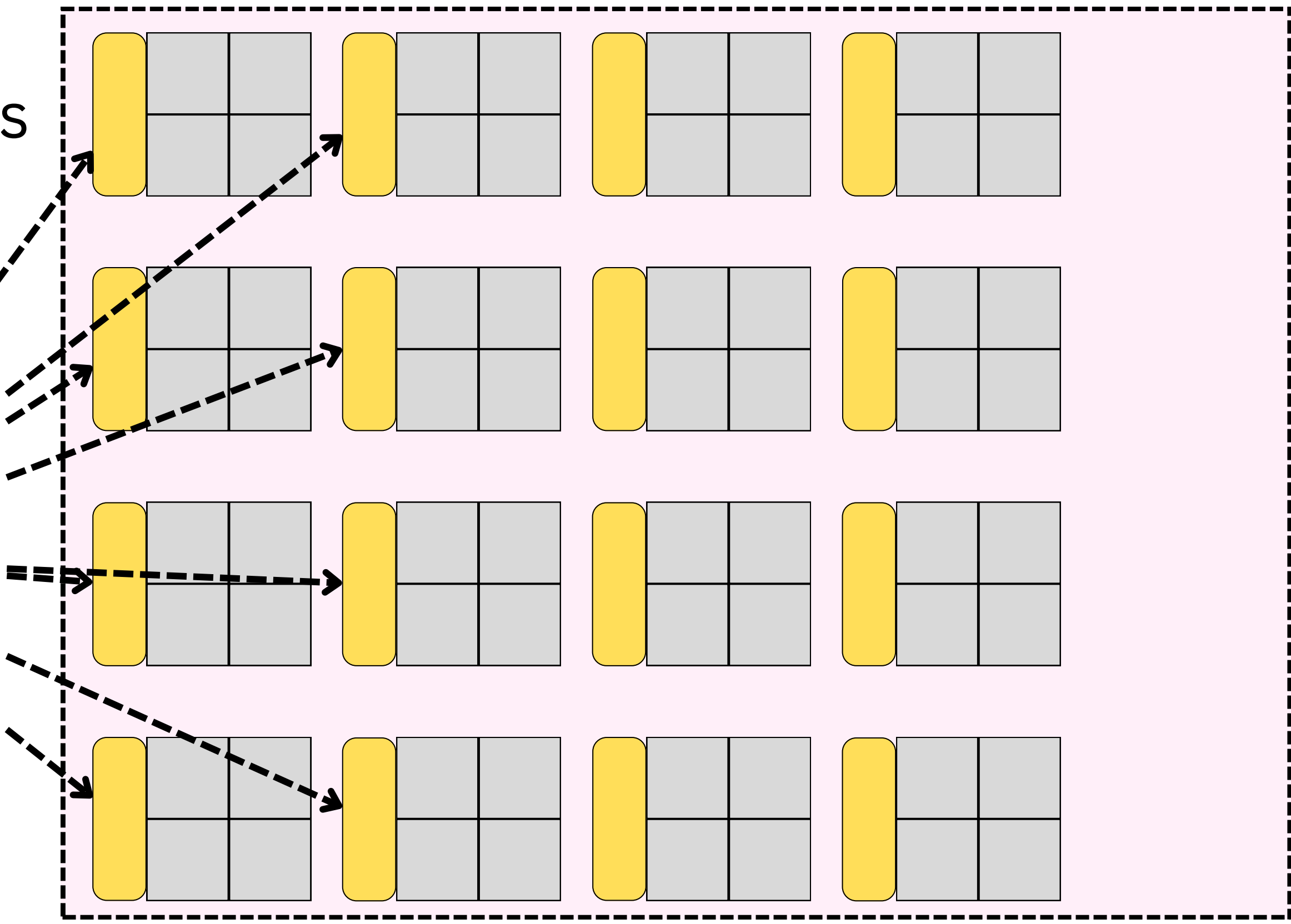




The GPU Architecture

Thread blocks

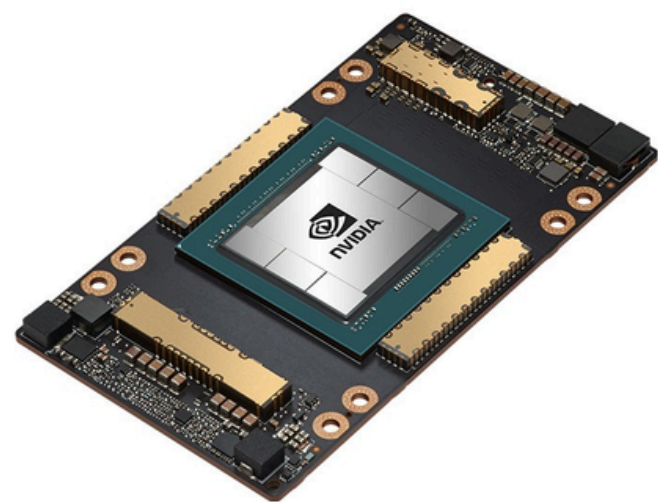
Threads in
each block
have fast
shared
memory





The GPU Architecture

NVIDIA A100 – Ampere (2020)



~17 MB of
Shared
Memory

NVIDIA H100 – Hopper (2022)



~24 MB of
Shared
Memory

GeForce RTX 4090 Ada Lovelace (2022)



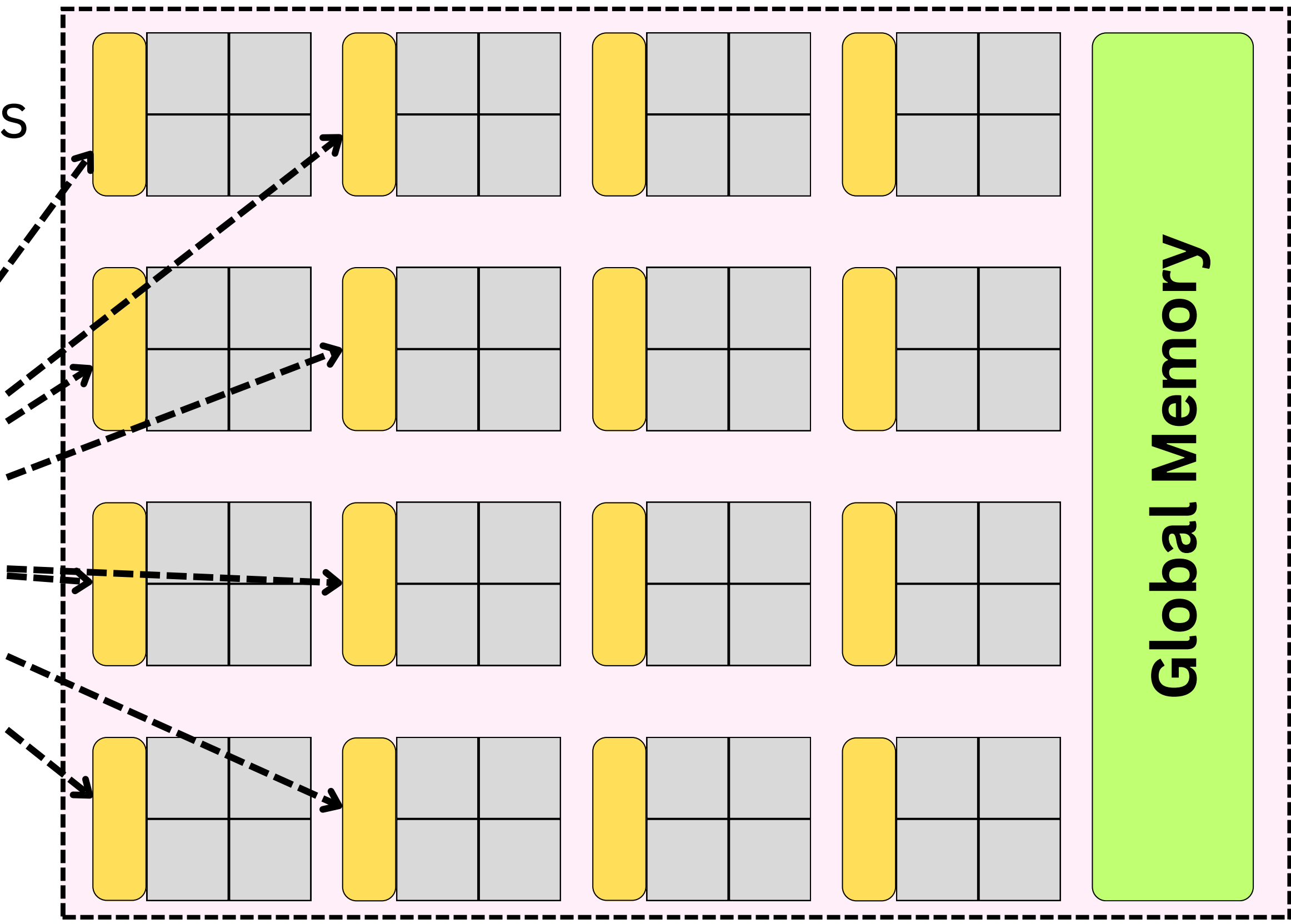
~24 MB of
Shared
Memory



The GPU Architecture

Thread blocks

Threads in each block have fast shared memory

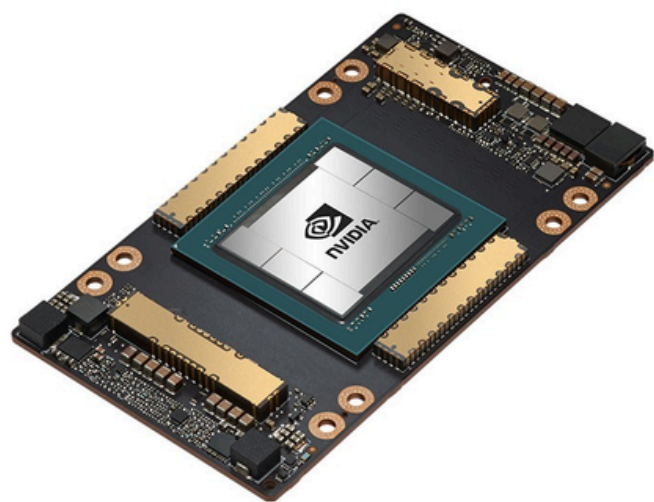


Global Memory or High Bandwidth Memory



The GPU Architecture

NVIDIA A100 – Ampere (2020)



~80 GB of HBM

NVIDIA H100 – Hopper (2022)



~80 GB of HBM

**GeForce RTX 4090
Ada Lovelace (2022)**



~24 GB of HBM

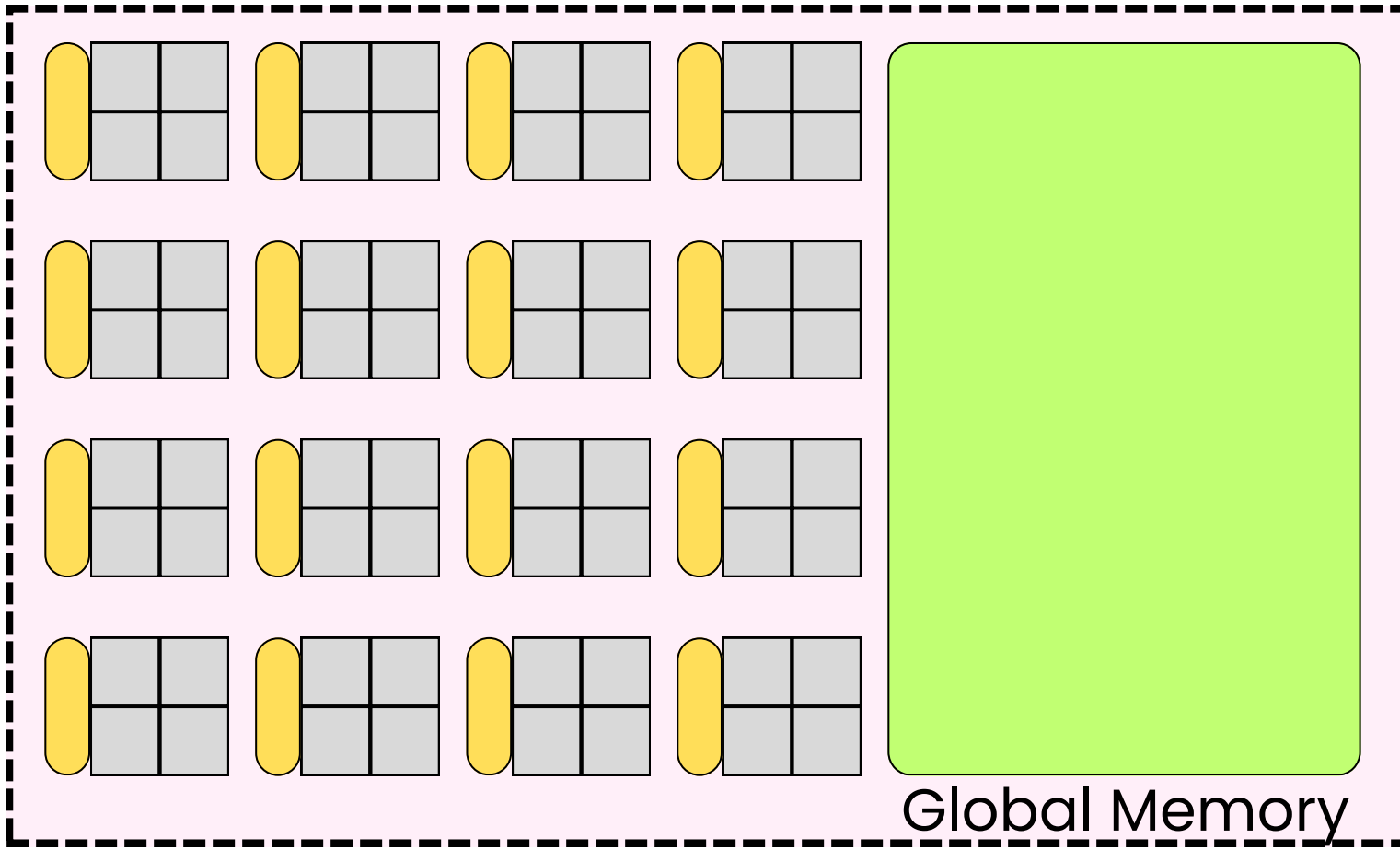
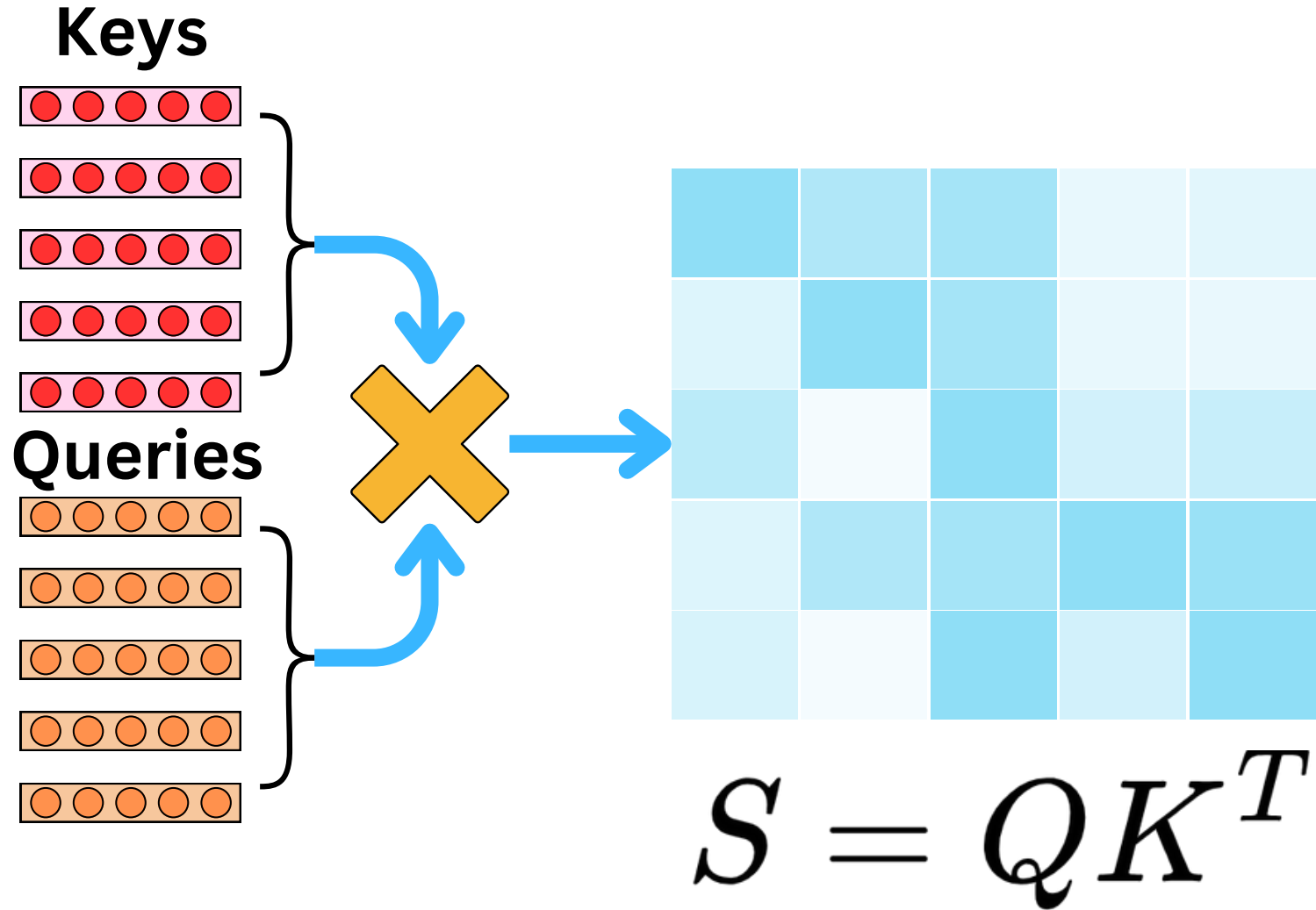


The GPU Architecture

- HBM Bandwidth: 1.5–2.0TB/s
- SRAM Bandwidth: 19TB/s ~ 10x HBM

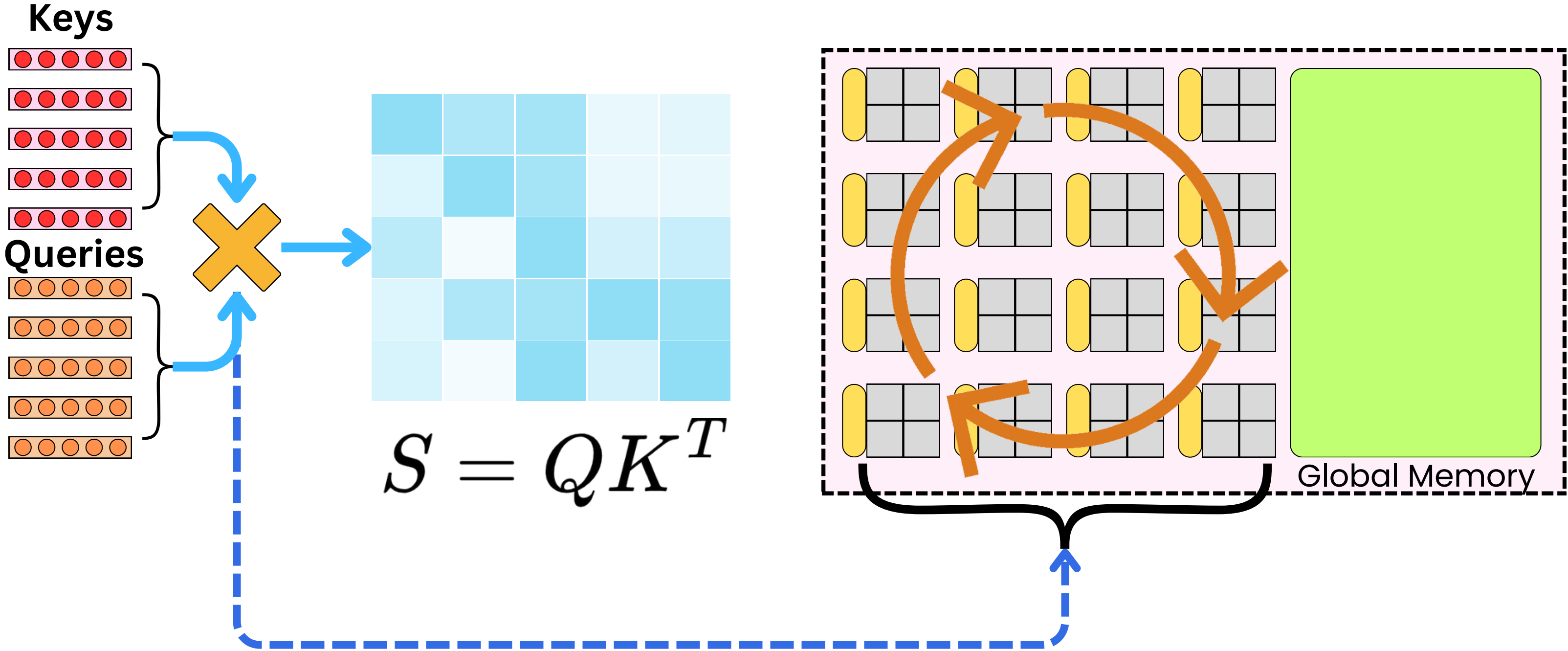


The GPU Architecture



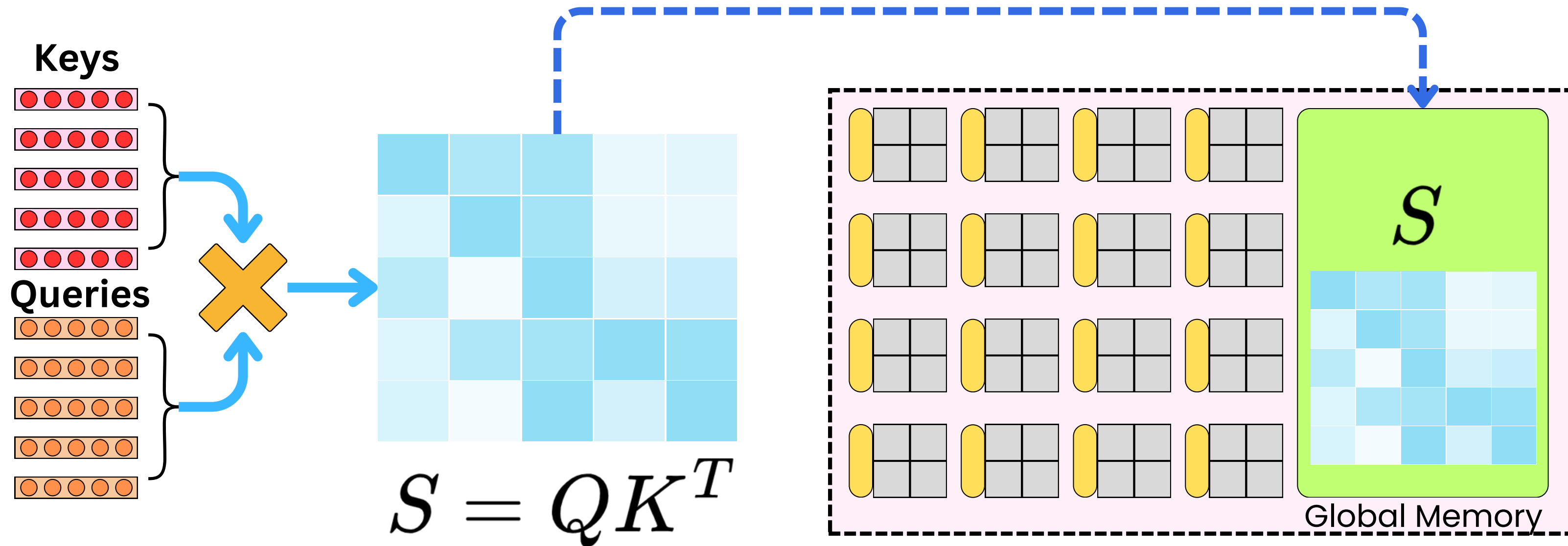


The GPU Architecture





The GPU Architecture

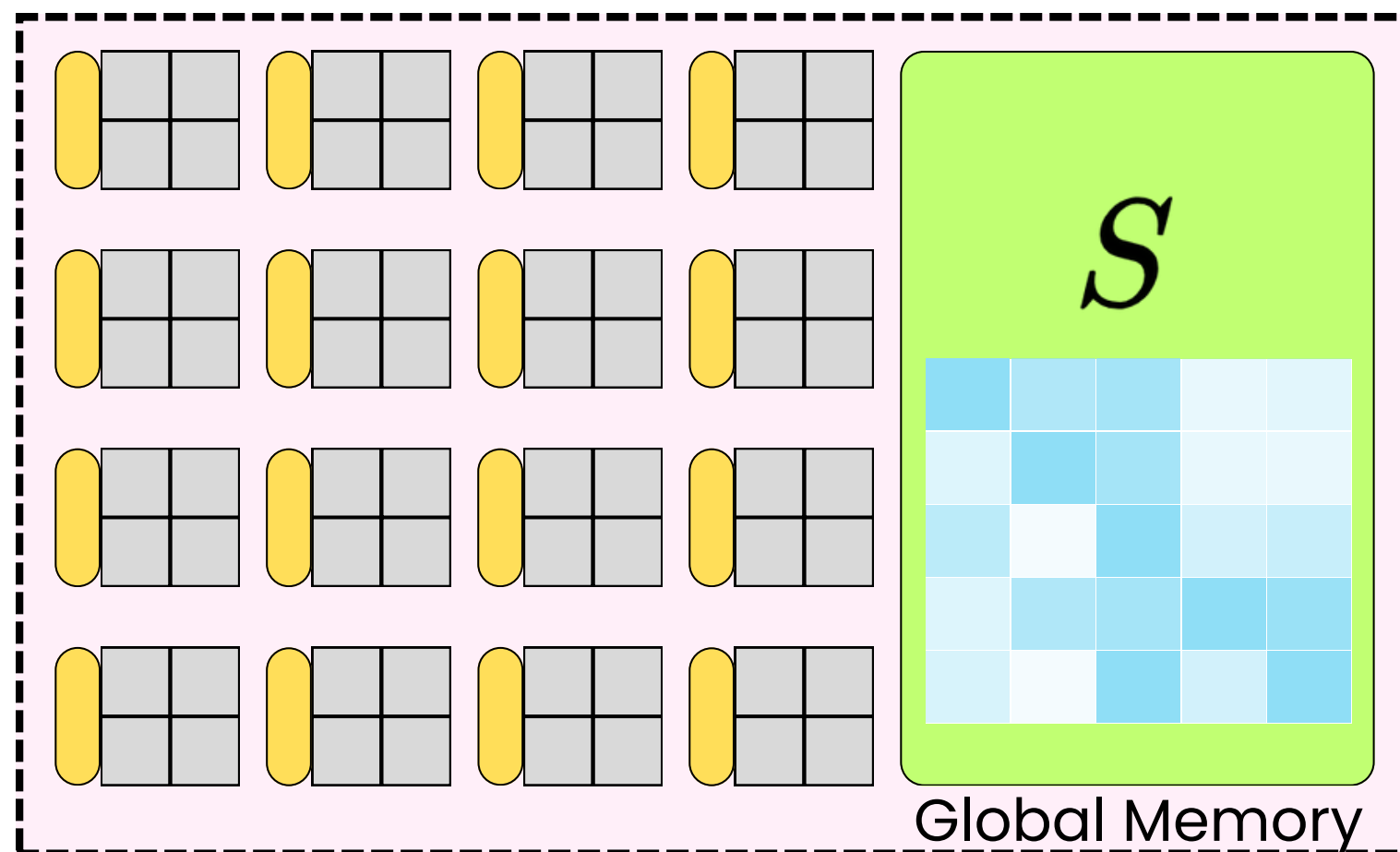
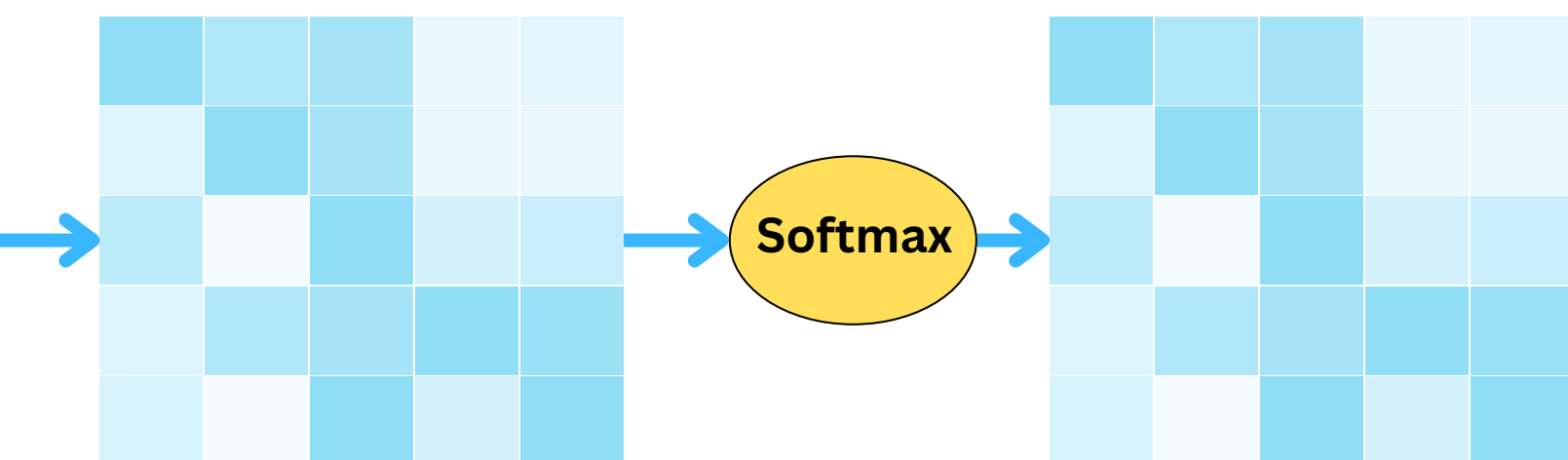




The GPU Architecture

$$\text{Attention} = \text{Softmax}(S)$$

Self-attentions

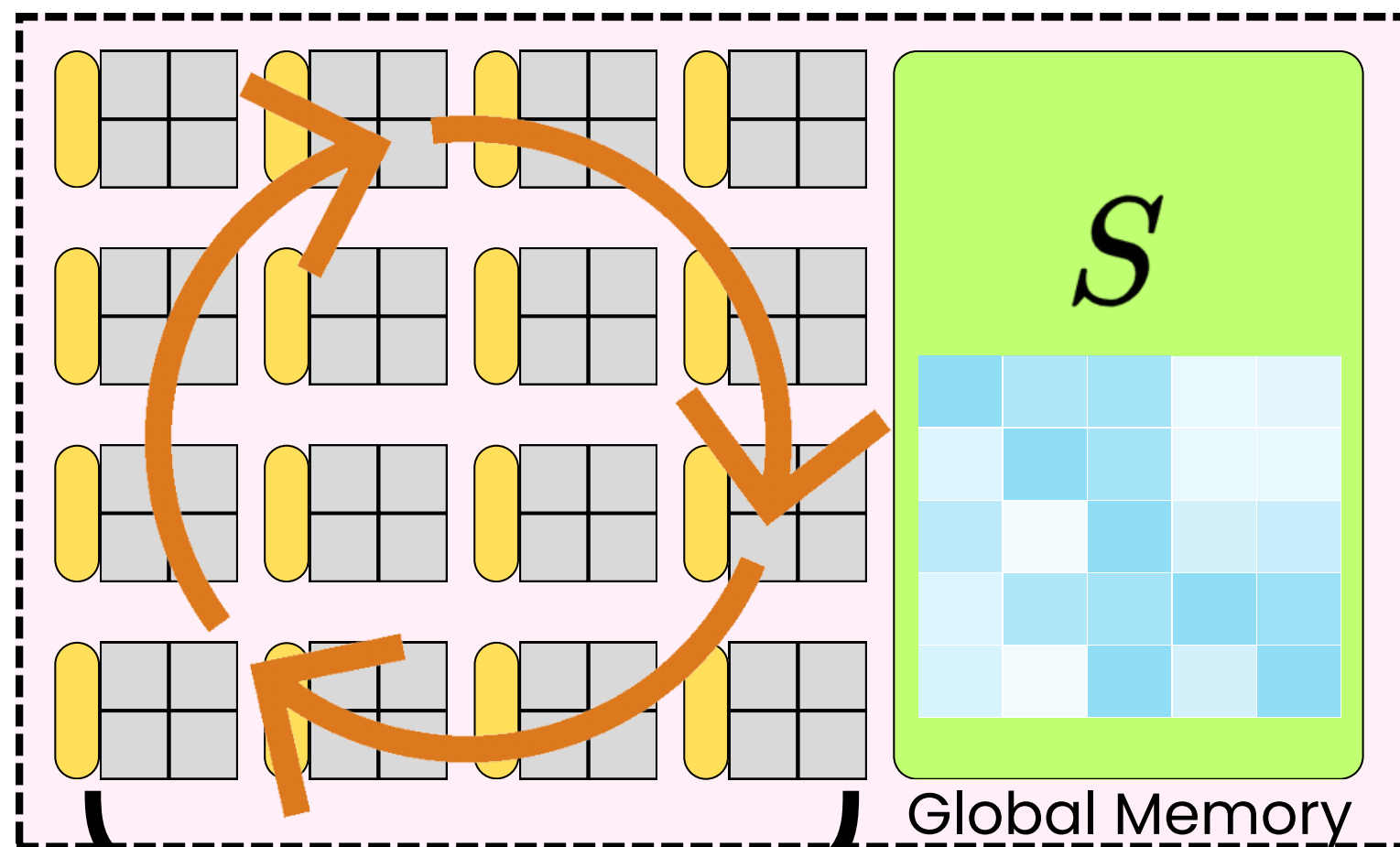
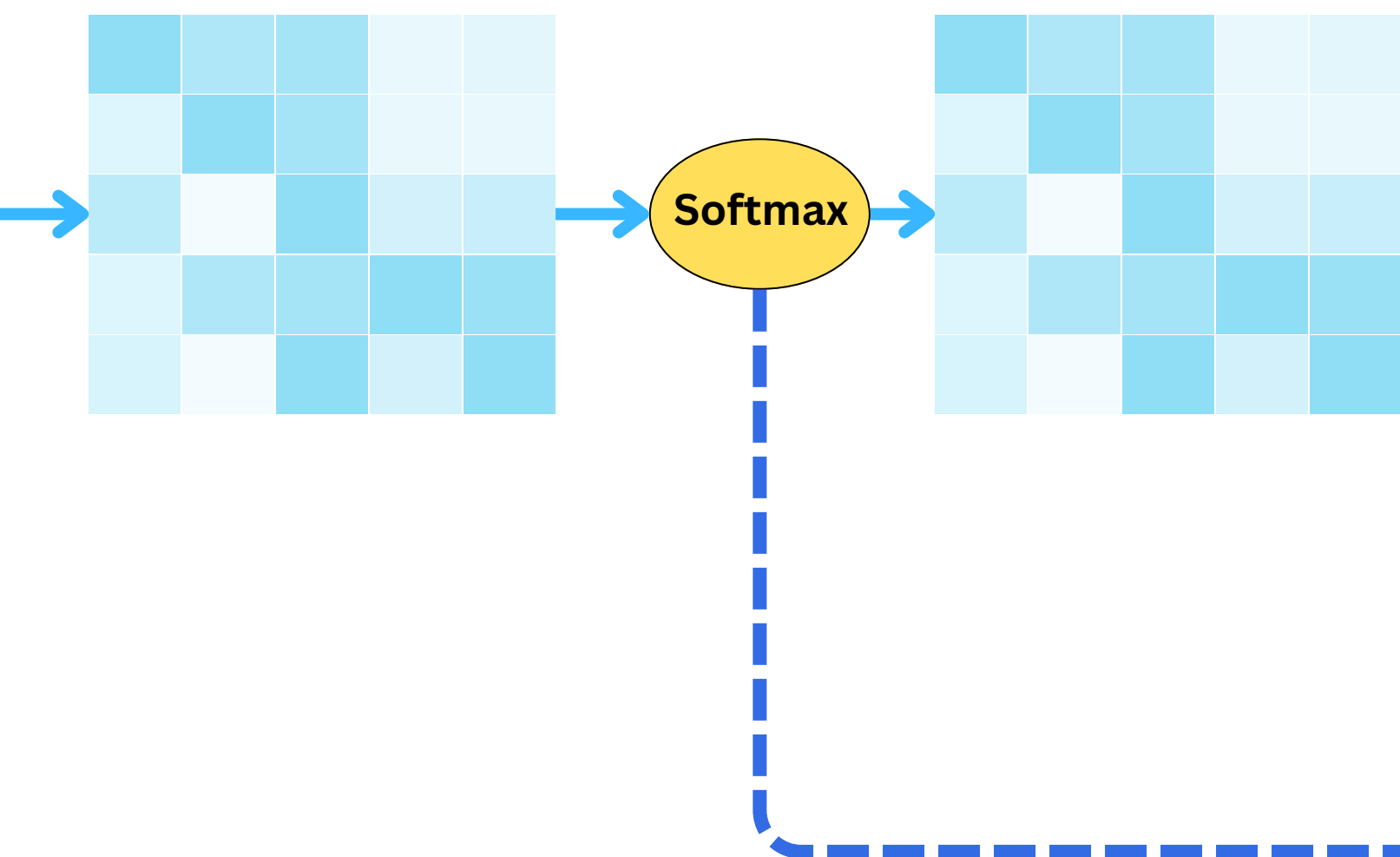




The GPU Architecture

$$\text{Attention} = \text{Softmax}(S)$$

Self-attentions

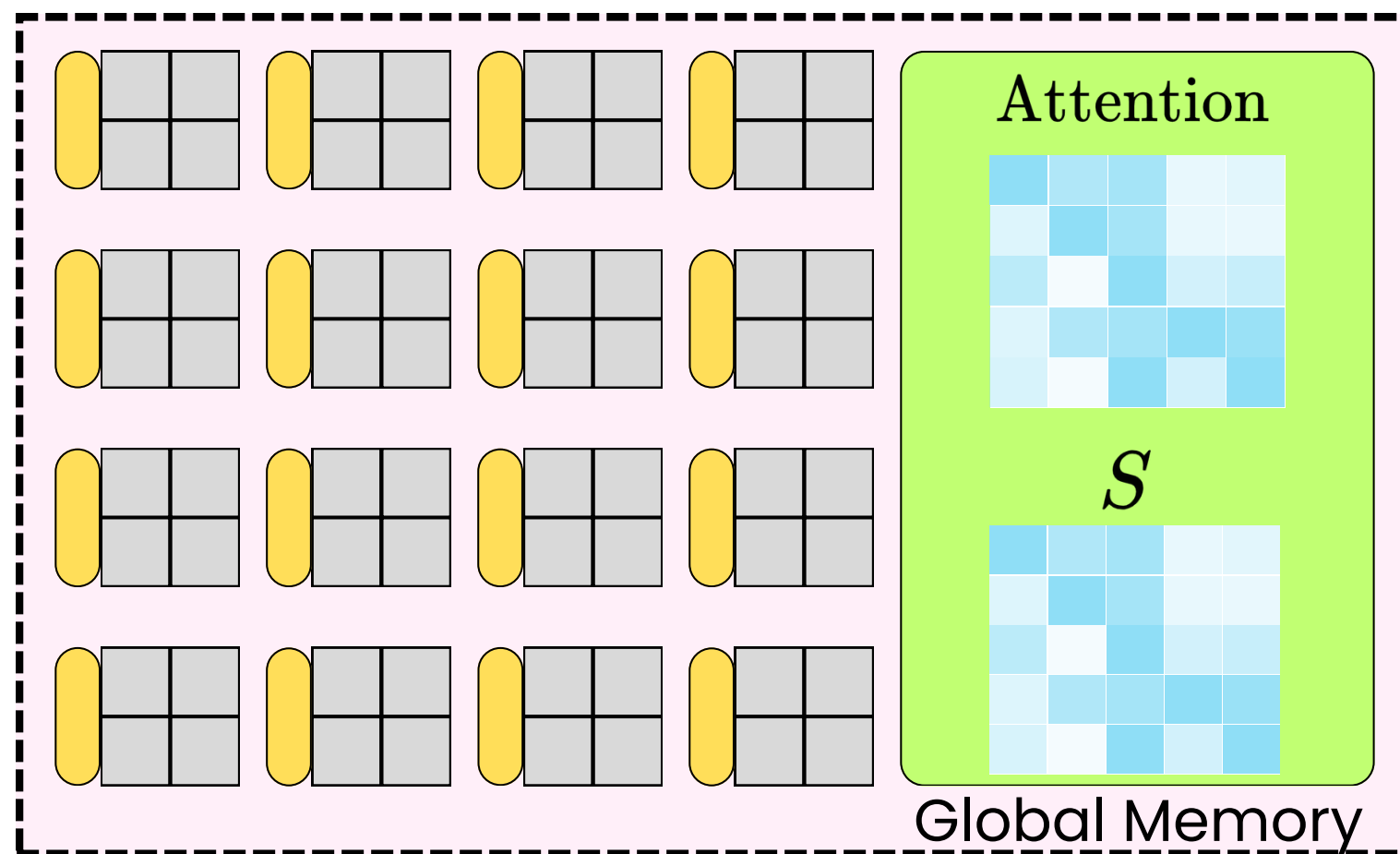
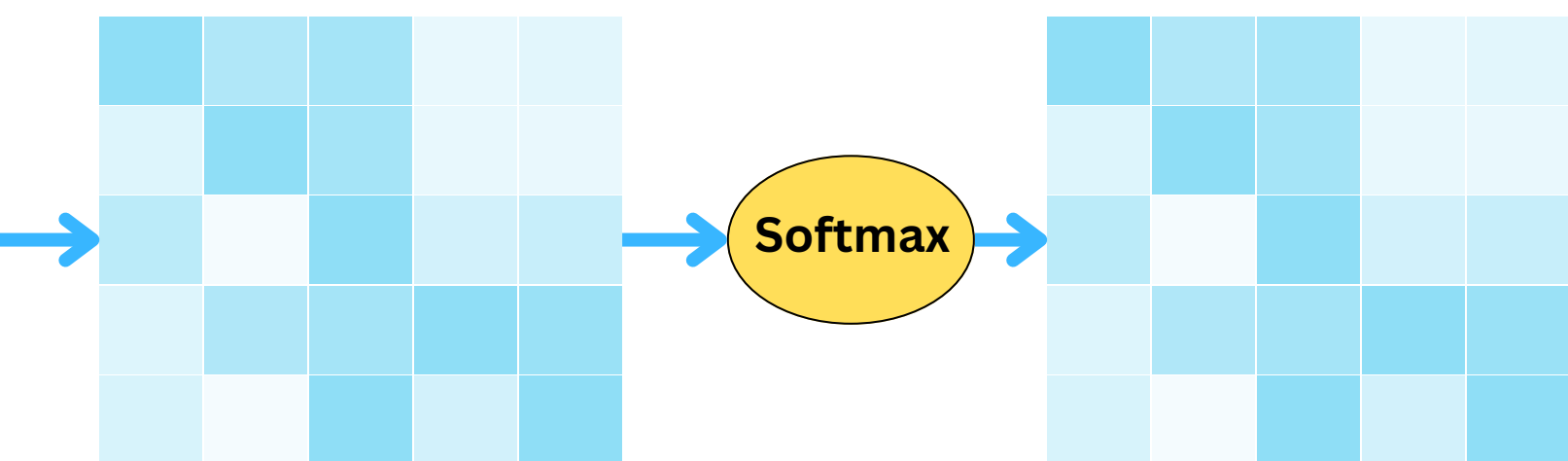




The GPU Architecture

$$\text{Attention} = \text{Softmax}(S)$$

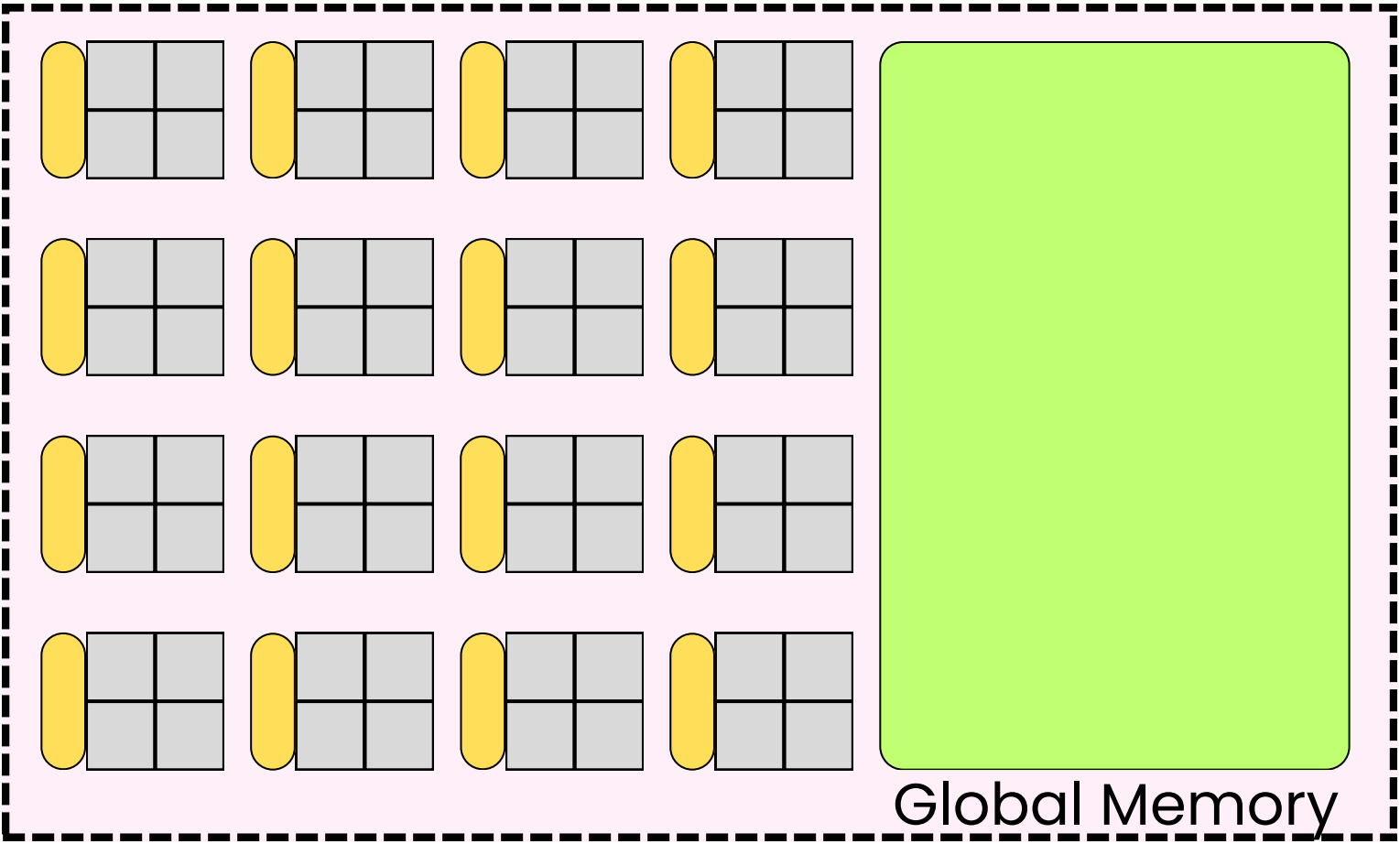
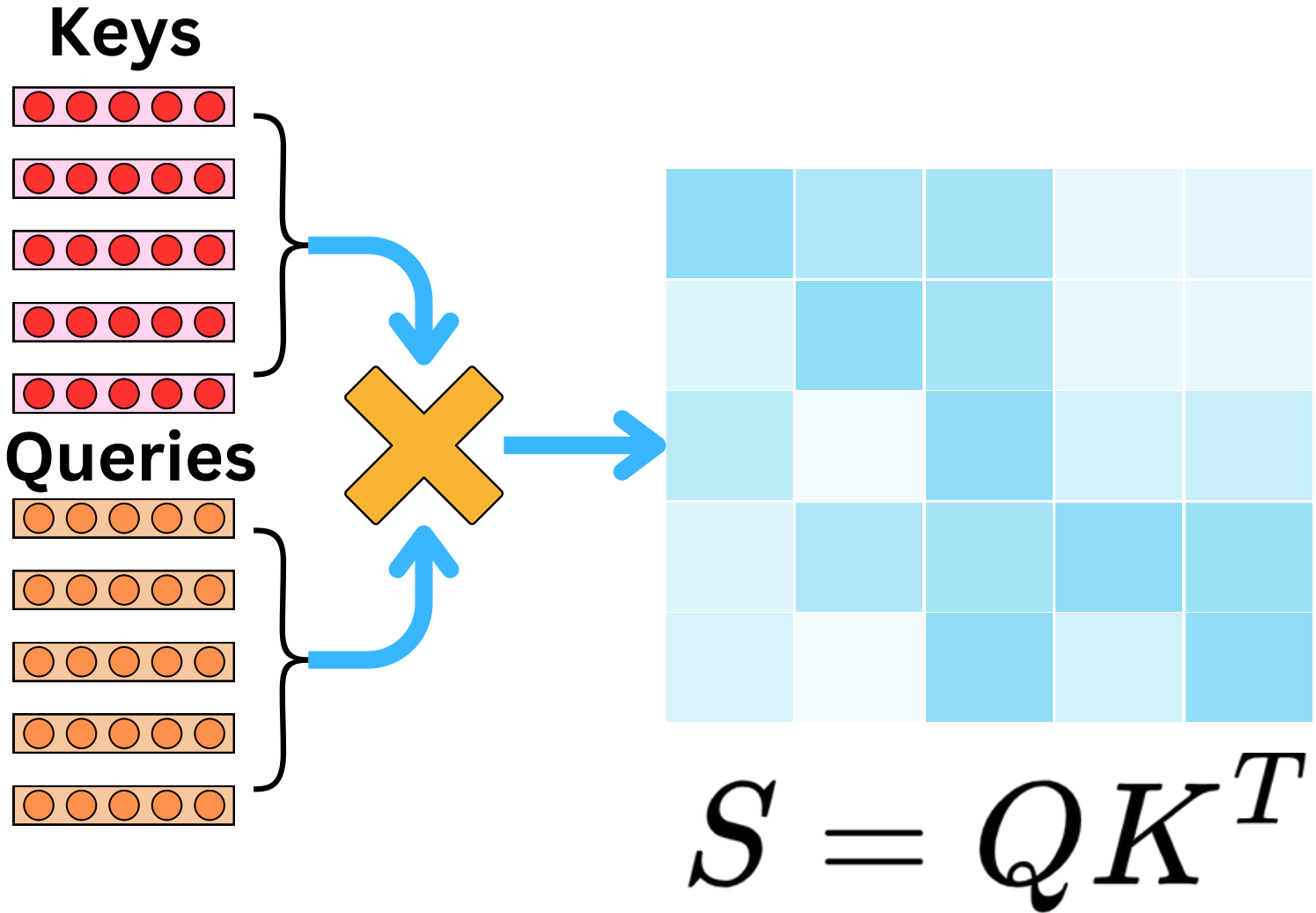
Self-attentions





The GPU Architecture

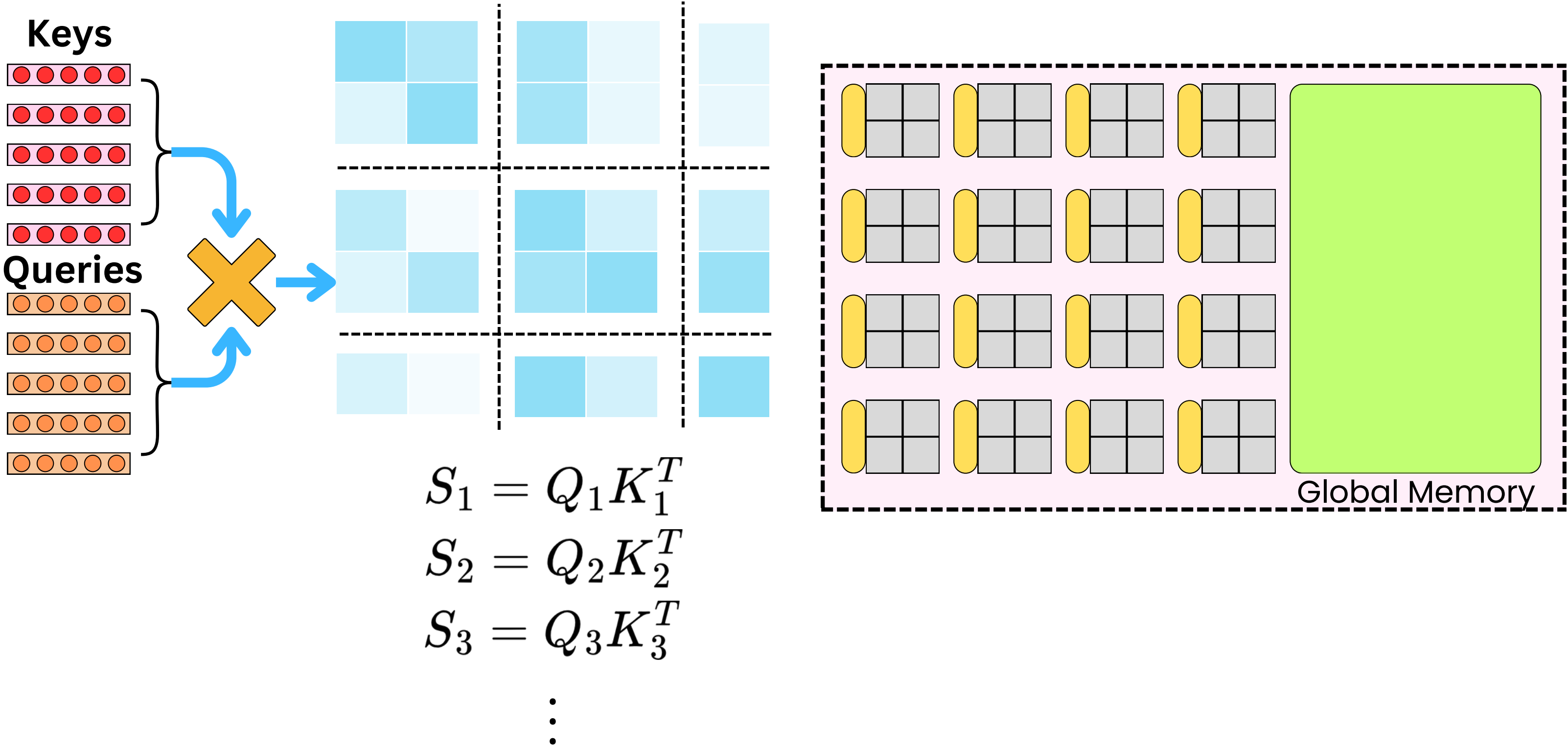
Flash-Attention: tiling the matrix computations





The GPU Architecture

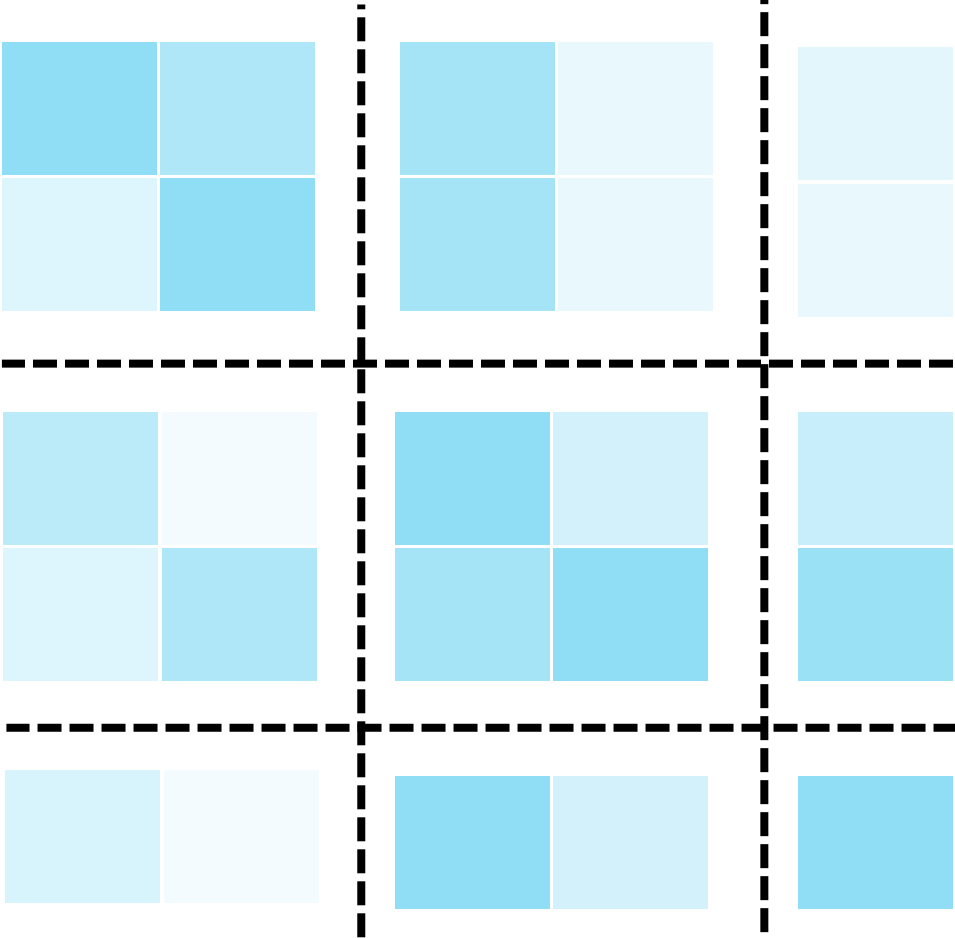
Flash-Attention: tiling the matrix computations



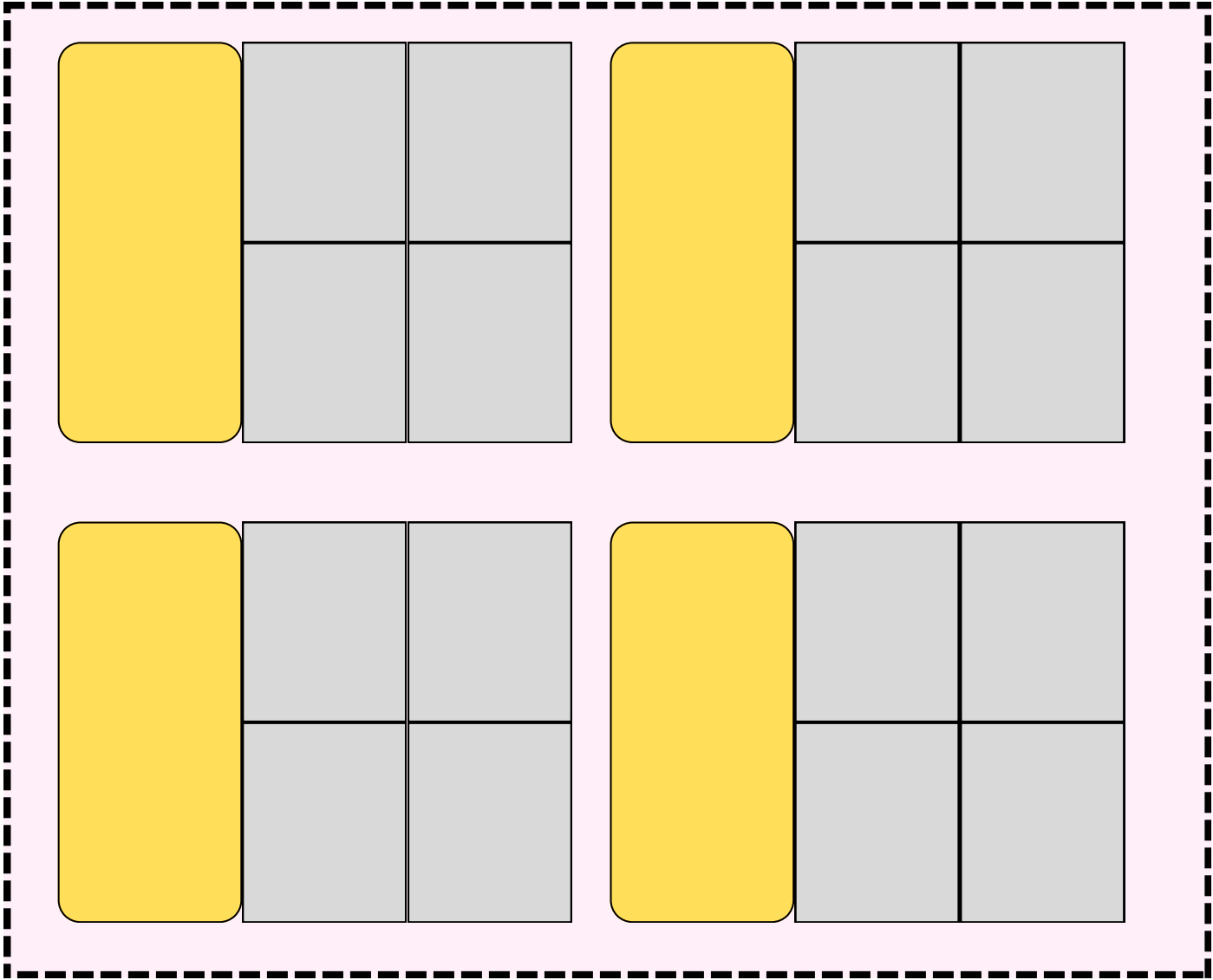


The GPU Architecture

Flash-Attention: tiling the matrix computations



$$\begin{aligned} S_1 &= Q_1 K_1^T \\ S_2 &= Q_2 K_2^T \\ S_3 &= Q_3 K_3^T \end{aligned}$$



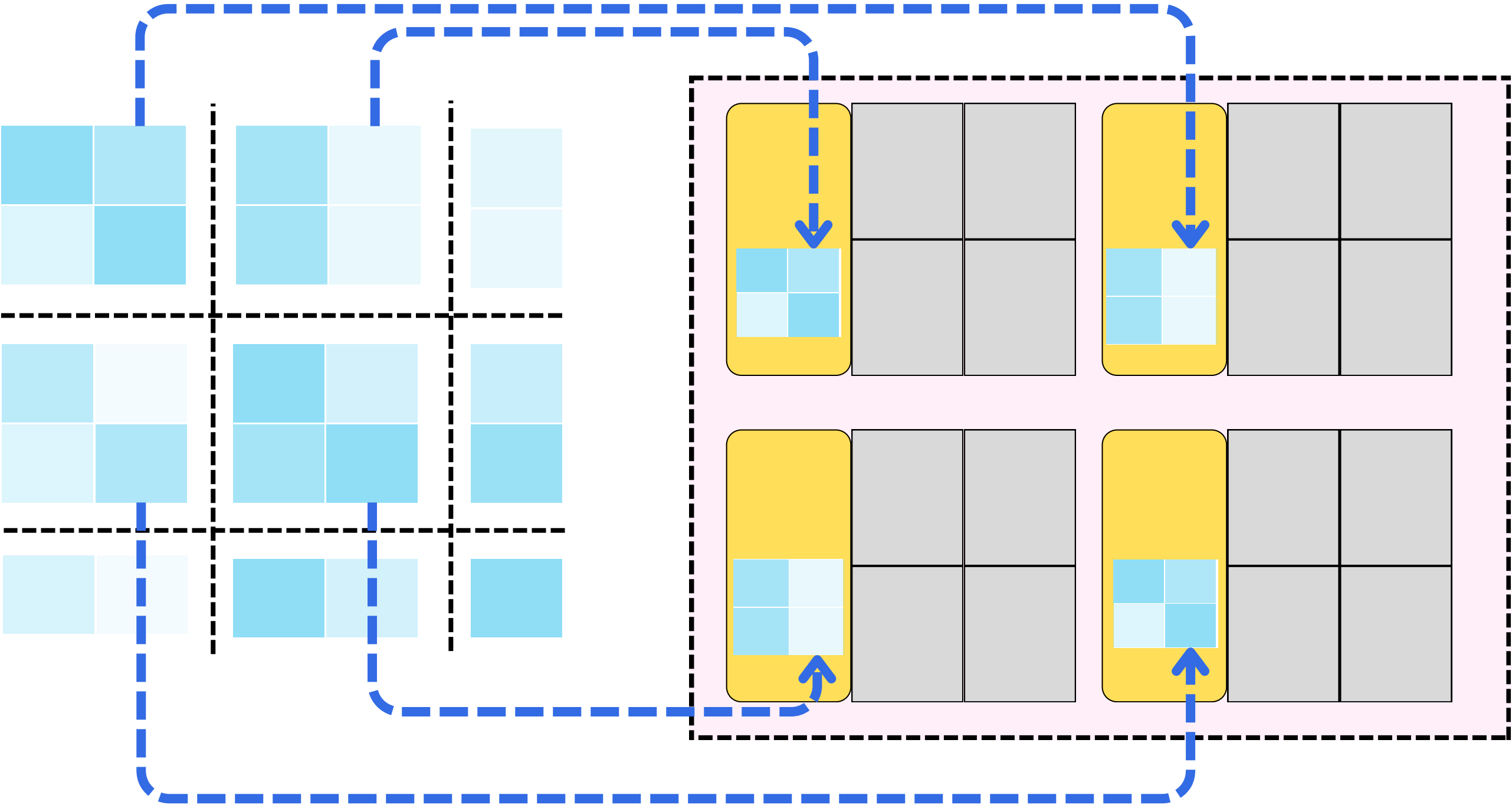




The GPU Architecture

Flash-Attention: tiling the matrix computations

Much faster retrieval of intermediate data



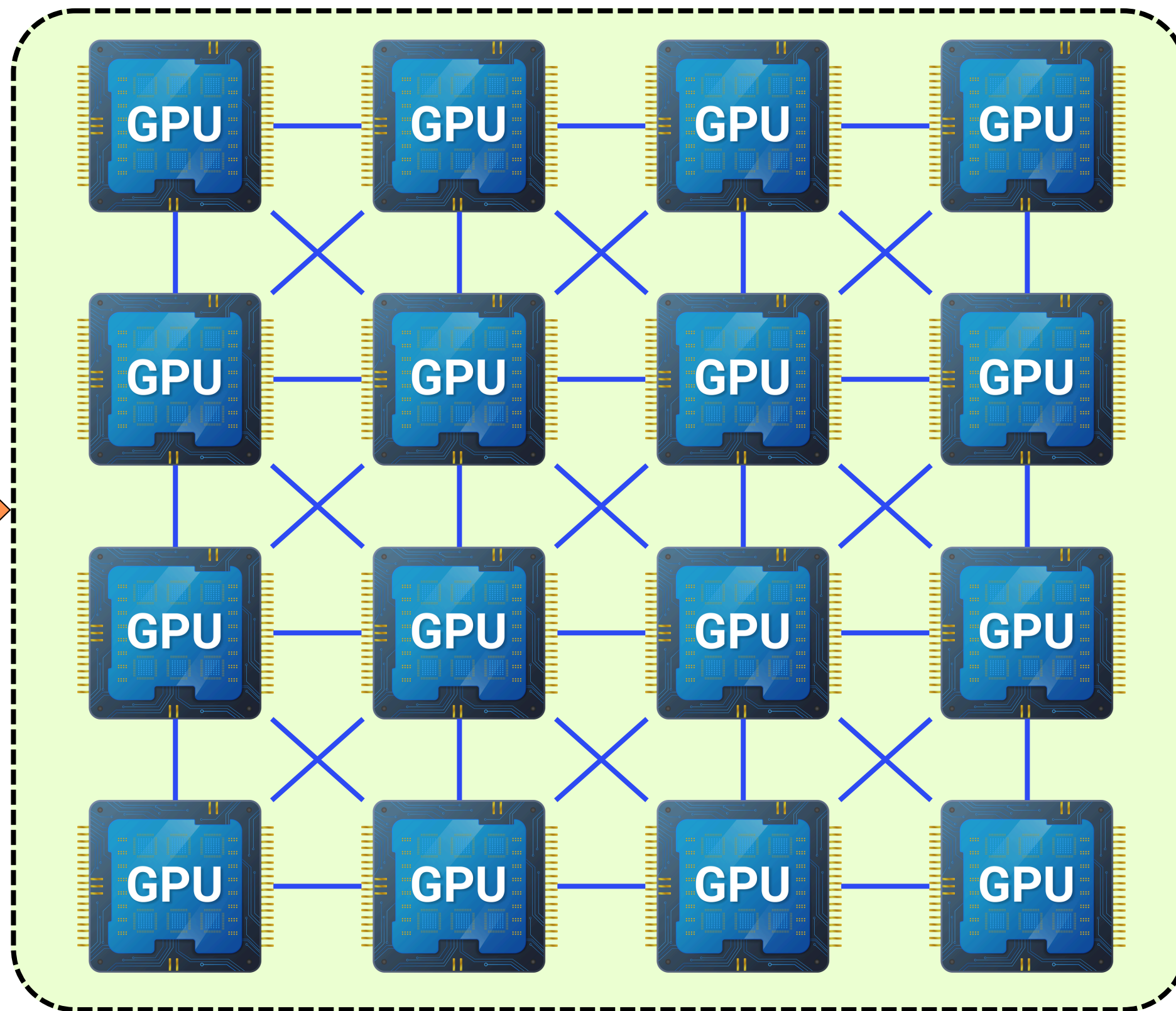
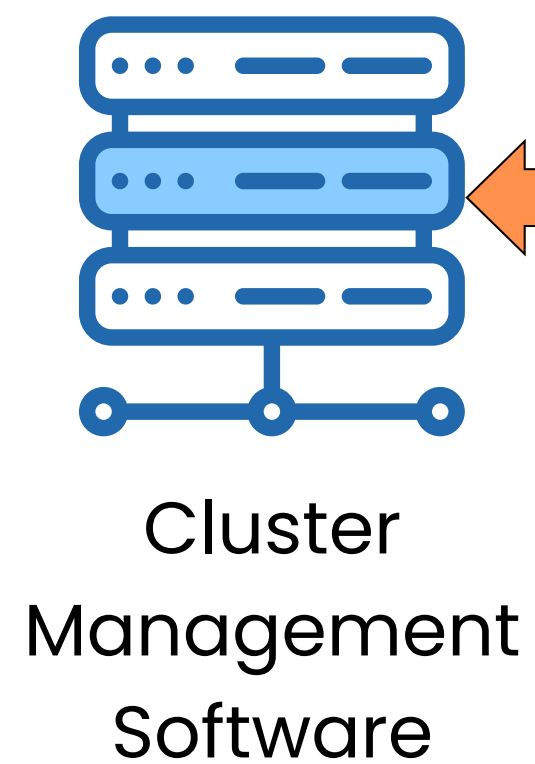


Parallelism with GPUs

- Data Parallelism (significantly speed up training)
- Model parallelism (fit very large models onto limited hardware)



GPU Clusters

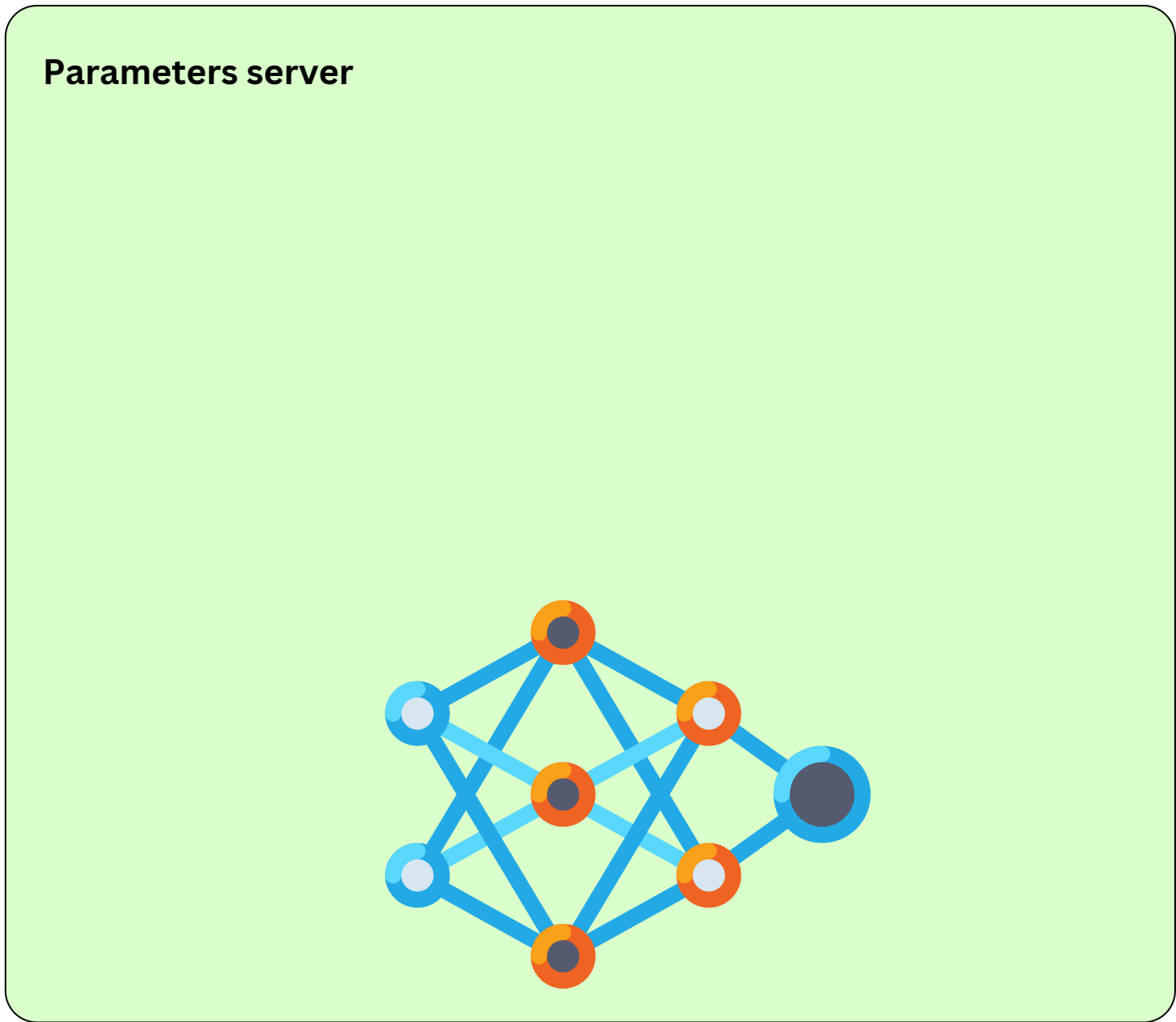
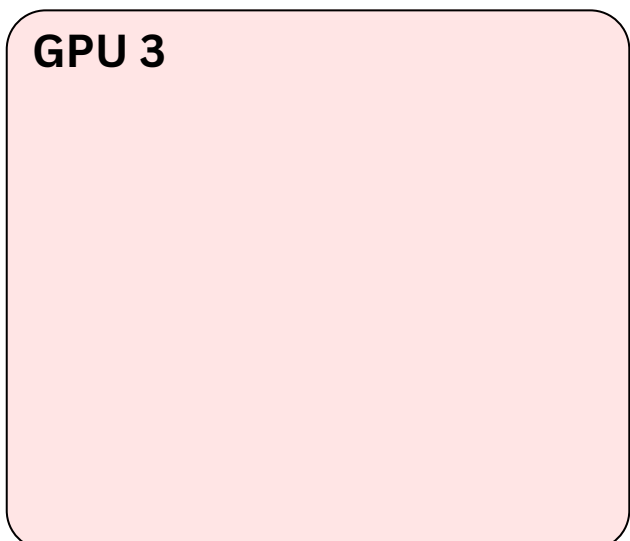
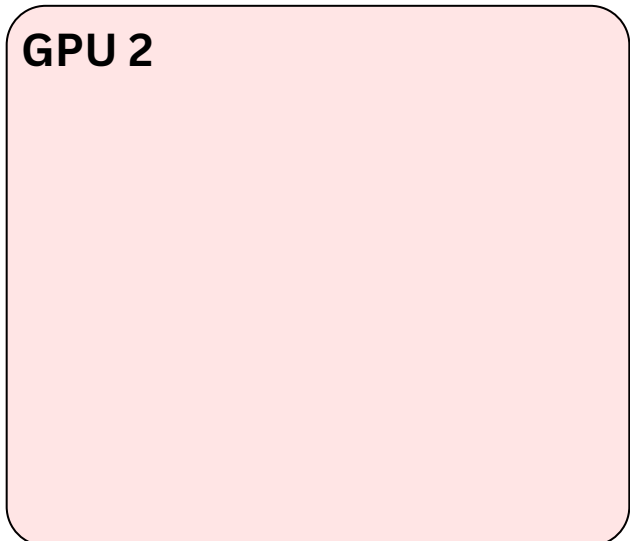
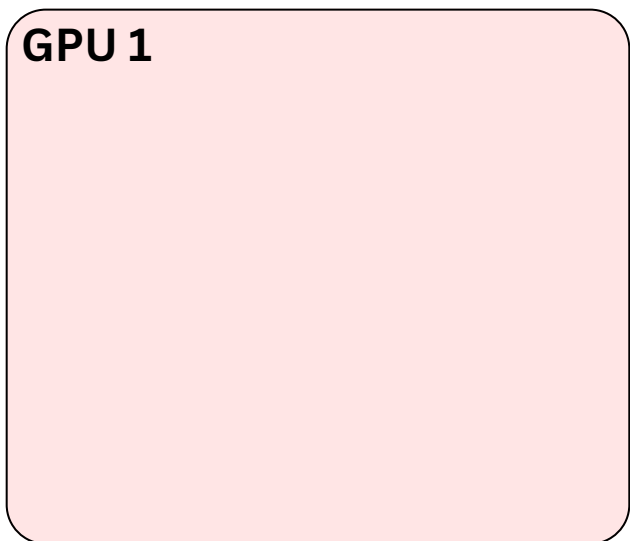
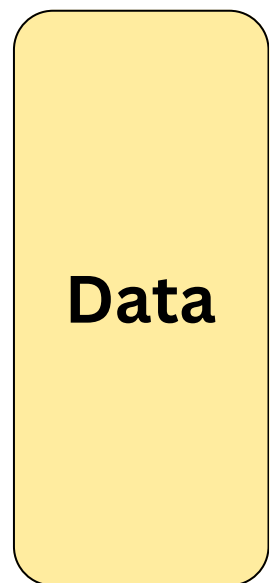


High-speed Interconnect:

- InfiniBand
- Ethernet
- NVLink
- PCIe

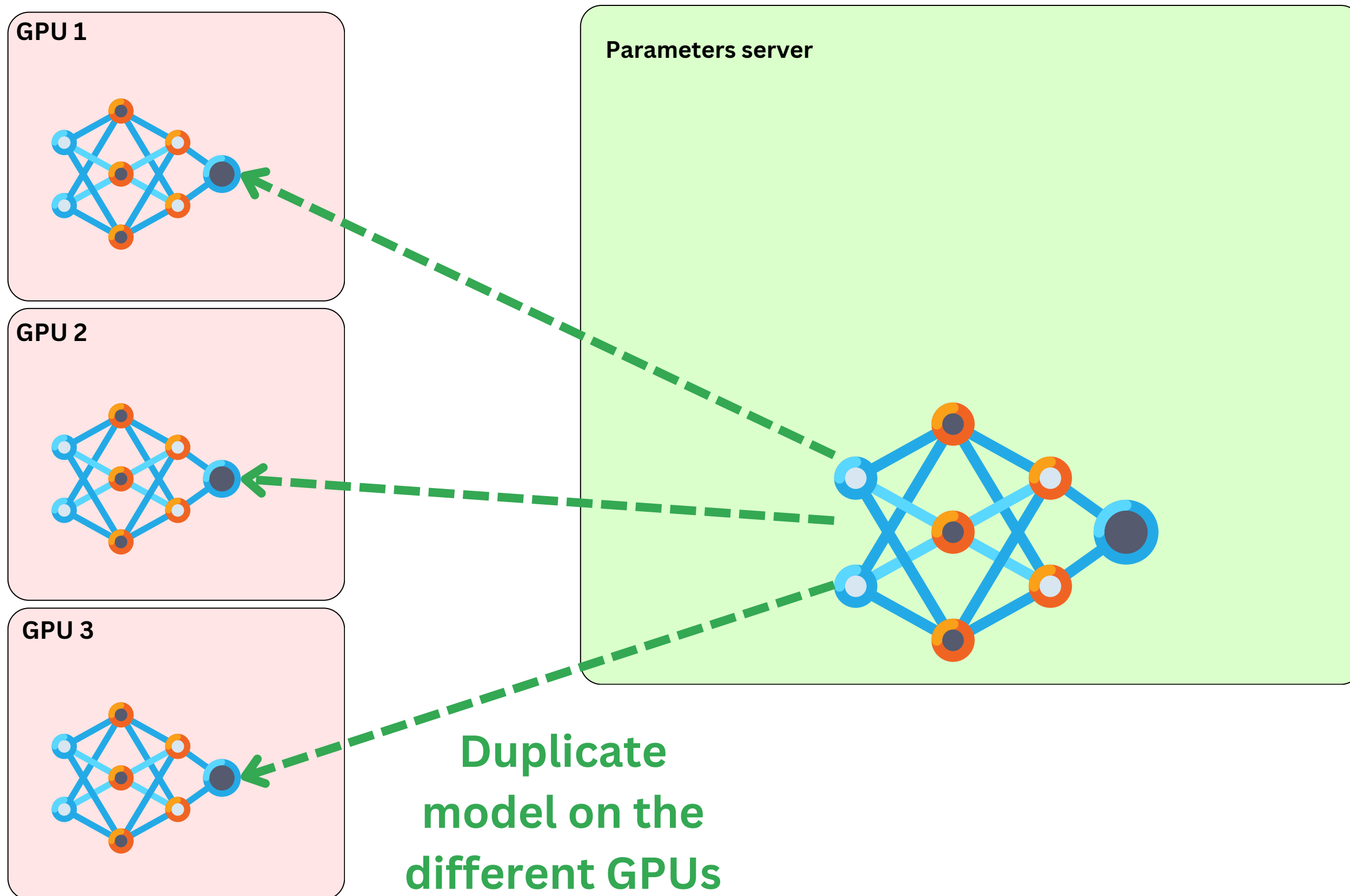
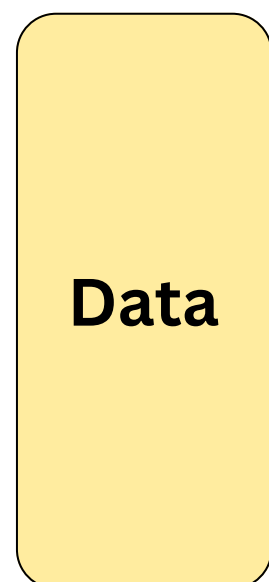


Data Parallelism



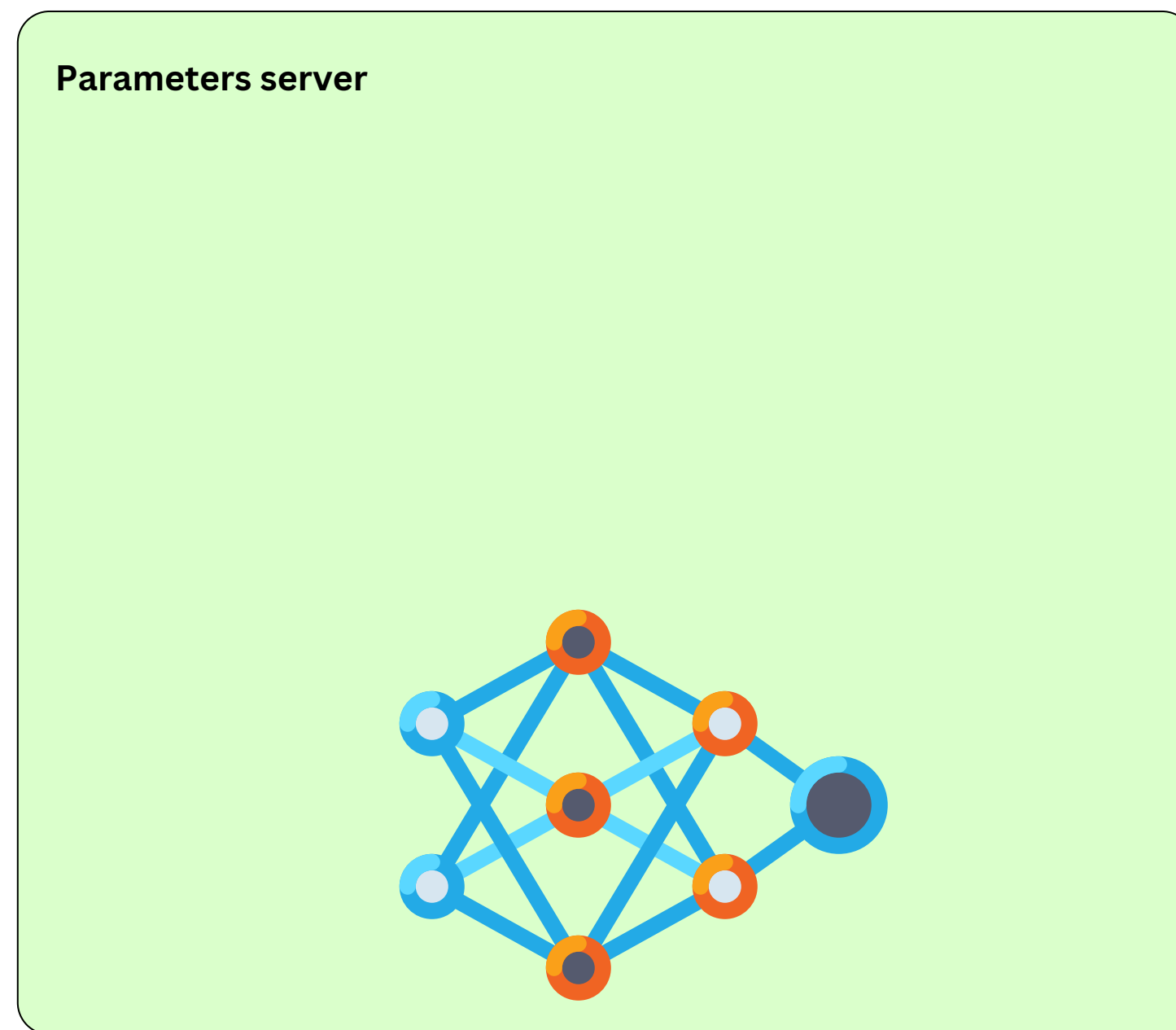
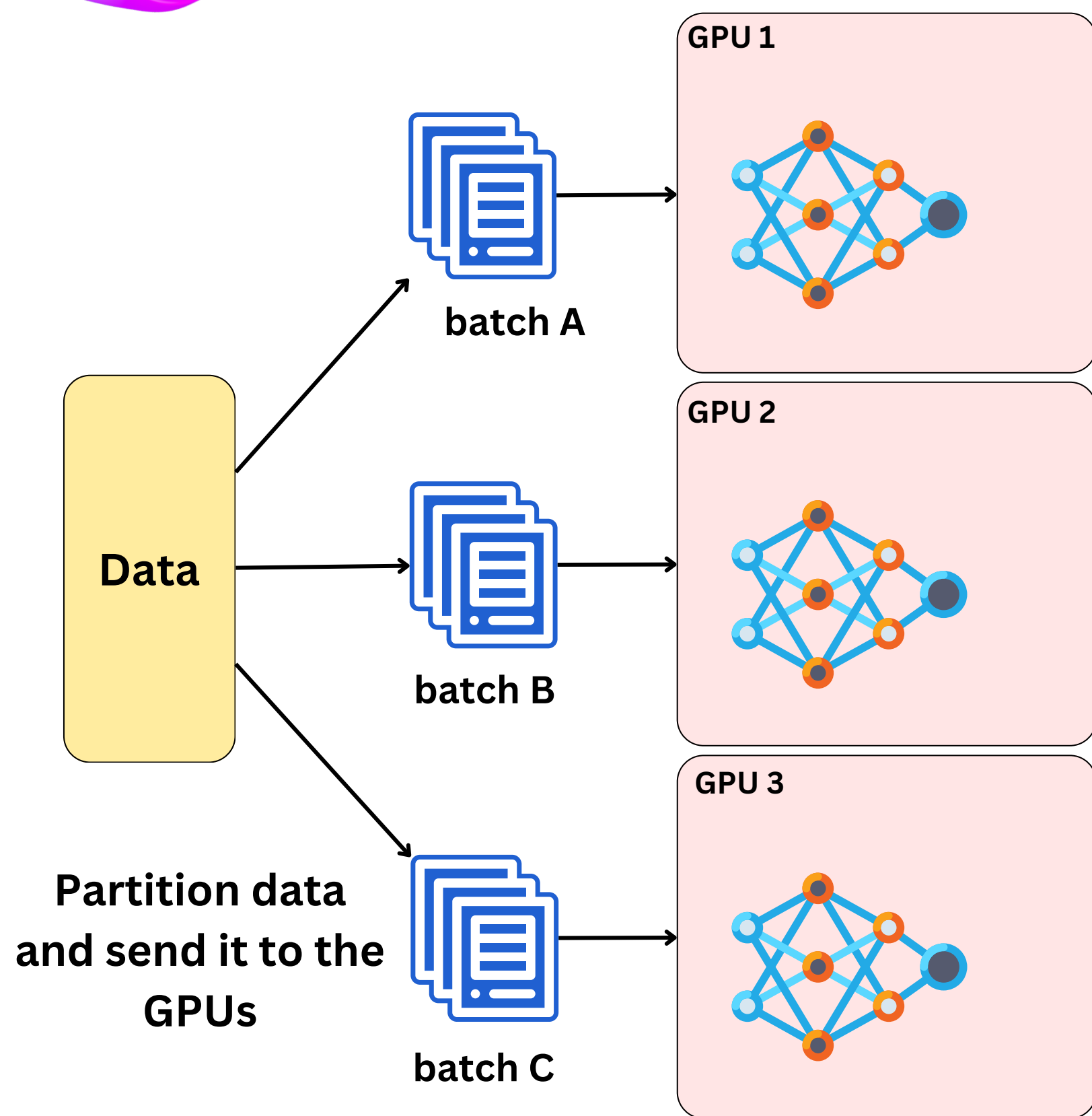


Data Parallelism



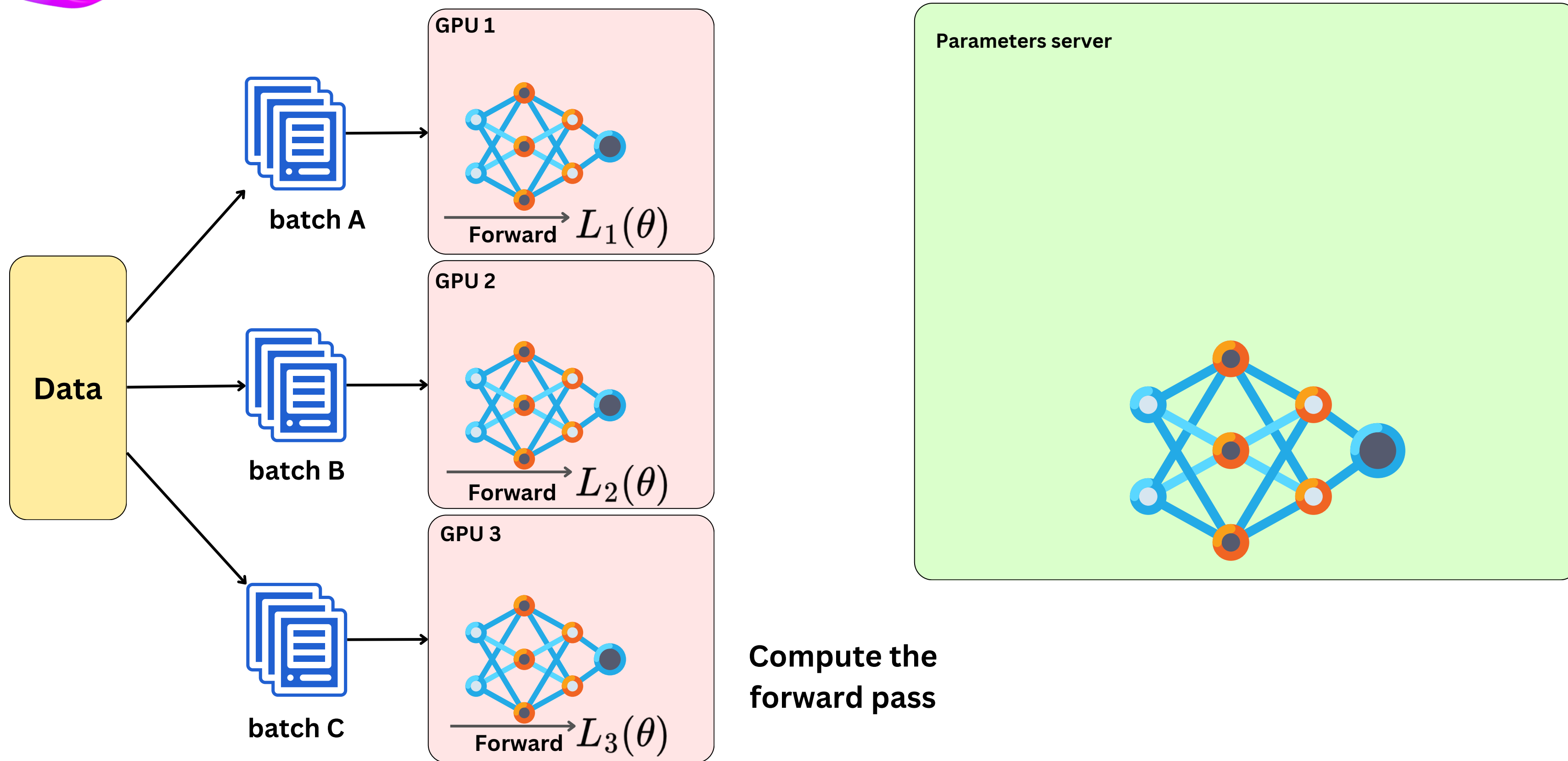


Data Parallelism



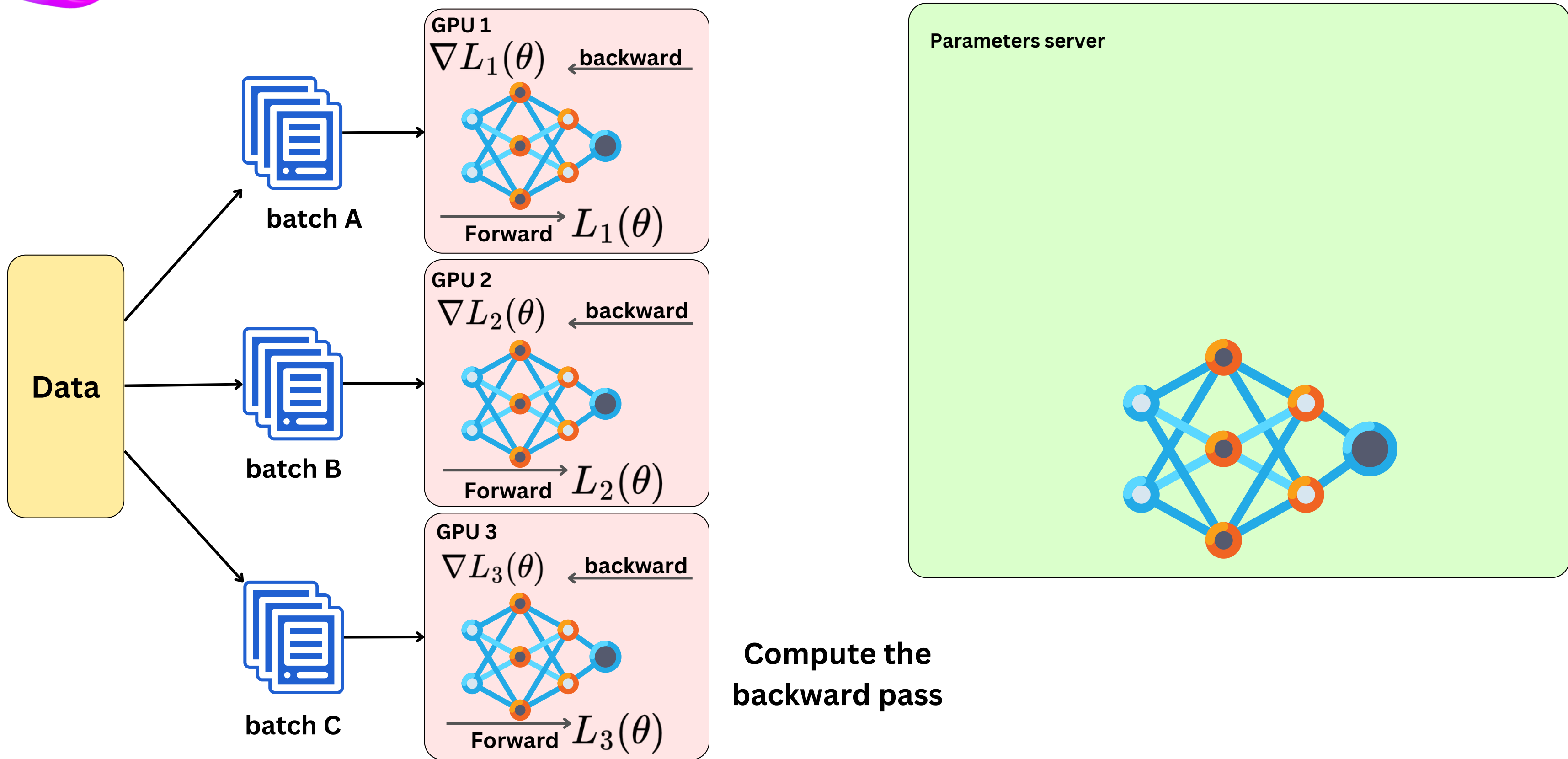


Data Parallelism



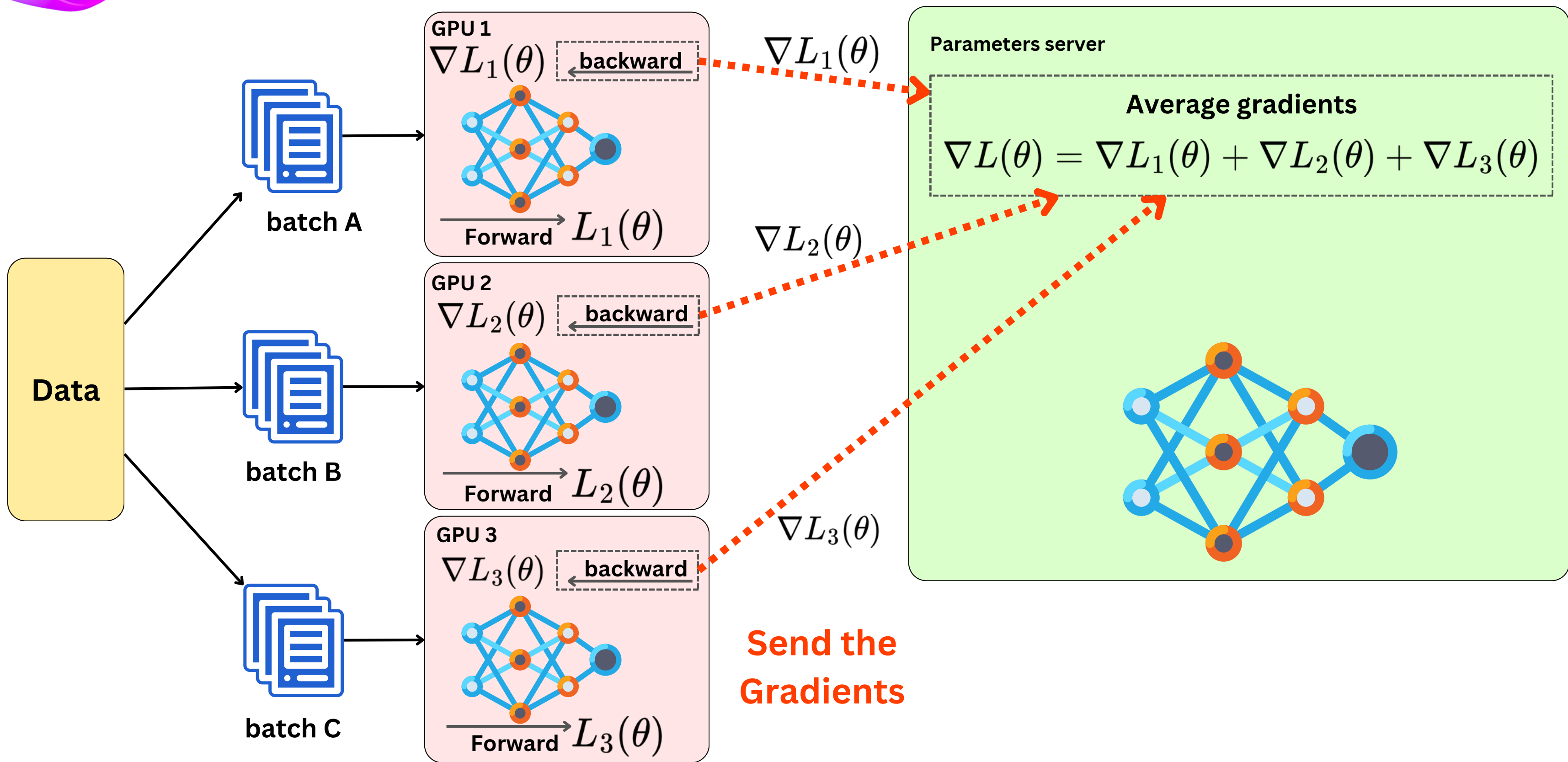


Data Parallelism



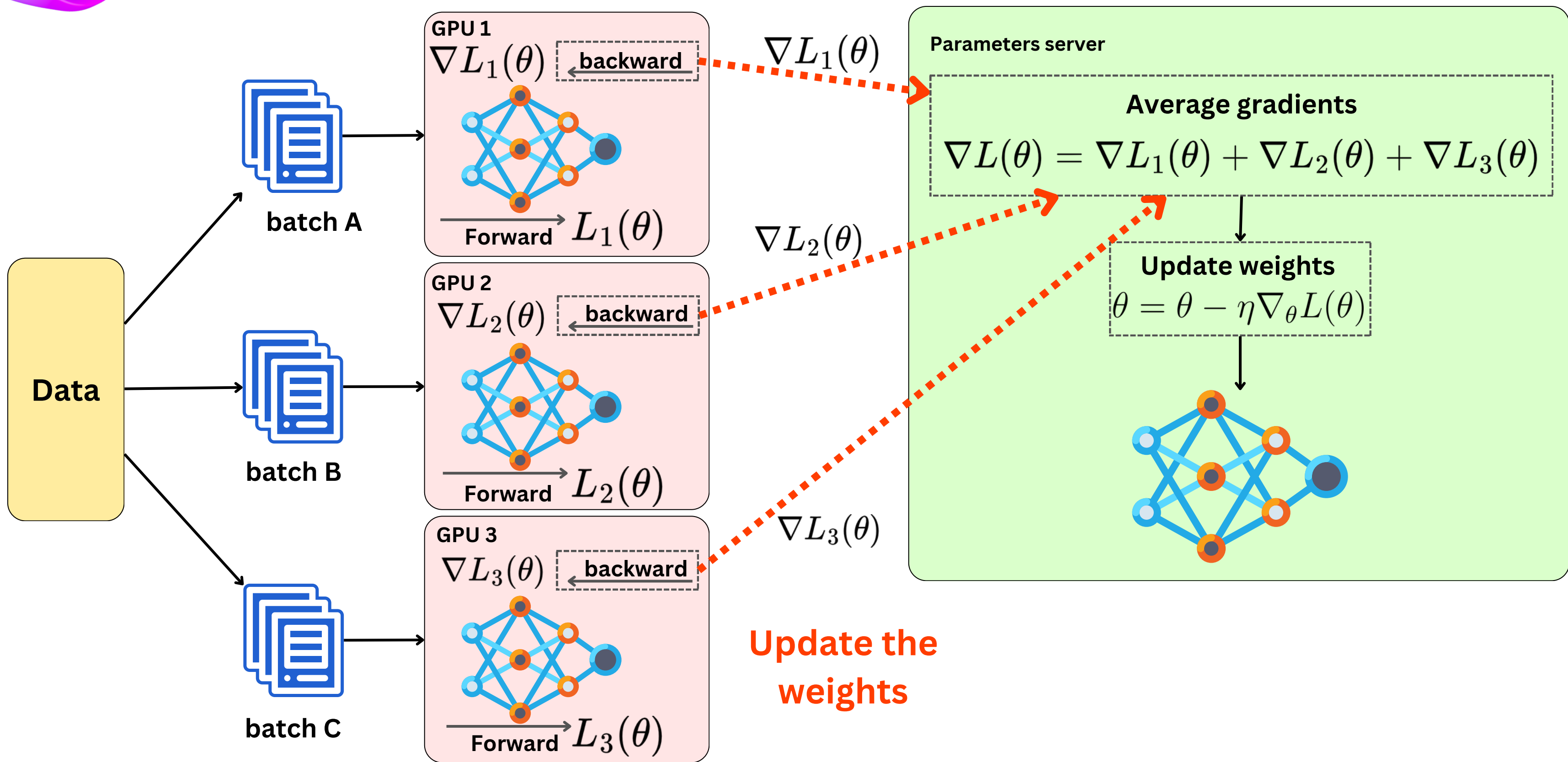


Data Parallelism



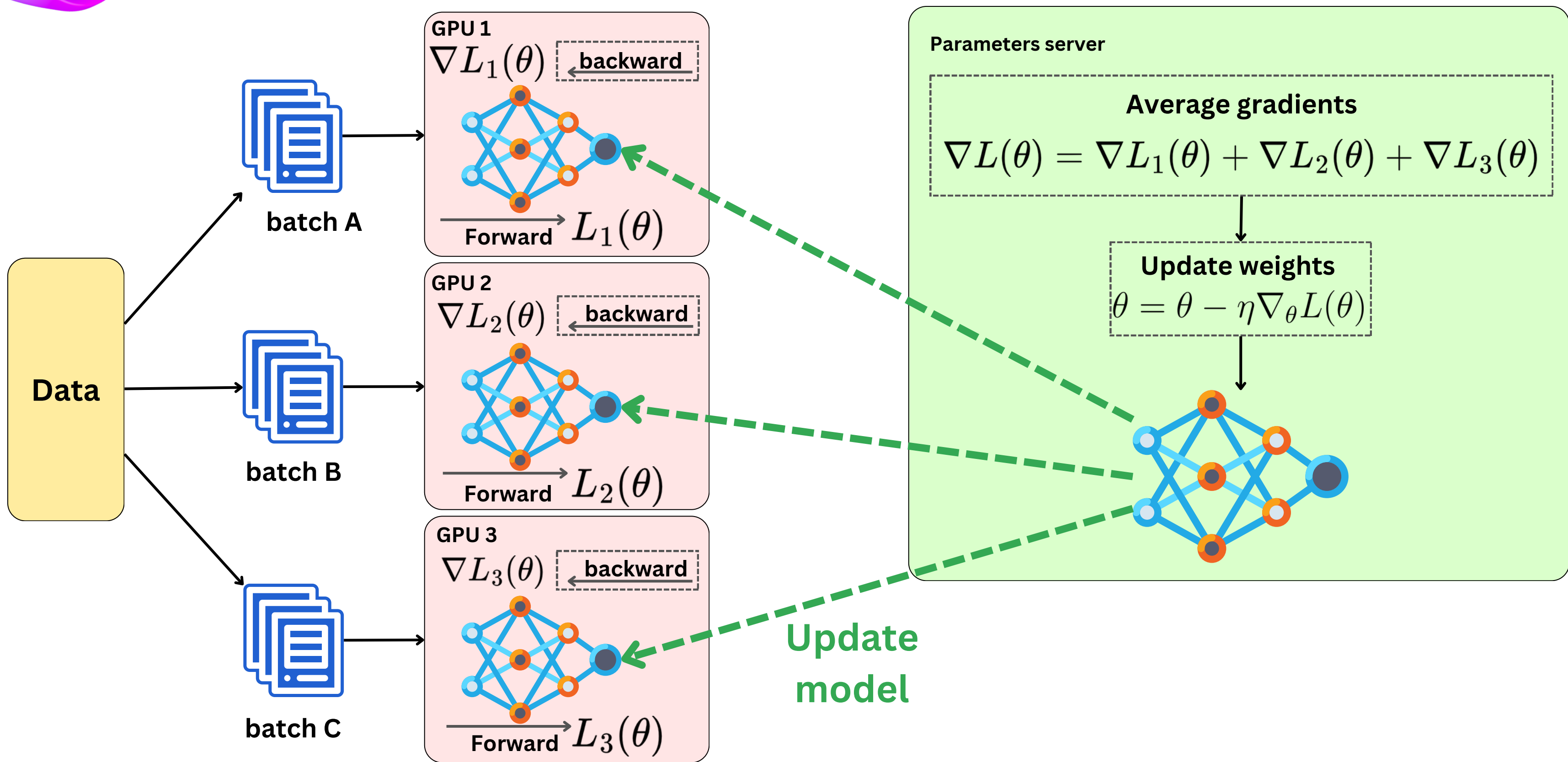


Data Parallelism



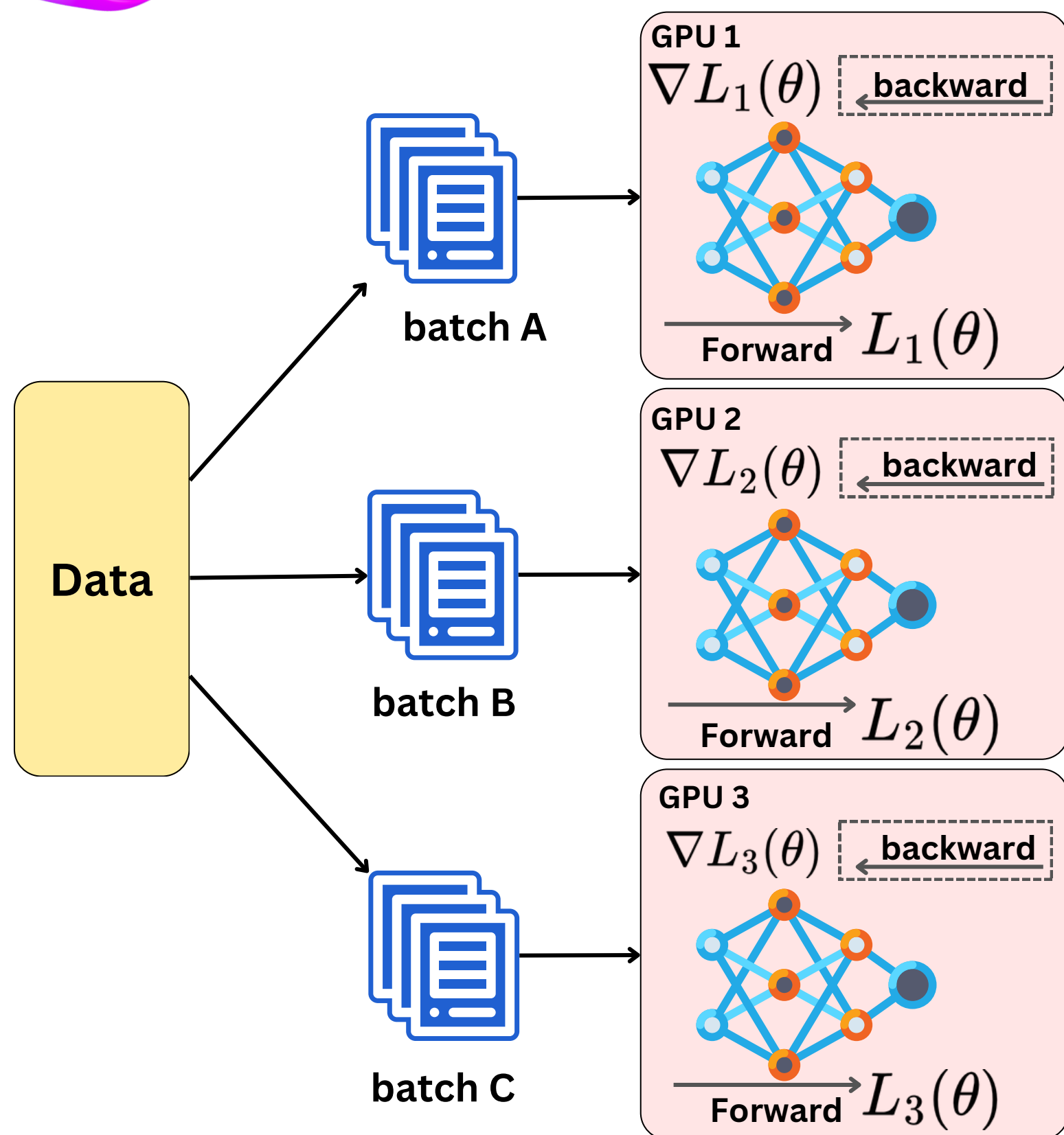


Data Parallelism

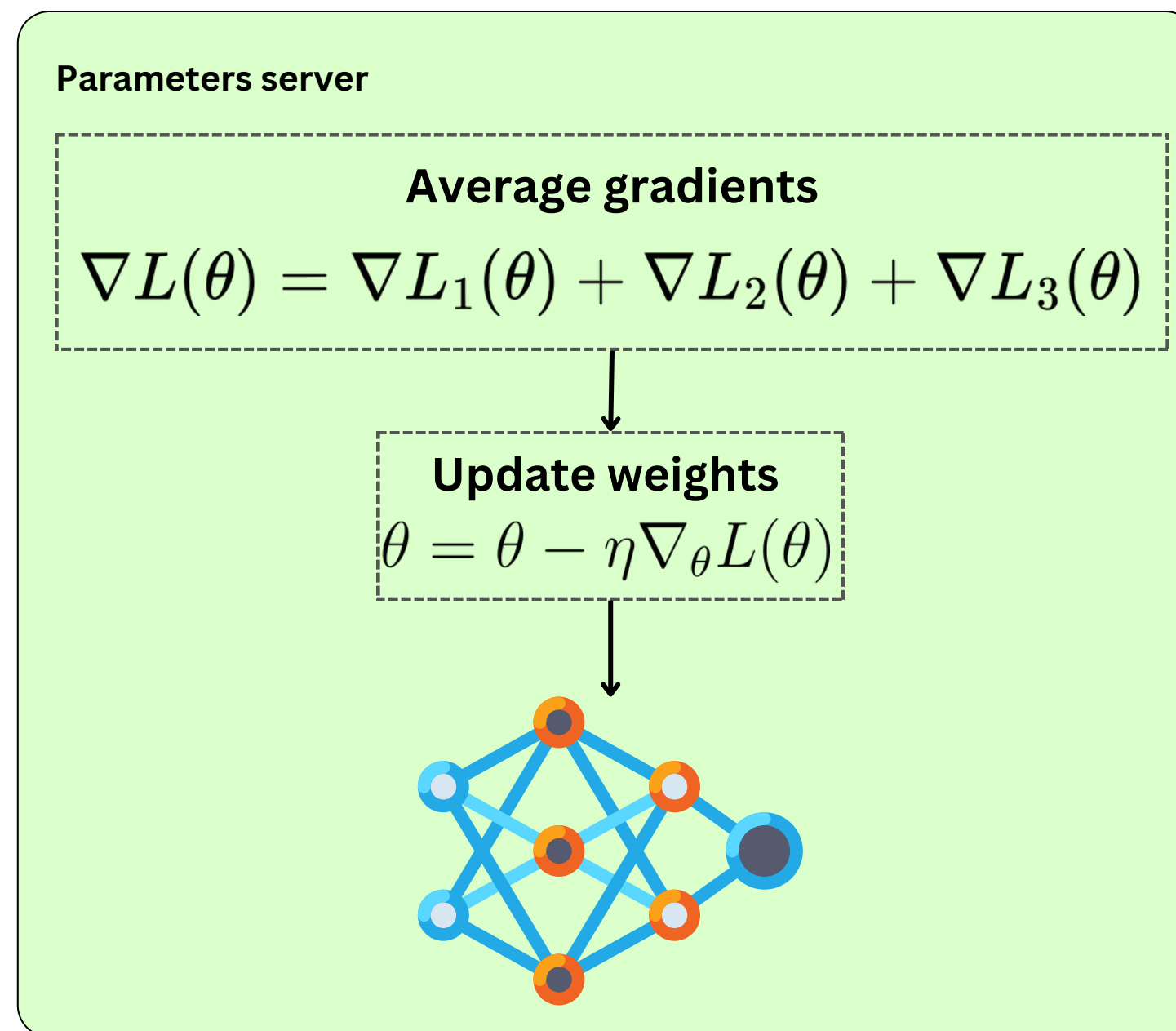




Data Parallelism

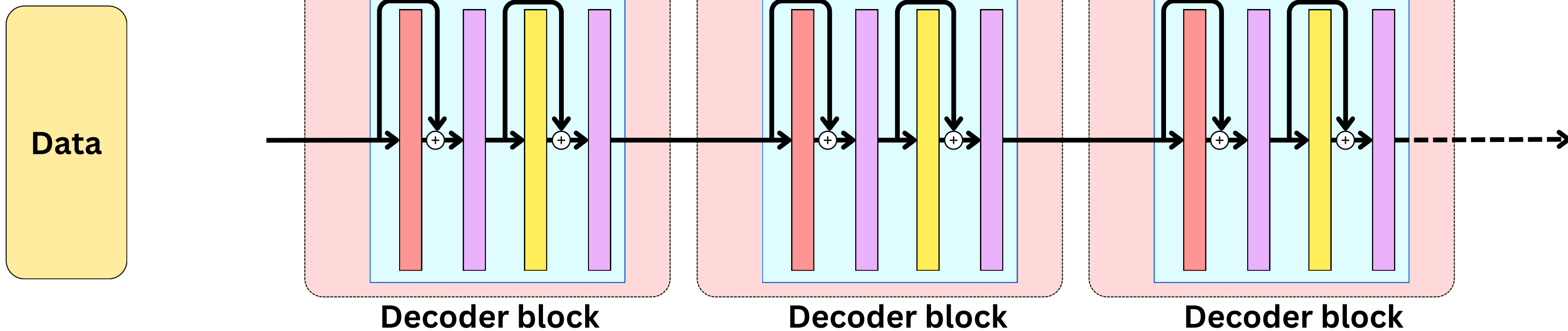


Centralized synchronous data parallelism



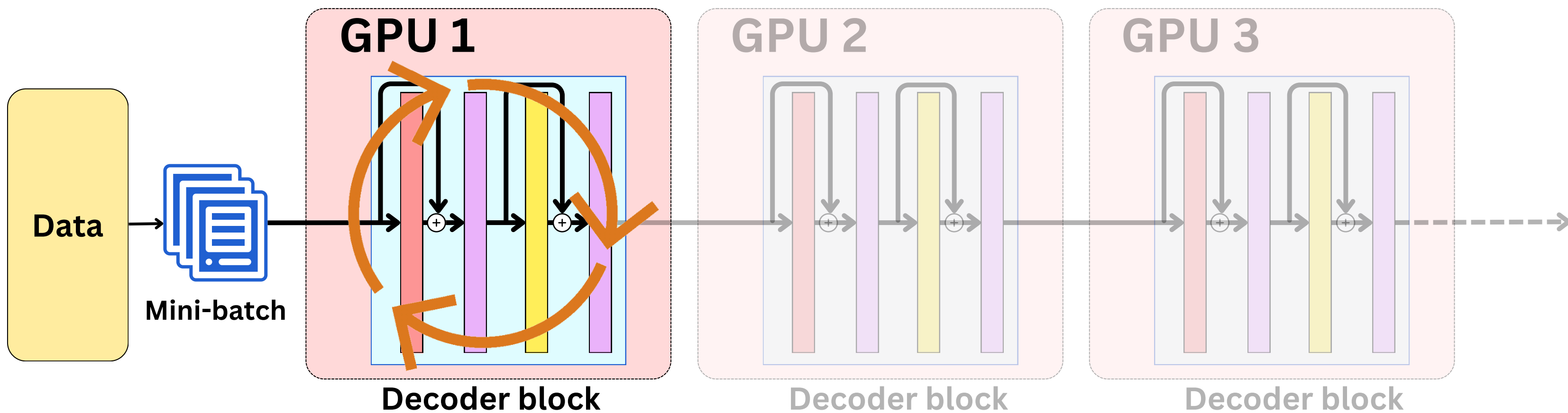


Naive Model Parallelism



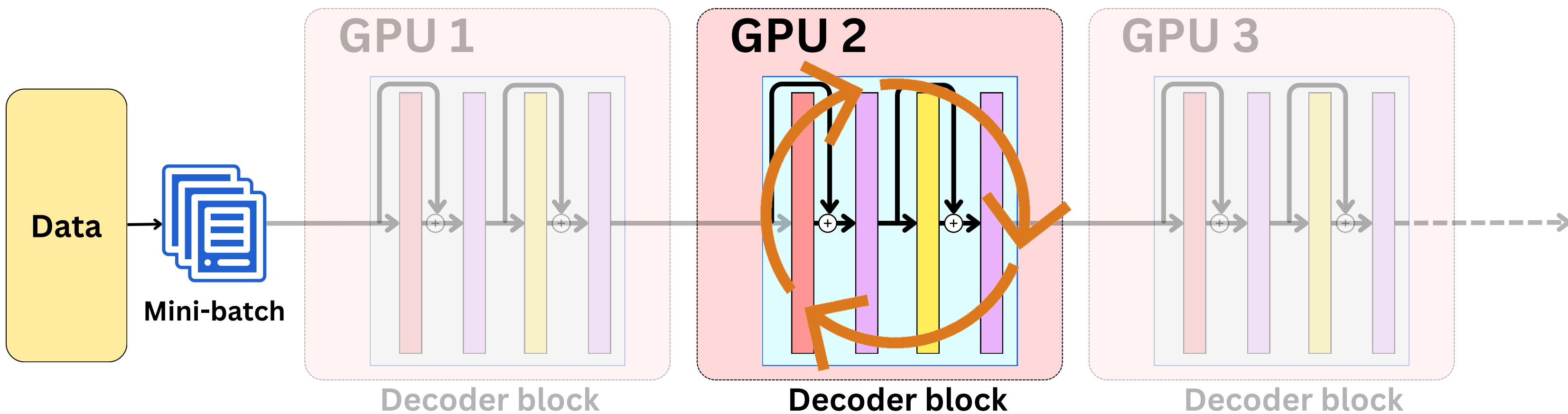


Naive Model Parallelism



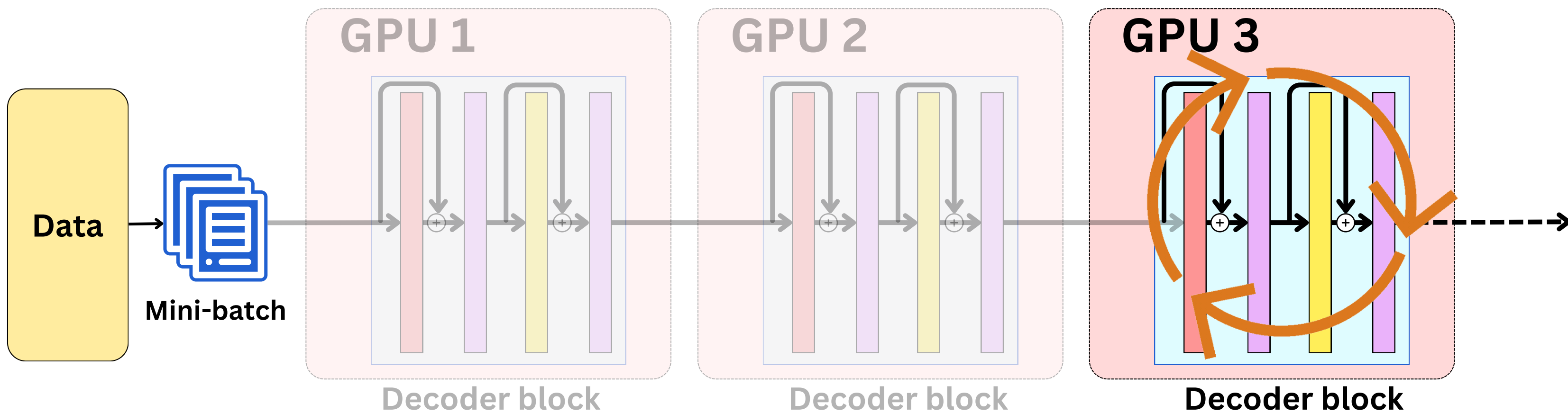


Naive Model Parallelism



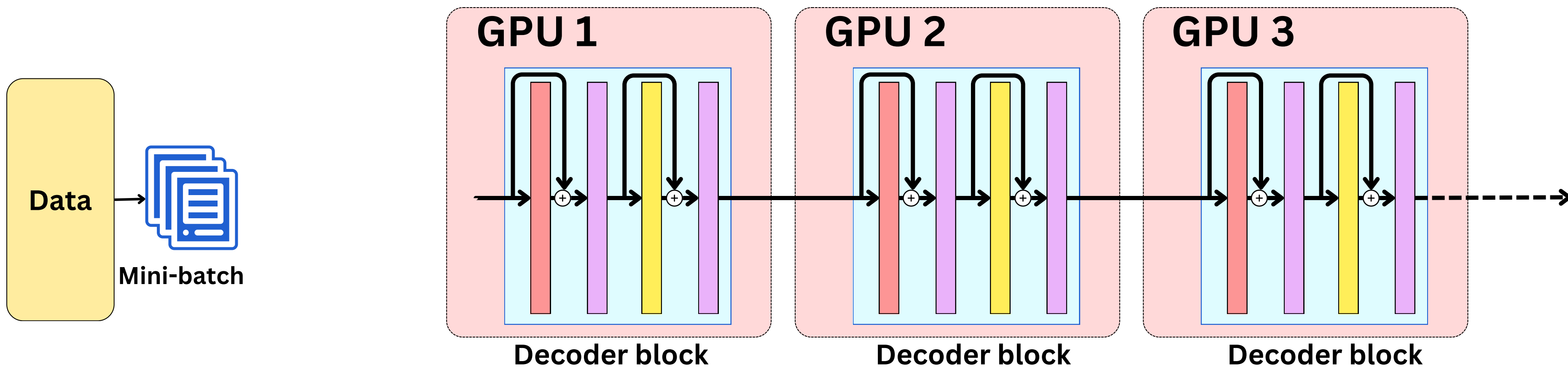


Naive Model Parallelism



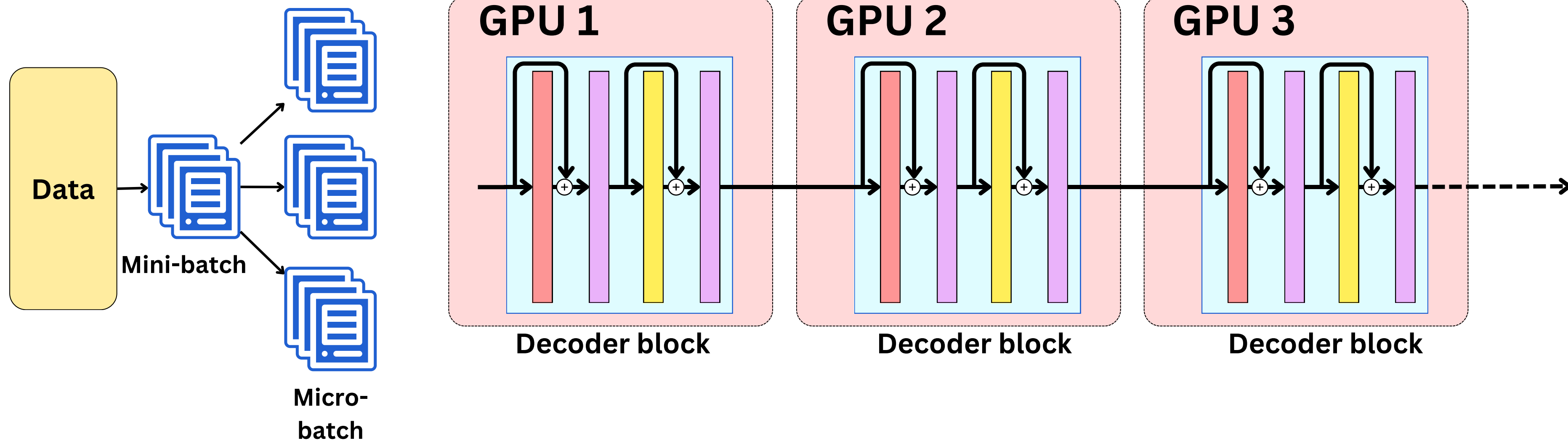


Pipeline Parallelism



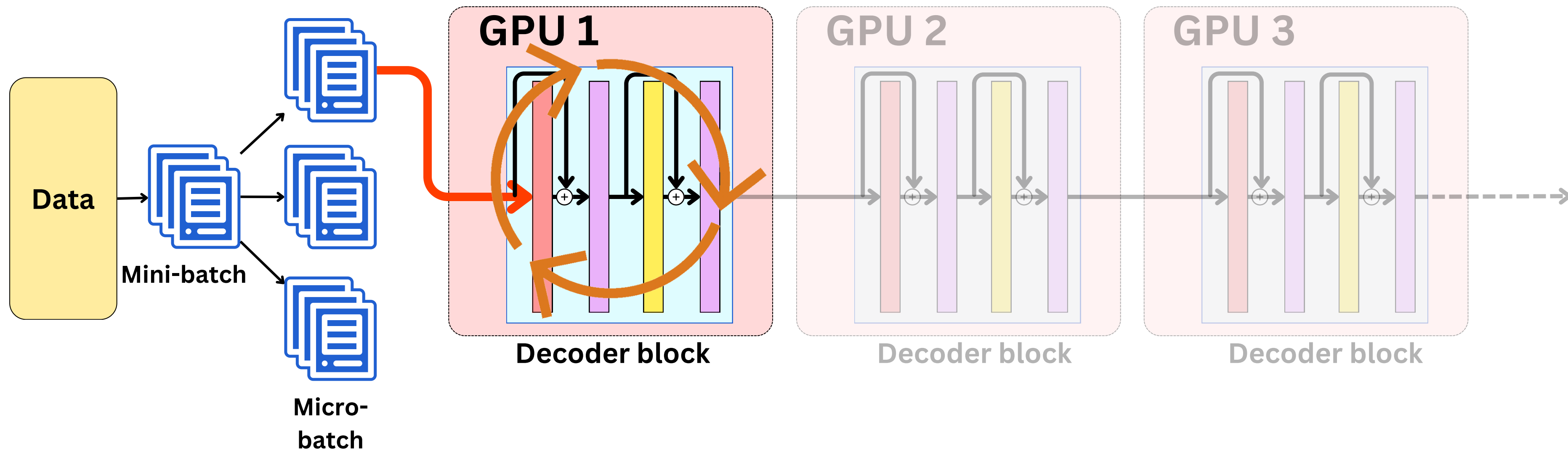


Pipeline Parallelism



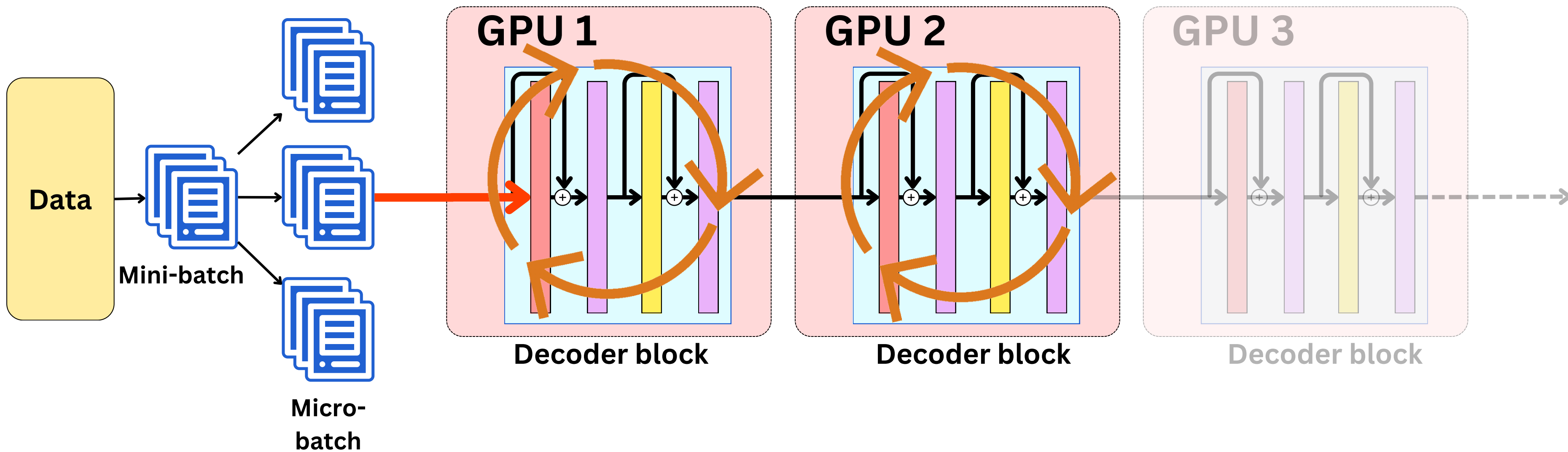


Pipeline Parallelism



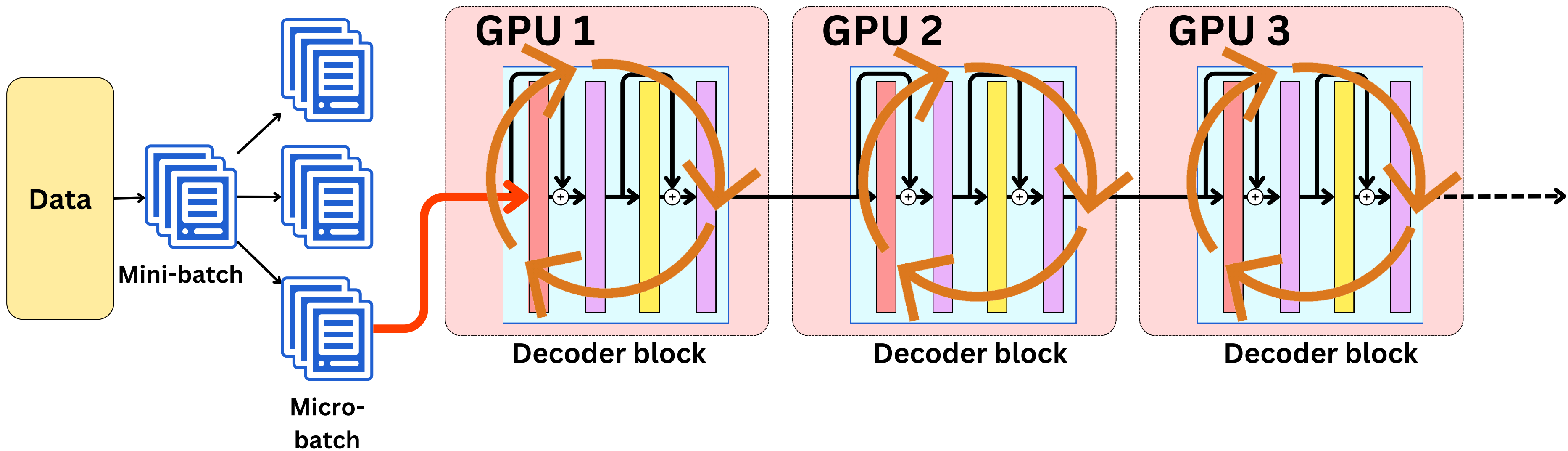


Pipeline Parallelism



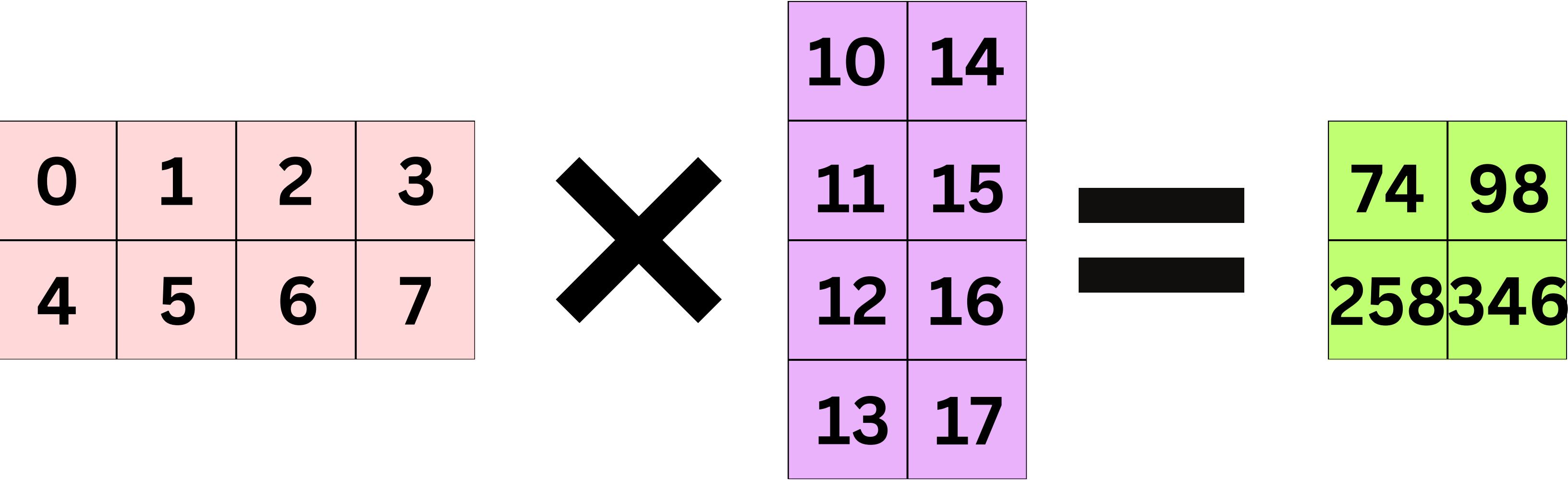


Pipeline Parallelism





Tensor Parallelism





Tensor Parallelism

Column parallelism

0	1	2	3
4	5	6	7

×

10	14
11	15
12	16
13	17

GPU 1

0	1	2	3
4	5	6	7

×

10
11
12
13

=

74
258

GPU 2

0	1	2	3
4	5	6	7

×

14
15
16
17

=

98
346



Tensor Parallelism

Column parallelism

0	1	2	3
4	5	6	7

×

10	14
11	15
12	16
13	17

GPU 1

0	1	2	3
4	5	6	7

×

10
11
12
13

=

74
258

GPU 2

0	1	2	3
4	5	6	7

×

14
15
16
17

=

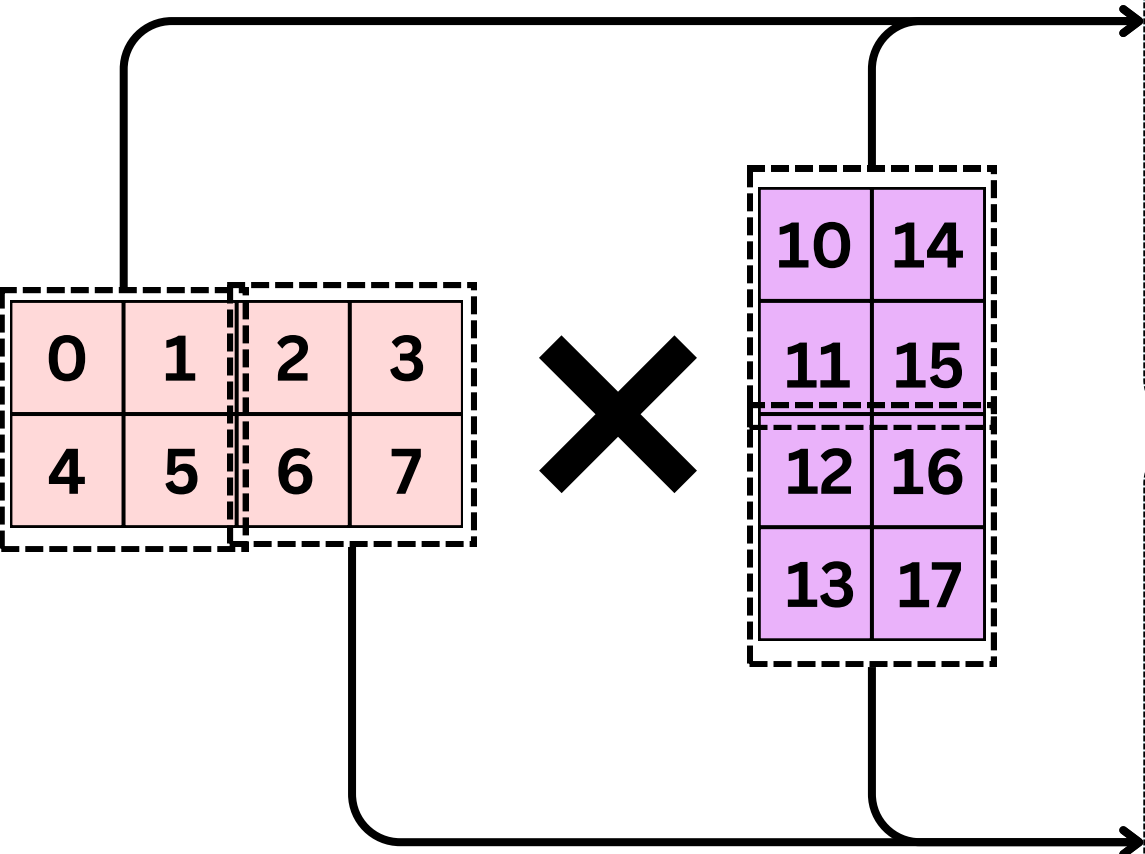
98
346

74	98
258	346



Tensor Parallelism

Row parallelism



GPU 1

$$\begin{bmatrix} 0 & 1 \\ 4 & 5 \end{bmatrix} \times \begin{bmatrix} 10 & 14 \\ 11 & 15 \end{bmatrix} = \begin{bmatrix} 11 & 15 \\ 95 & 131 \end{bmatrix}$$

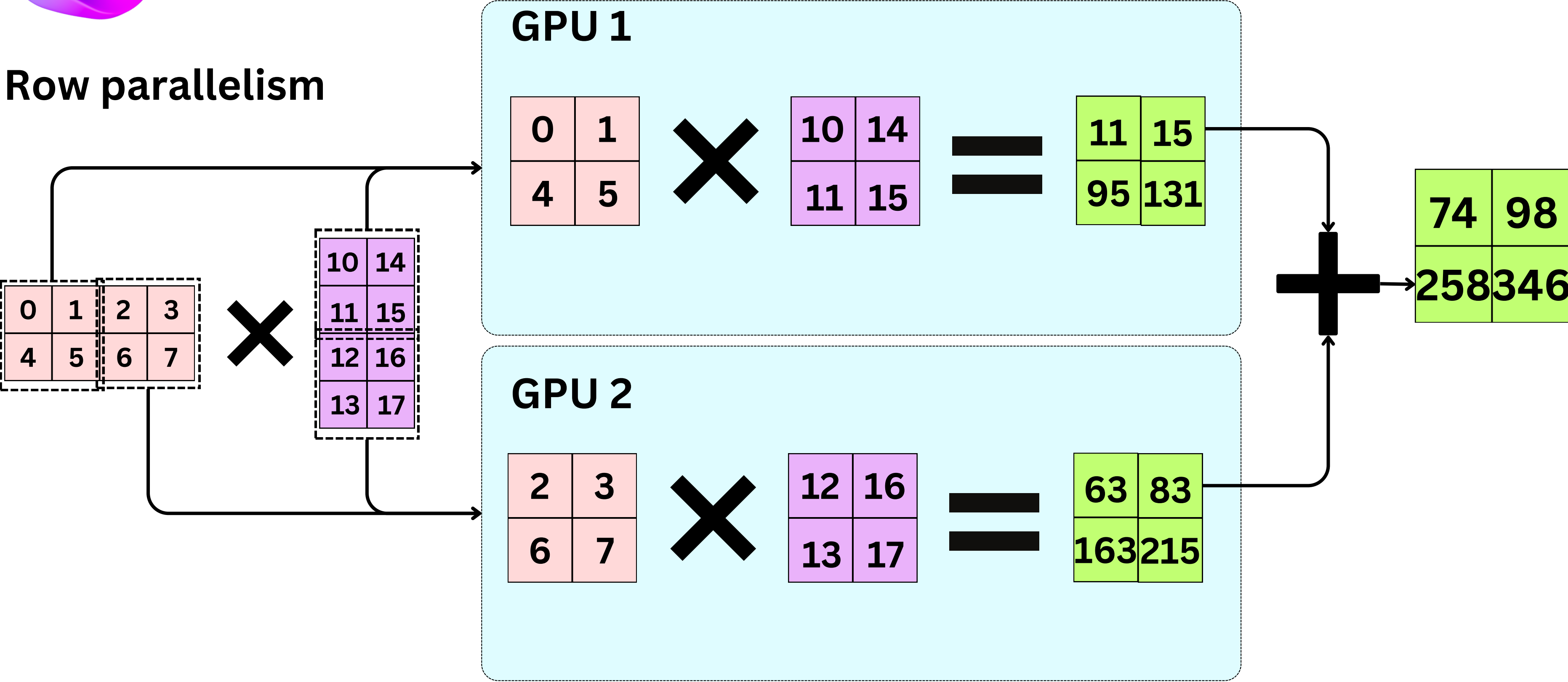
GPU 2

$$\begin{bmatrix} 2 & 3 \\ 6 & 7 \end{bmatrix} \times \begin{bmatrix} 12 & 16 \\ 13 & 17 \end{bmatrix} = \begin{bmatrix} 63 & 83 \\ 163 & 215 \end{bmatrix}$$



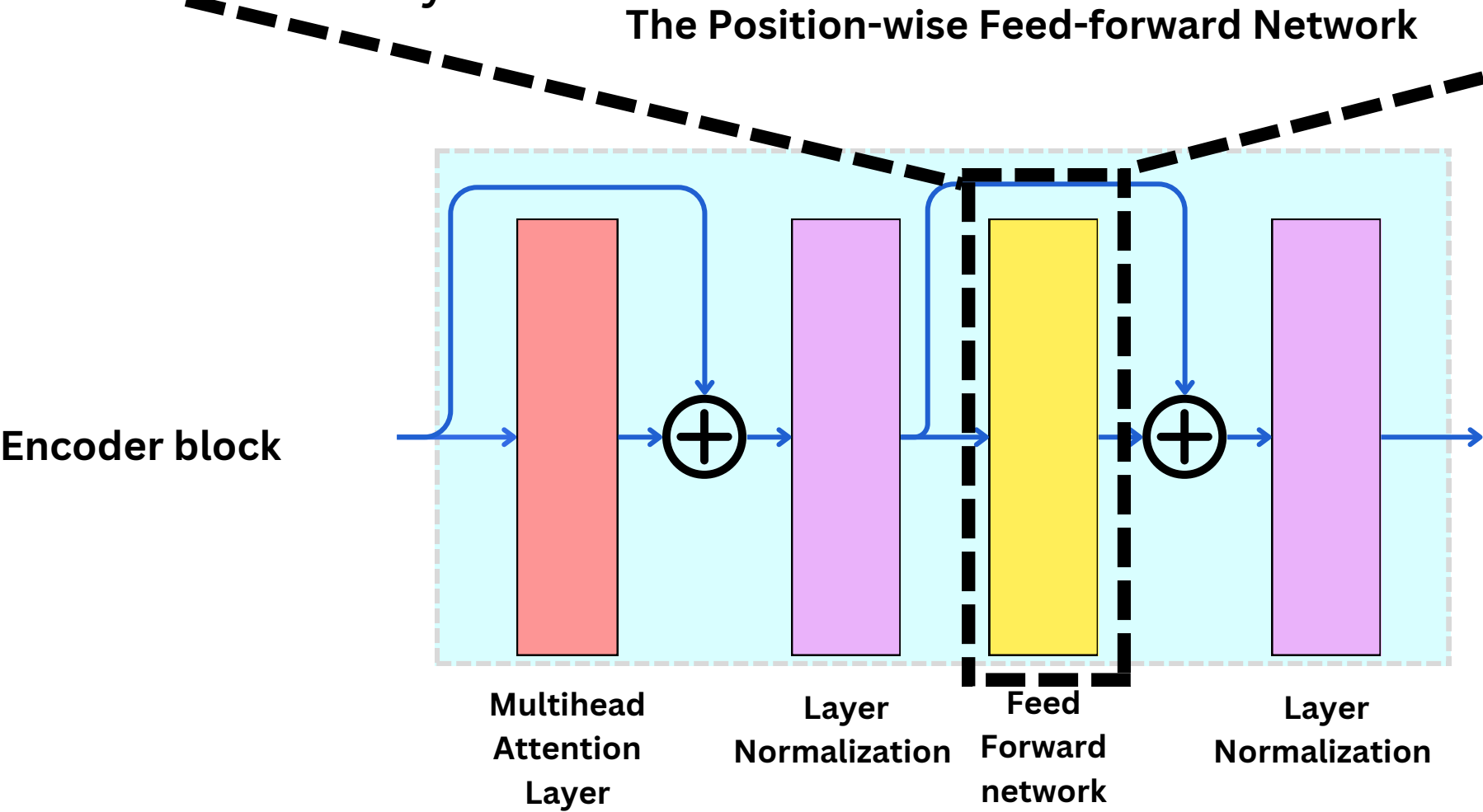
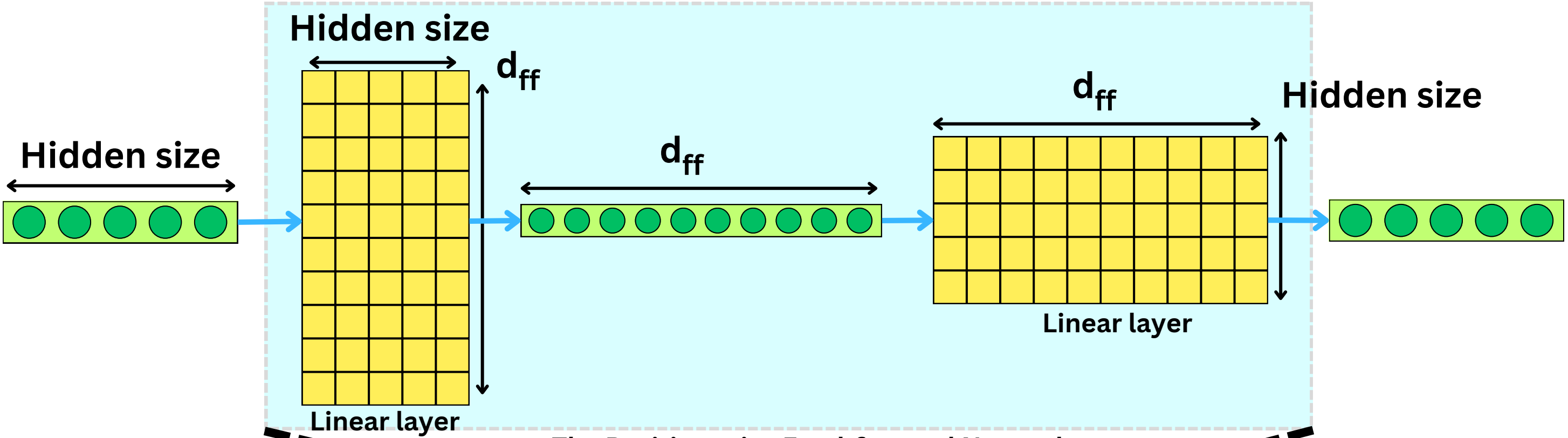
Tensor Parallelism

Row parallelism



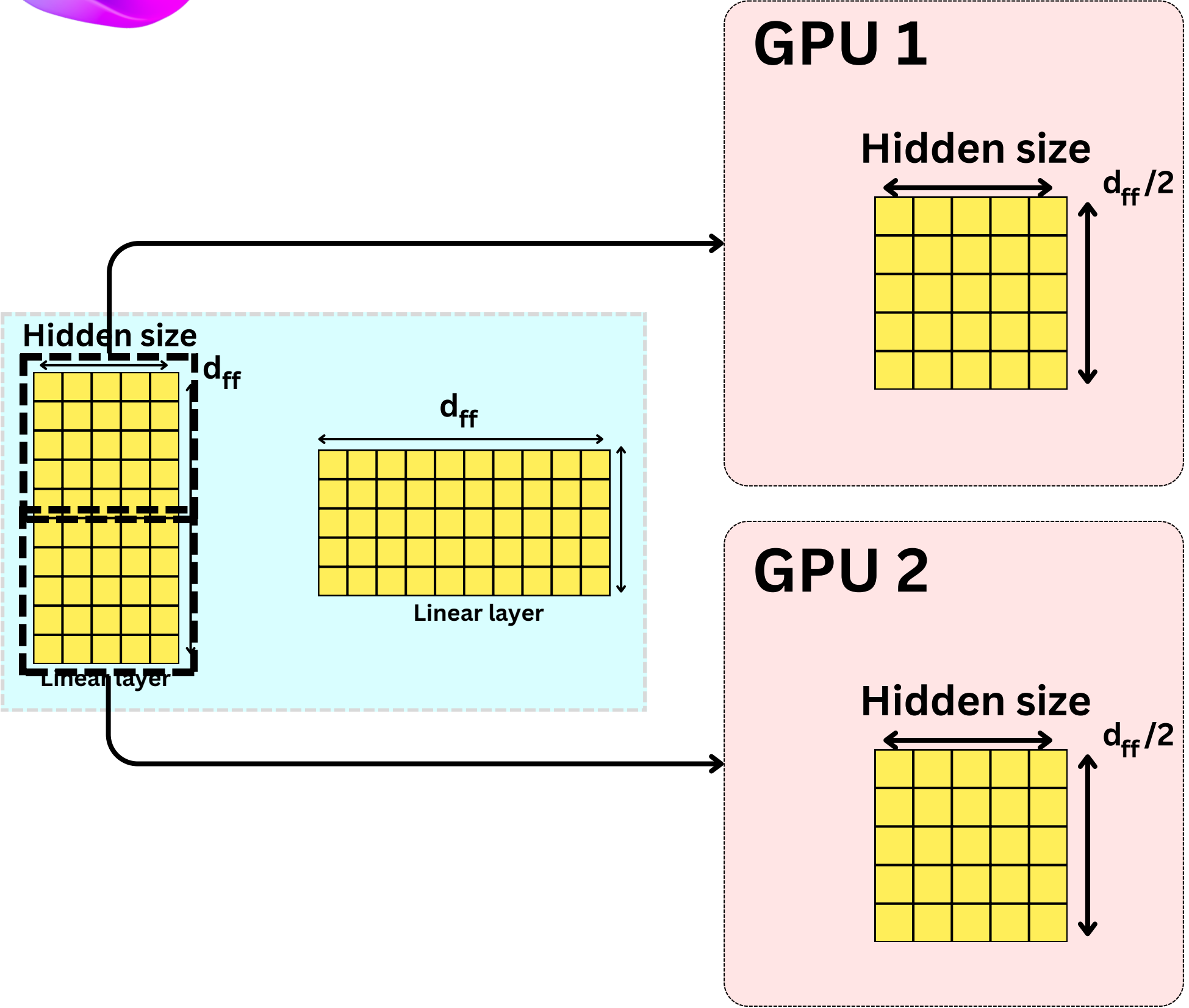


Tensor Parallelism



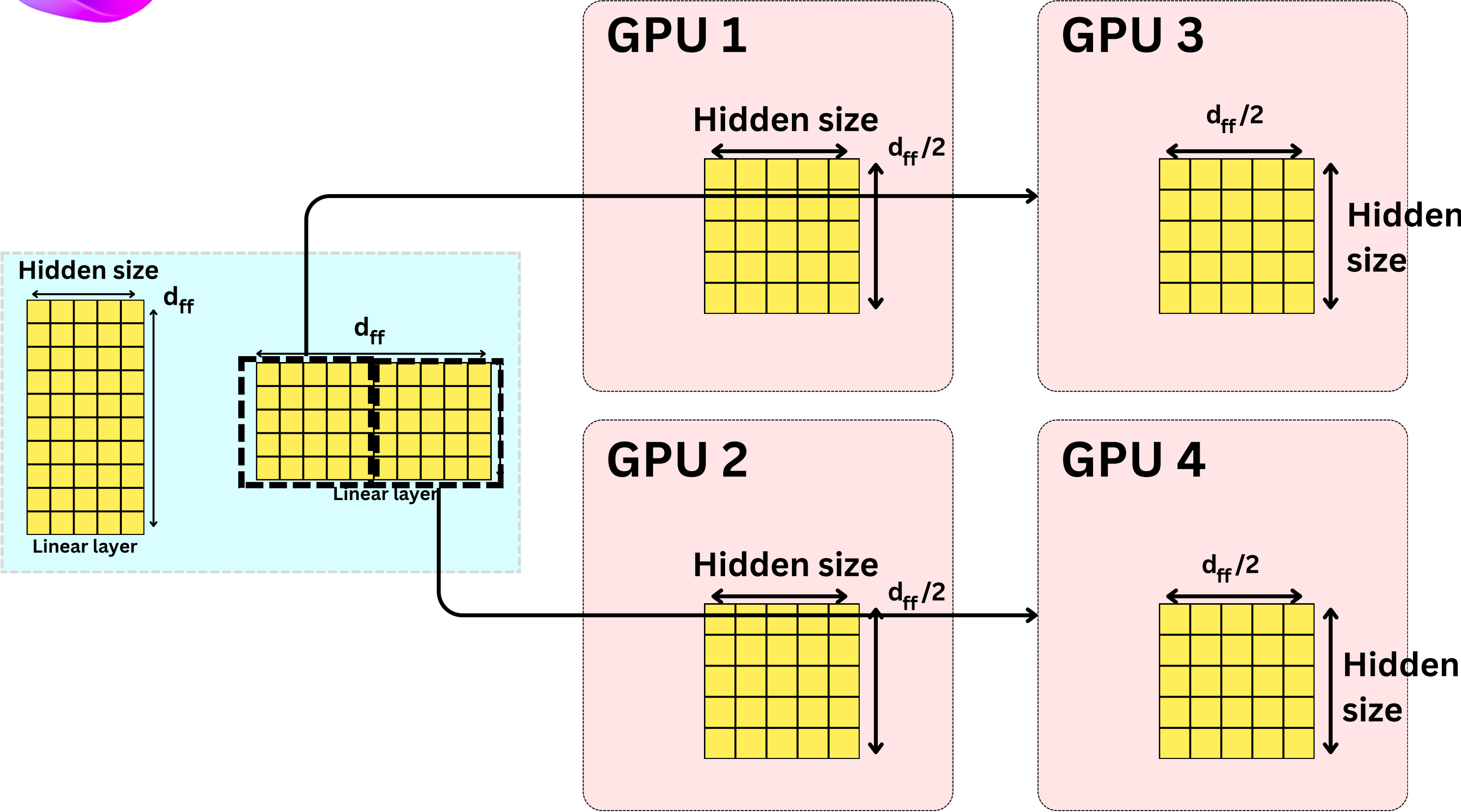


Tensor Parallelism



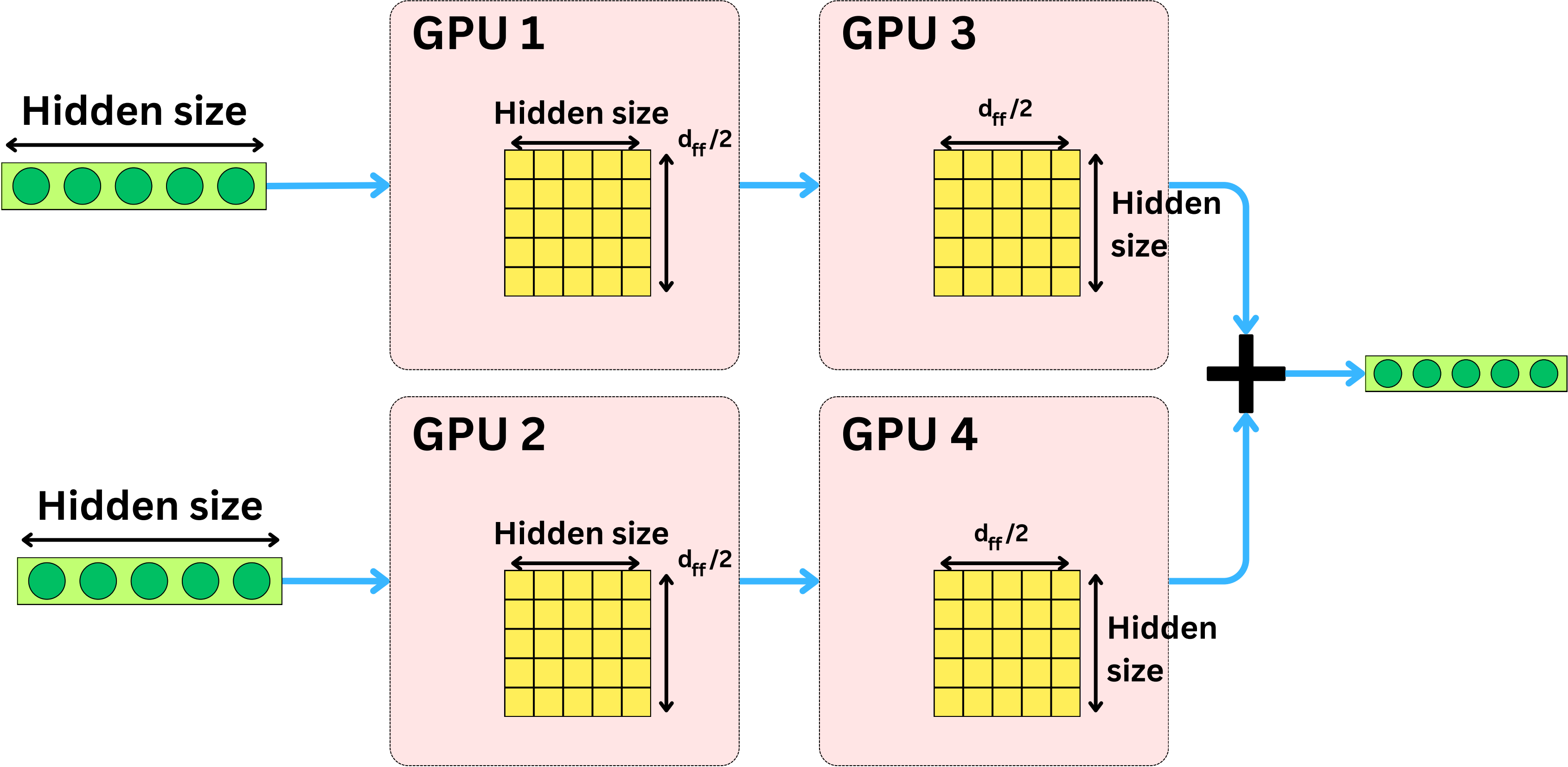


Tensor Parallelism



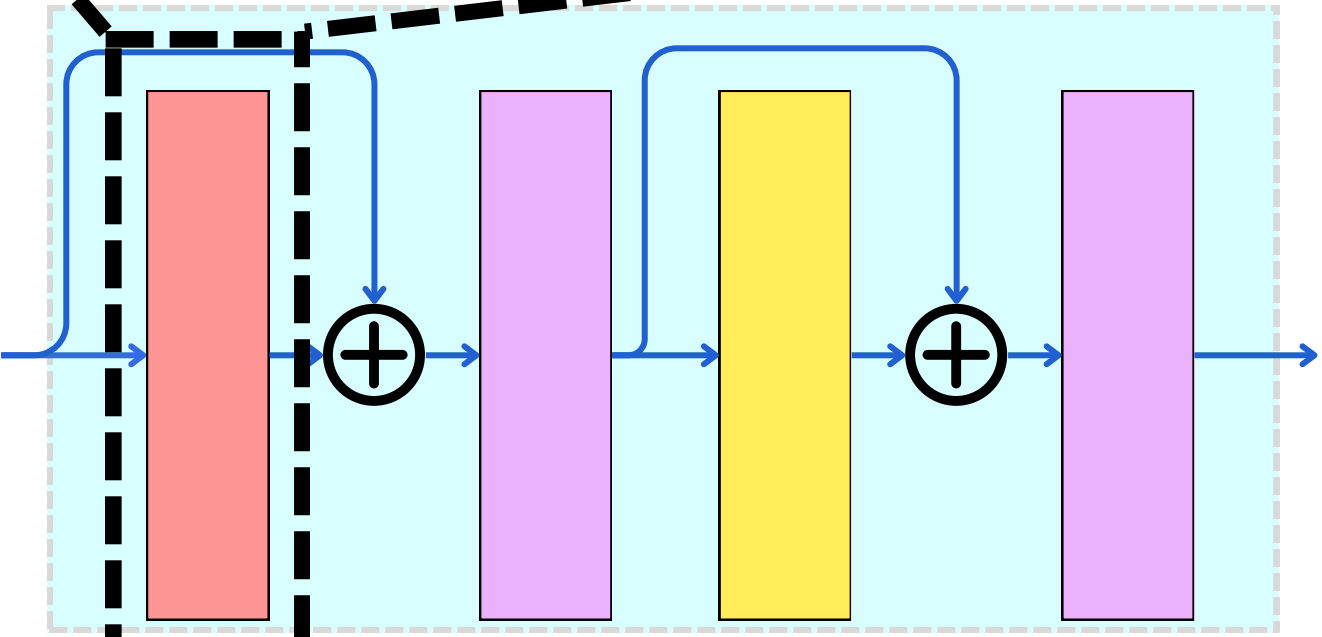
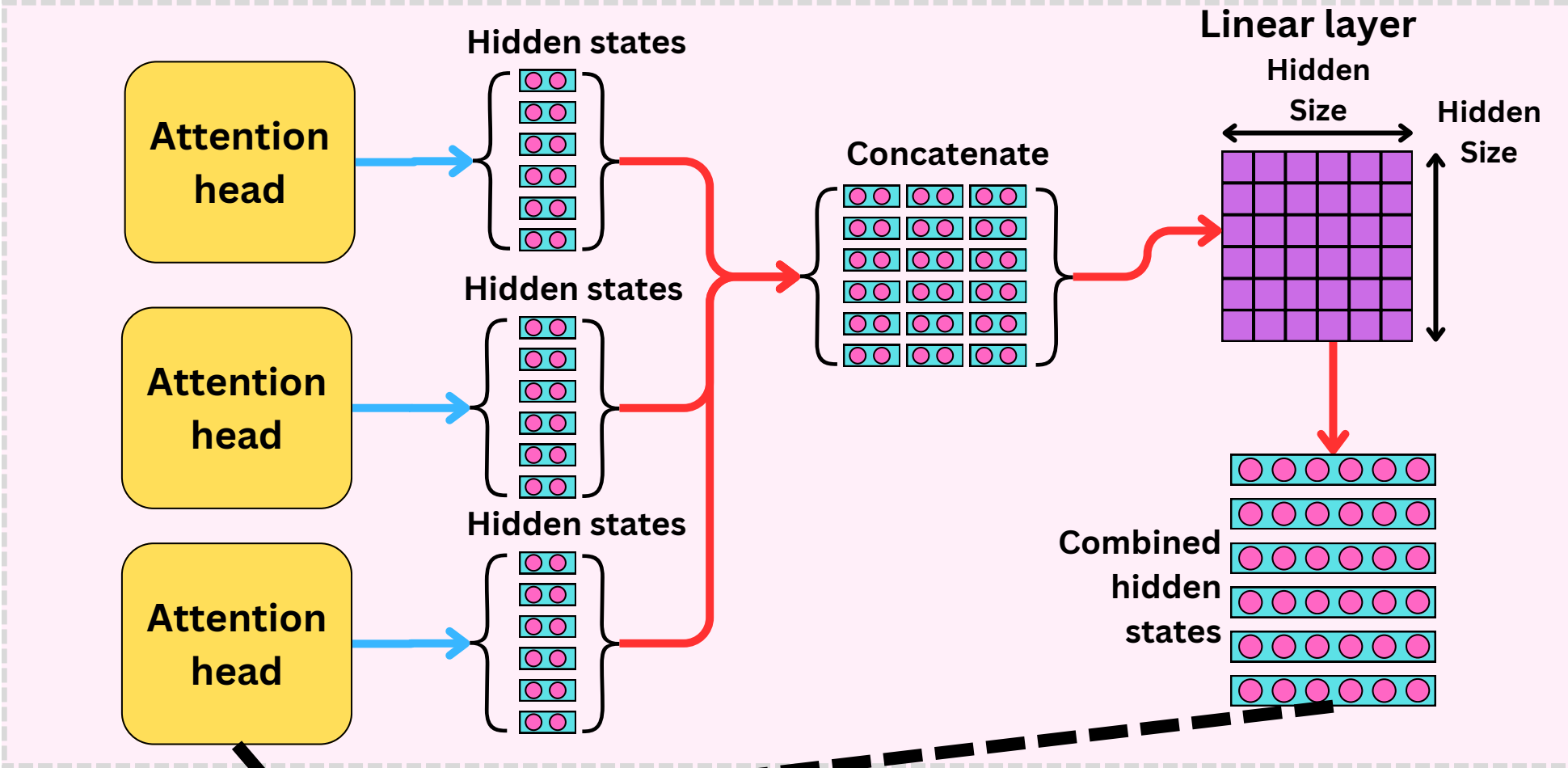


Tensor Parallelism





Tensor Parallelism



Multihead
Attention
Layer

Layer
Normalization

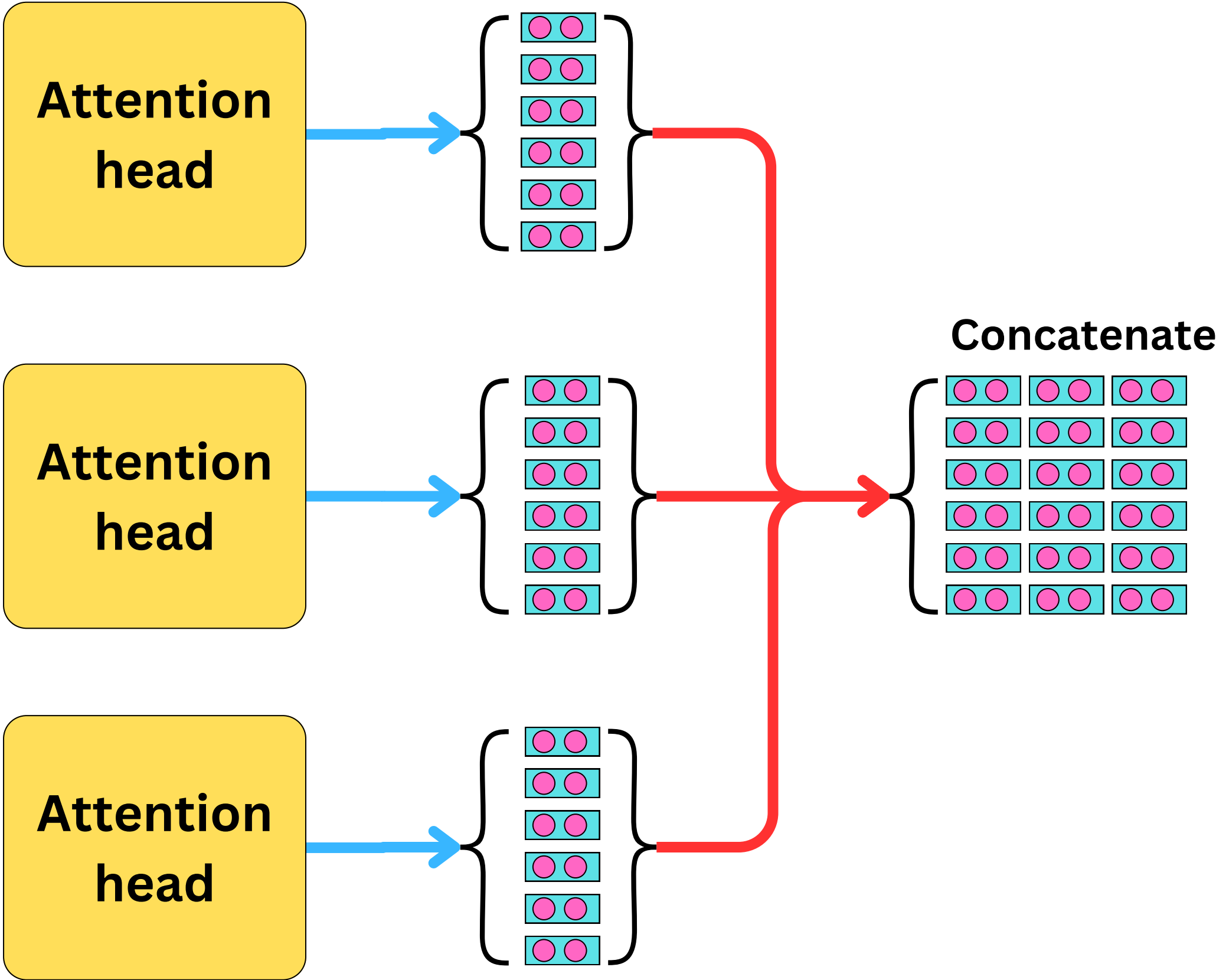
Feed
Forward
network

Layer
Normalization

Encoder block

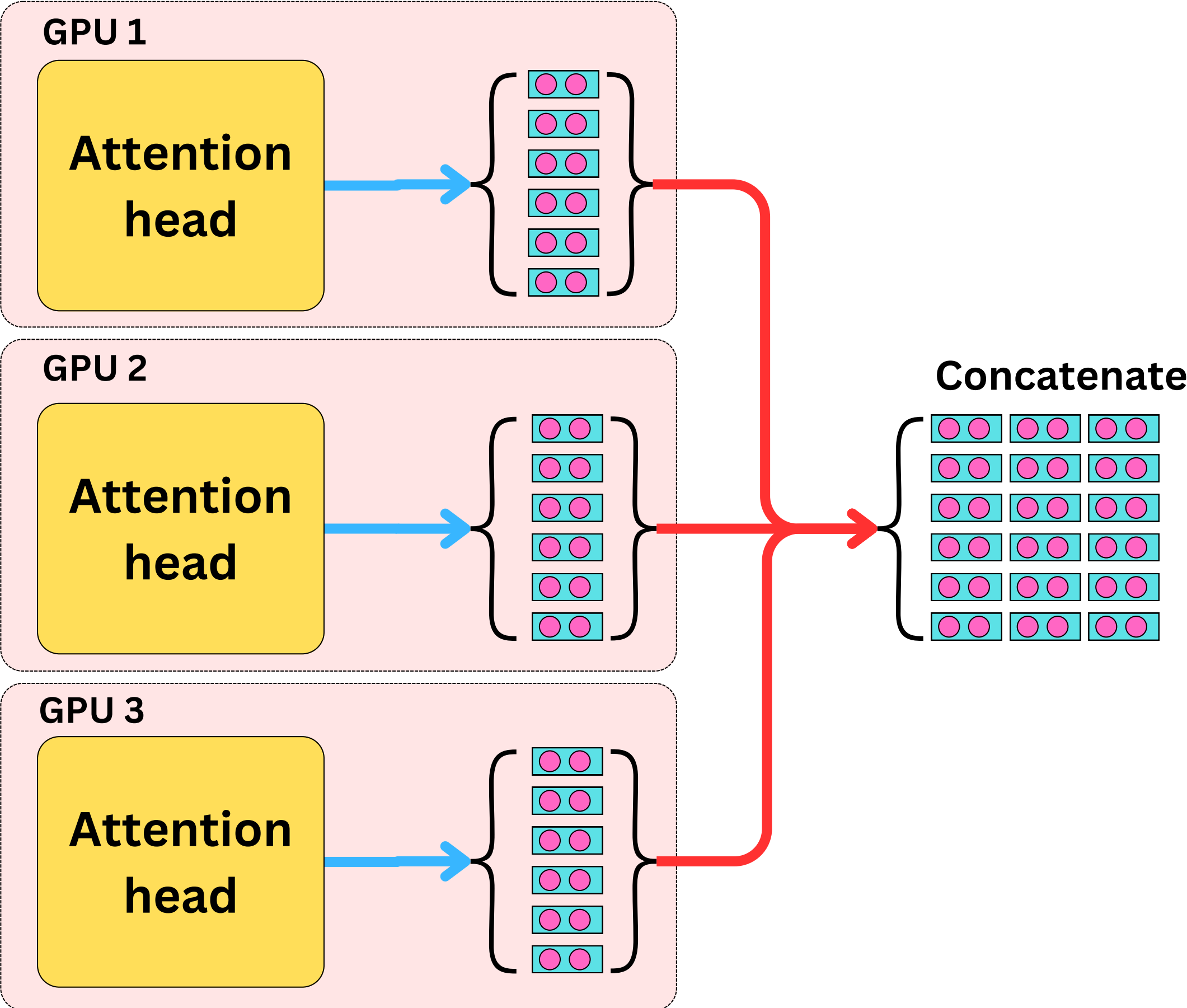


Tensor Parallelism





Tensor Parallelism





Zero Redundancy Optimizer

Model State

- Model parameters (forward pass)
 - float16
- The Gradients (backward pass)
 - float16
- The Optimizer parameters (to update the weights)
 - float32



Zero Redundancy Optimizer

Adam Optimizer

Momentum

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

Variance

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Weights update

$$\theta_t = \theta_{t-1} - \frac{\mu}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$



Zero Redundancy Optimizer

Adam Optimizer

m_{t-1} Momentum

v_{t-1} Variance

$\nabla_{\theta} \mathcal{L}$ Gradient

θ_{t-1} Model parameters

float32

float32

float32

float32

Momentum

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}$$

Variance

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2$$

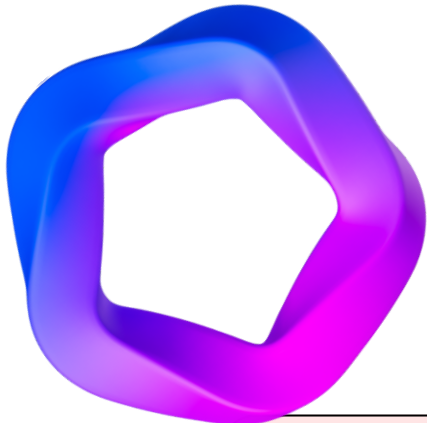
Weights update

$$\theta_t = \theta_{t-1} - \frac{\mu}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

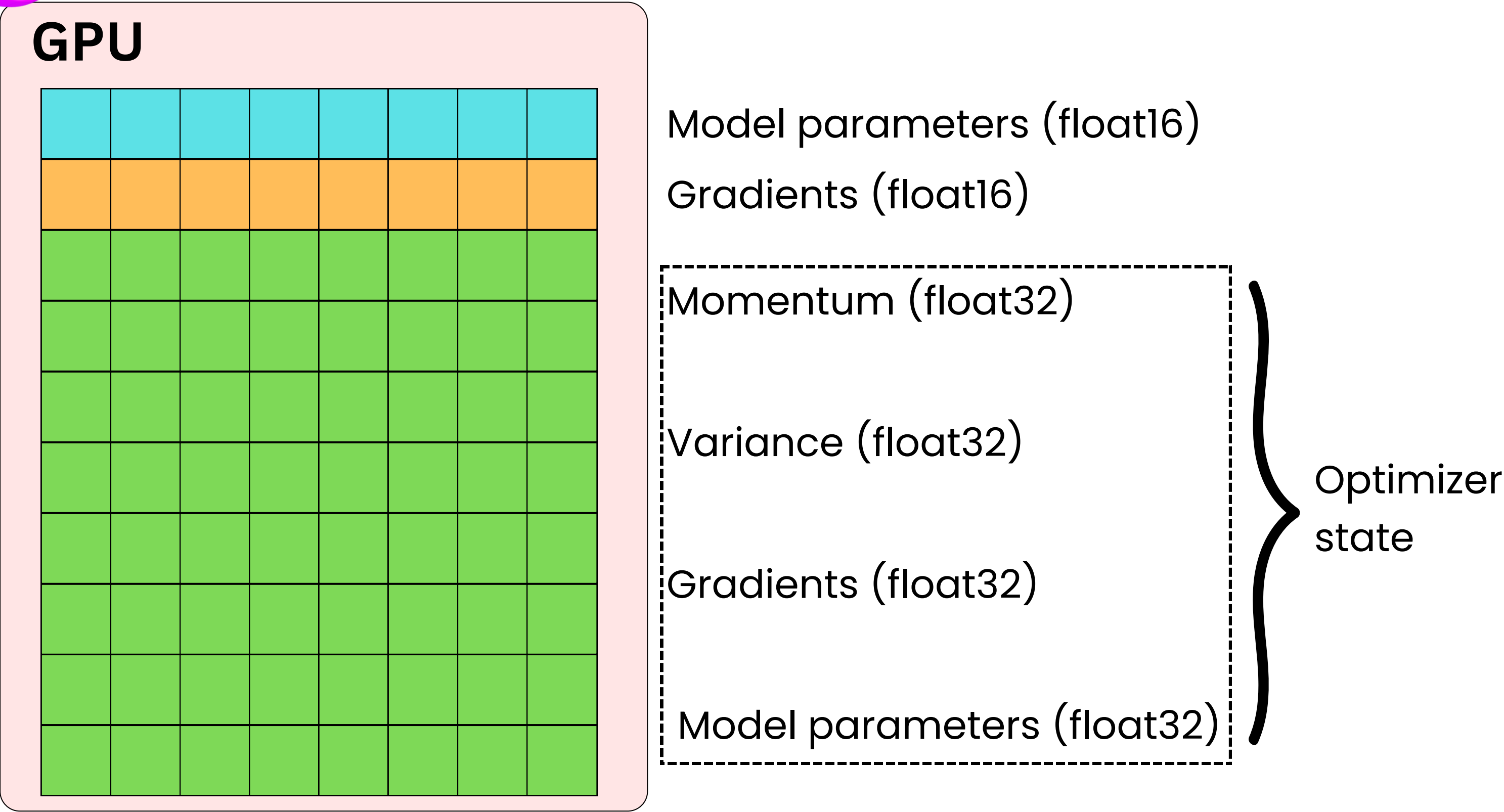
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

8x more memory
than the model
parameters!

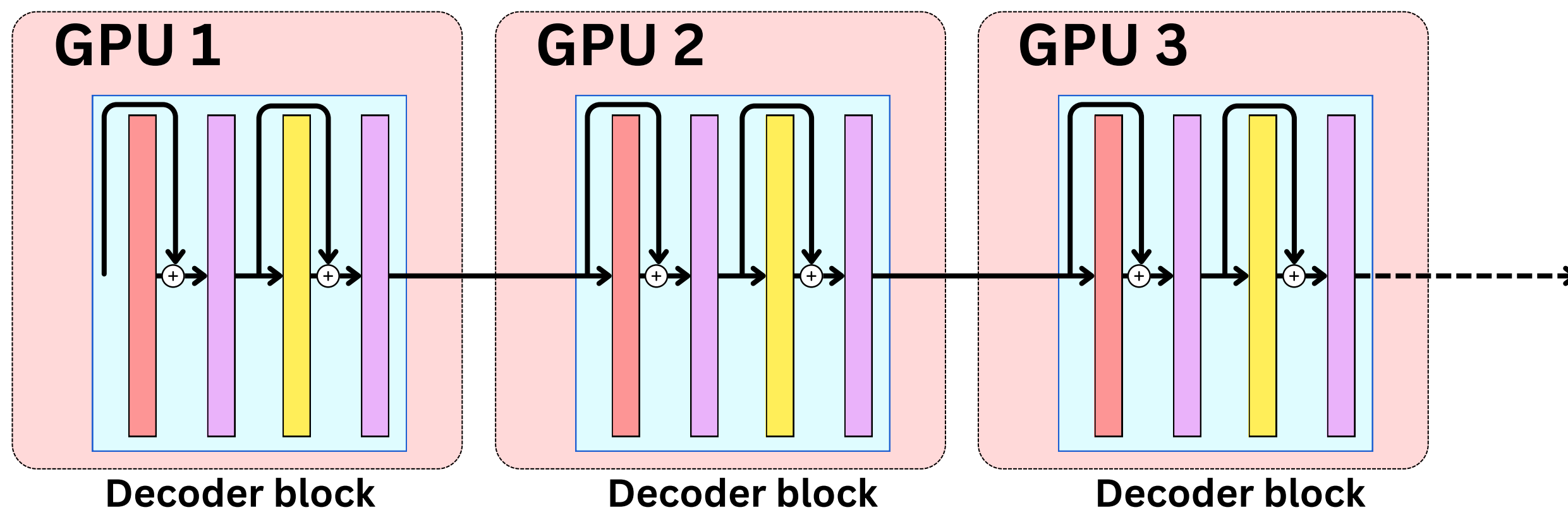


Zero Redundancy Optimizer



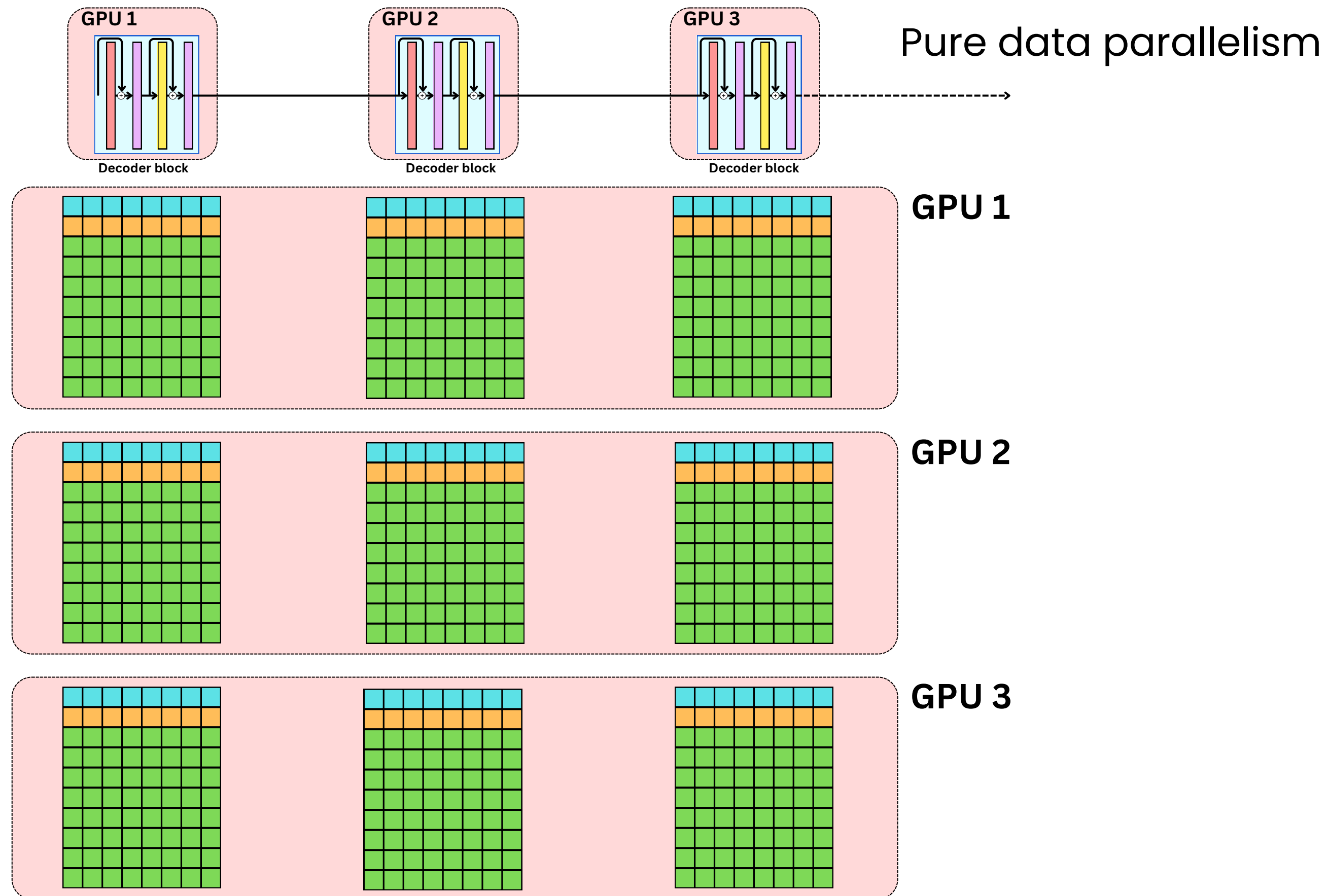


Zero Redundancy Optimizer



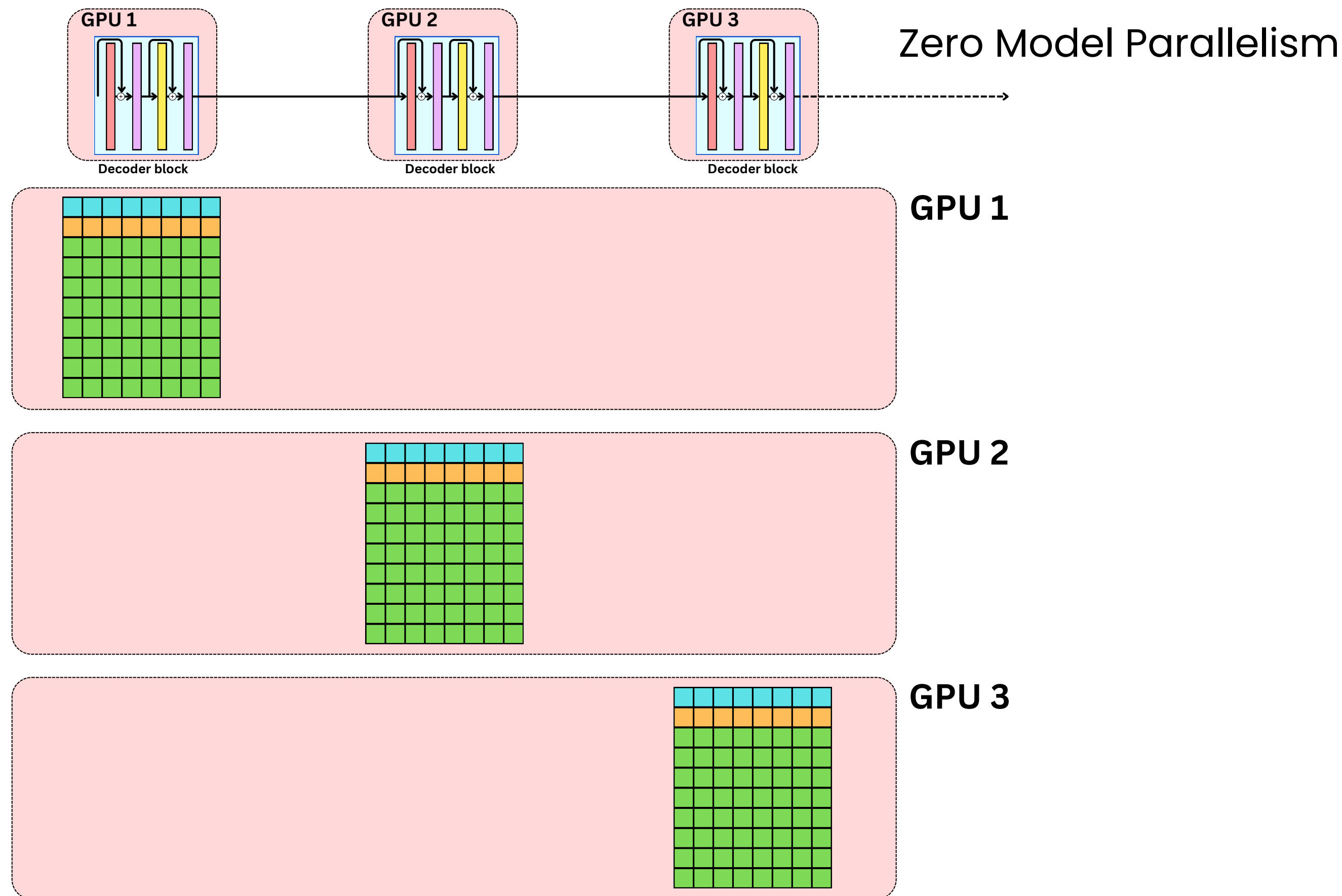


Zero Redundancy Optimizer



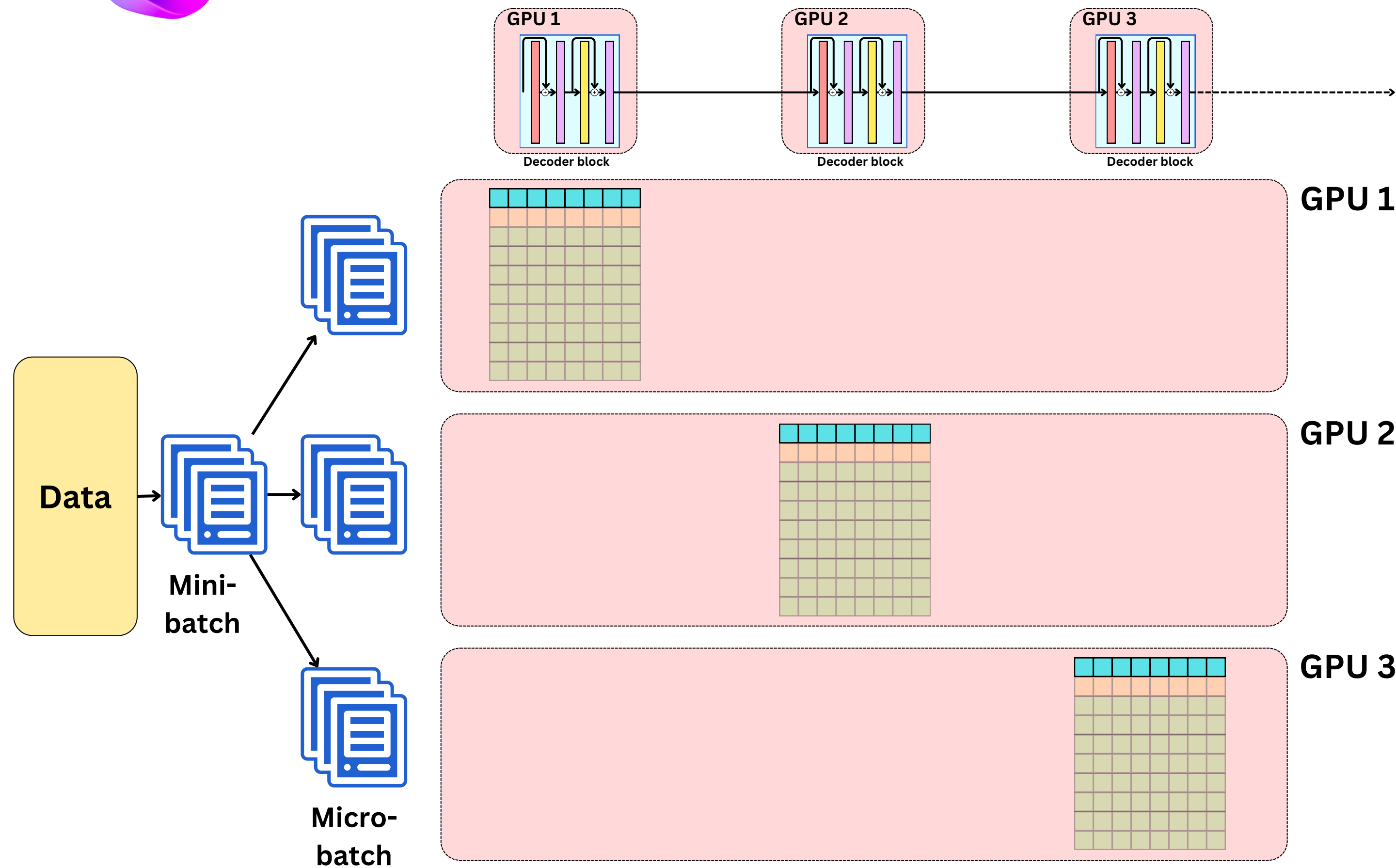


Zero Redundancy Optimizer



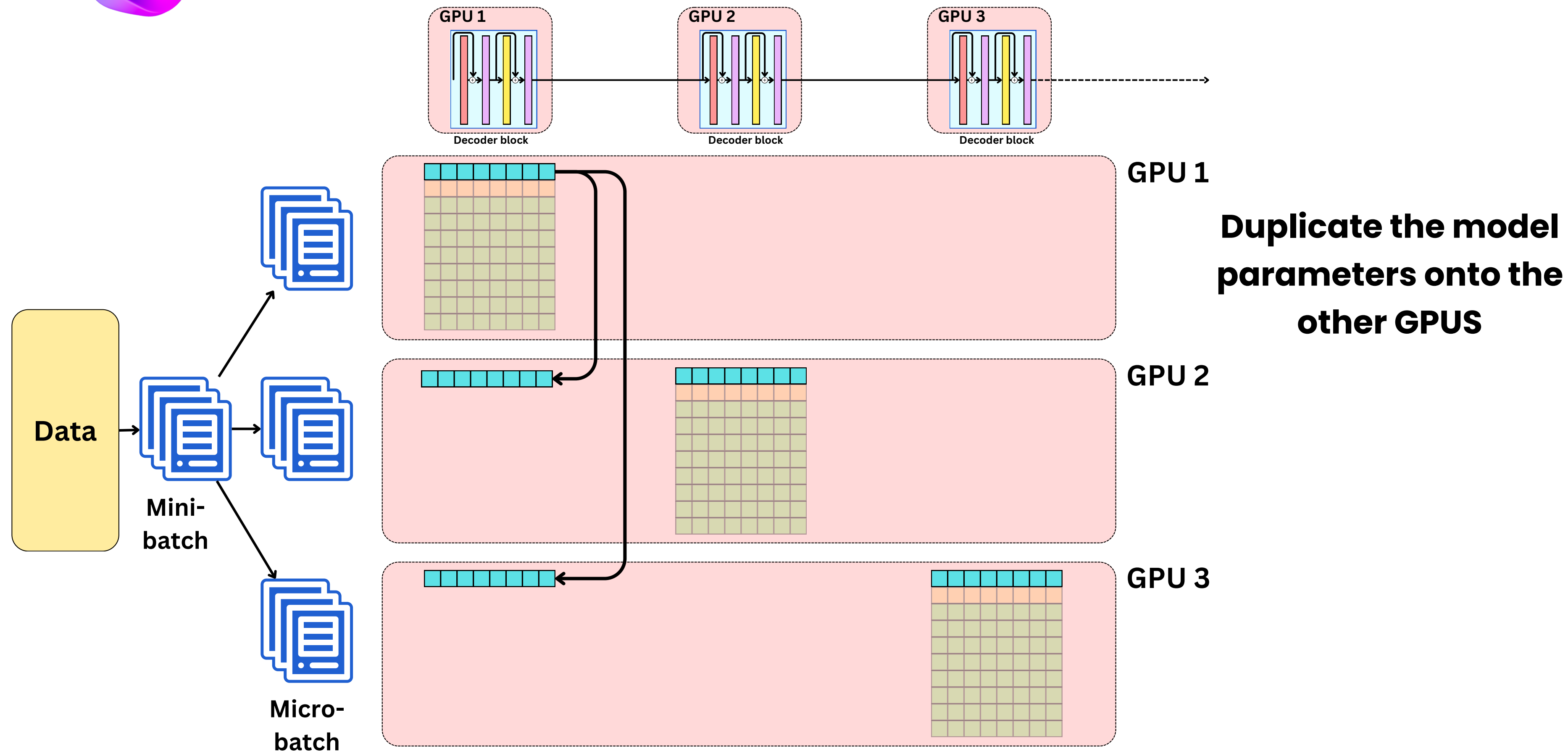


Zero Redundancy Optimizer



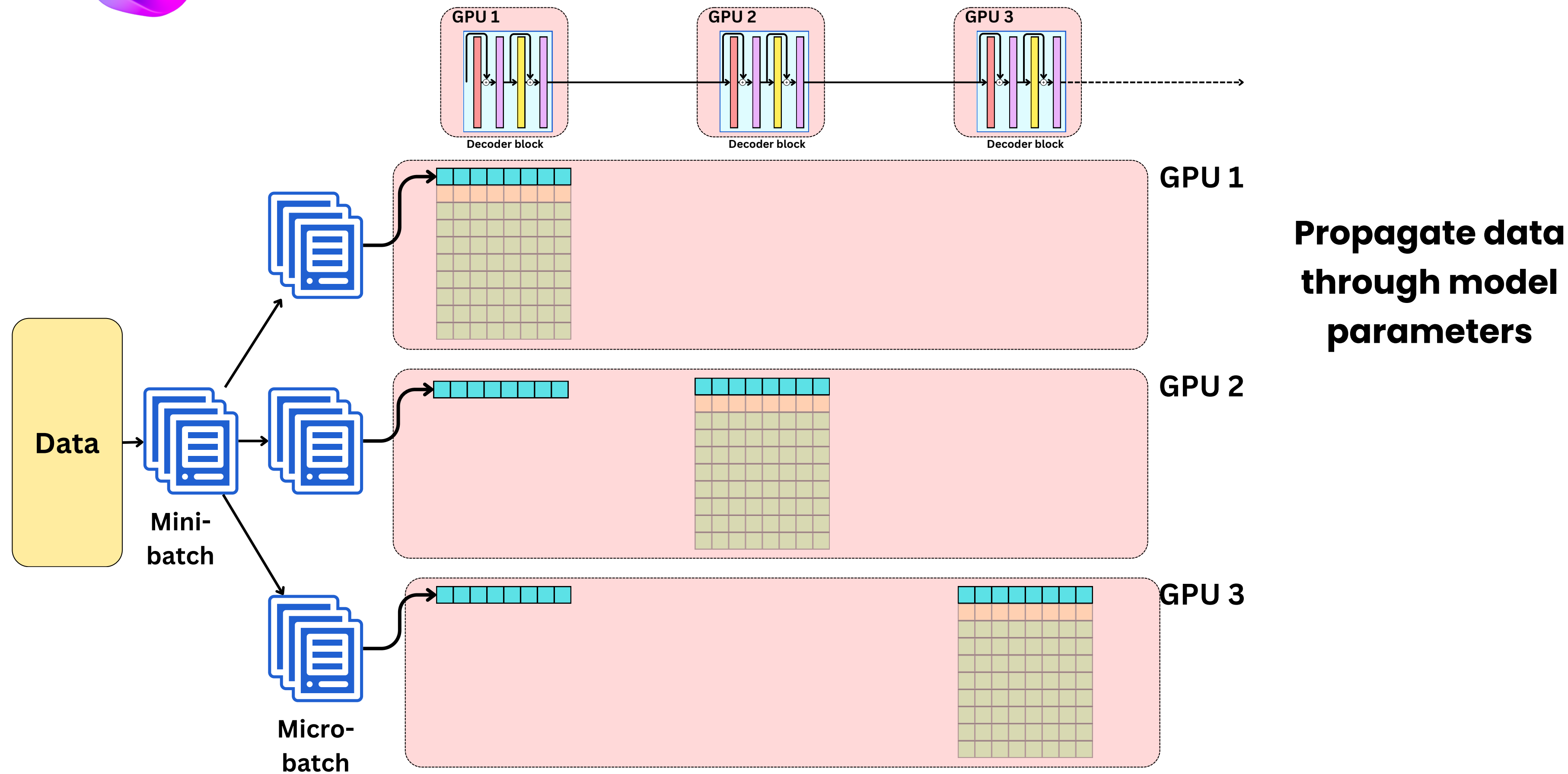


Zero Redundancy Optimizer



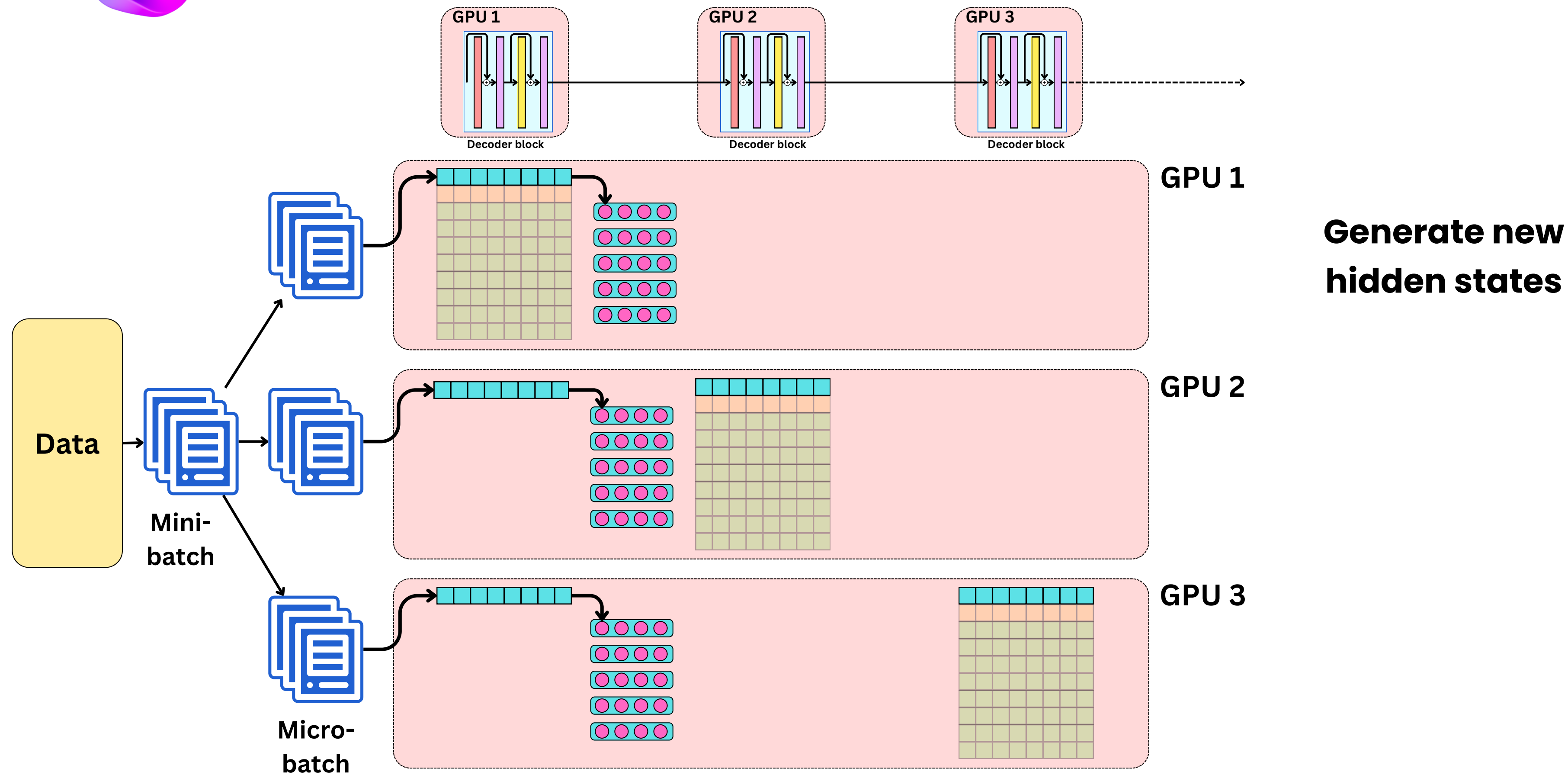


Zero Redundancy Optimizer



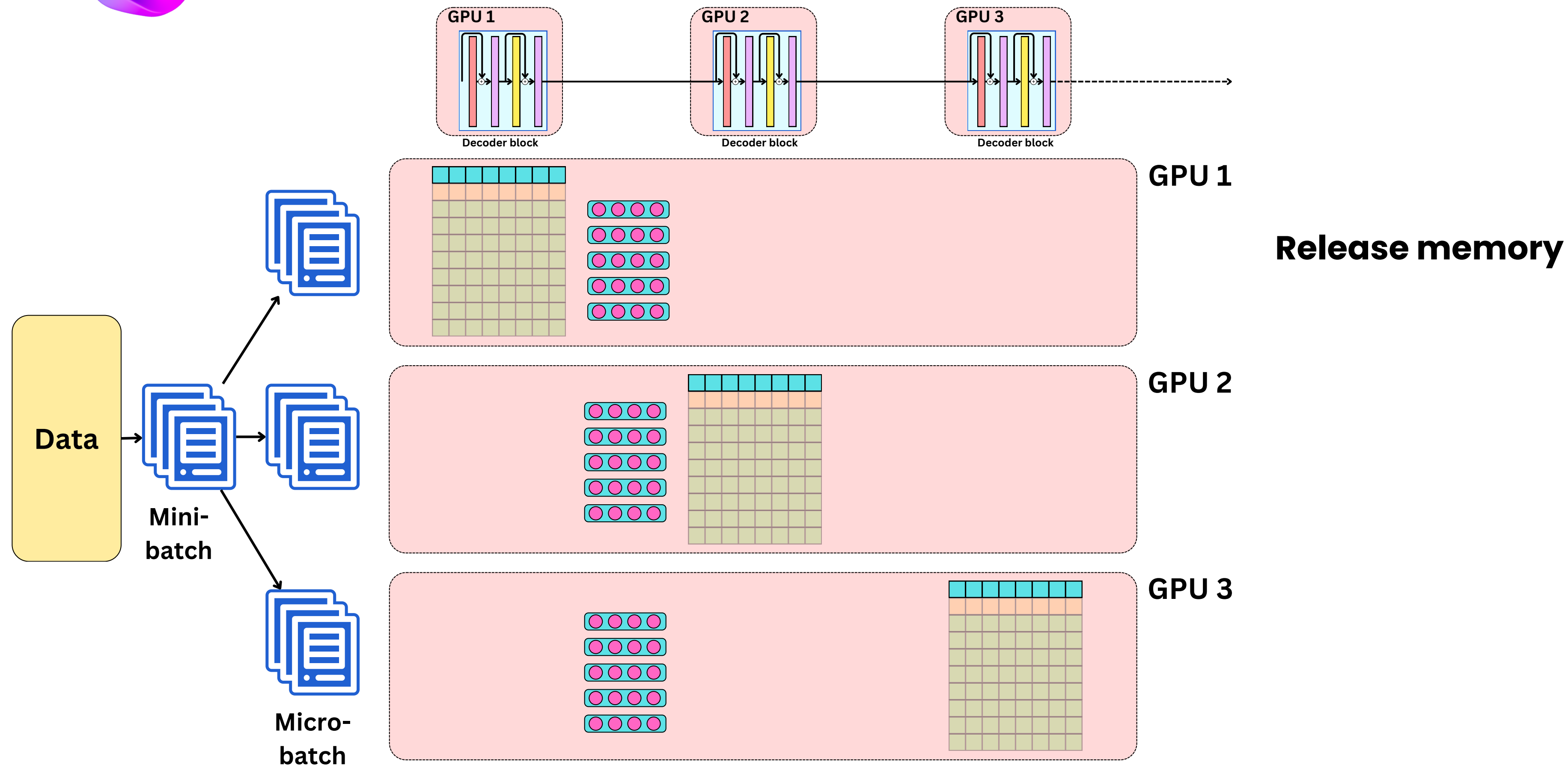


Zero Redundancy Optimizer



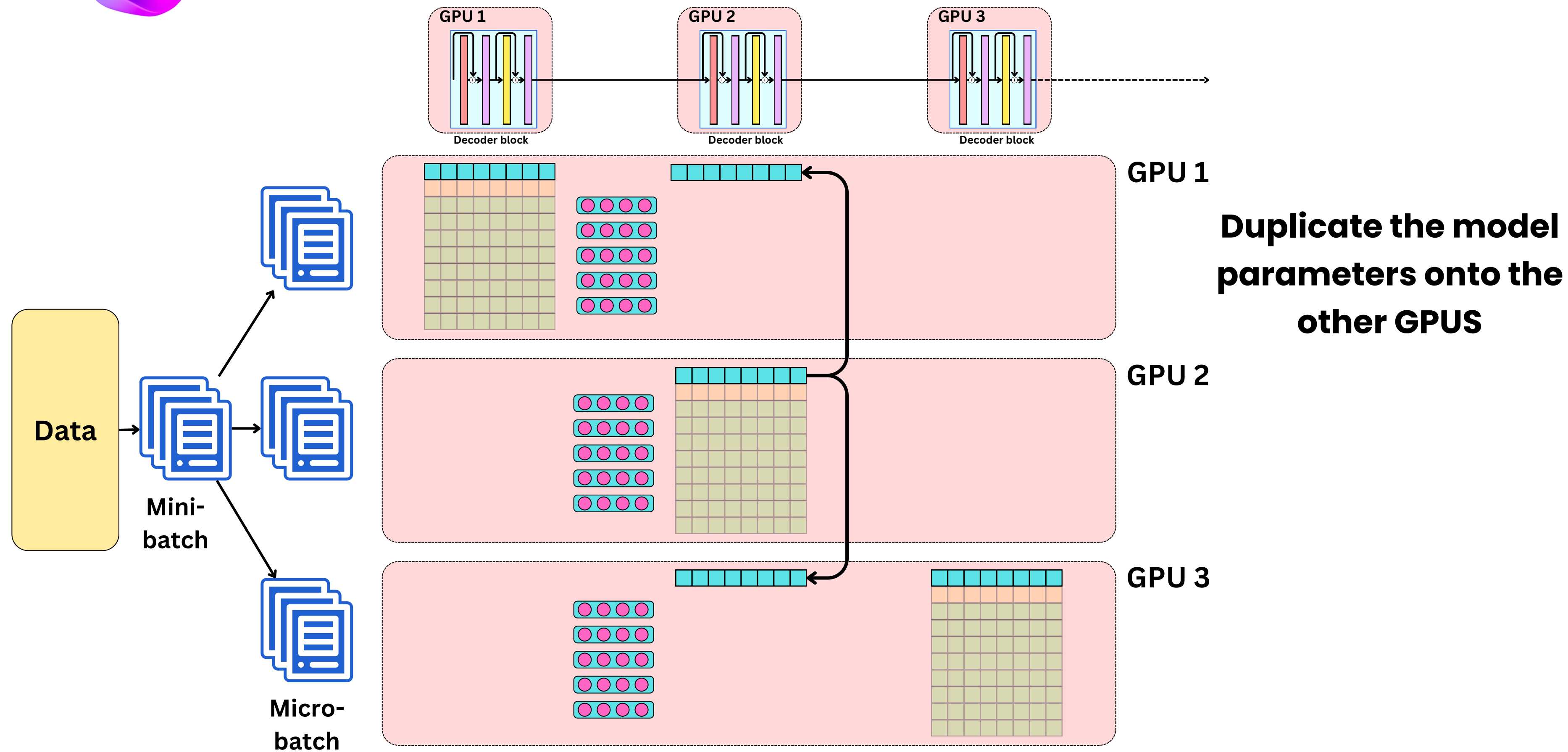


Zero Redundancy Optimizer



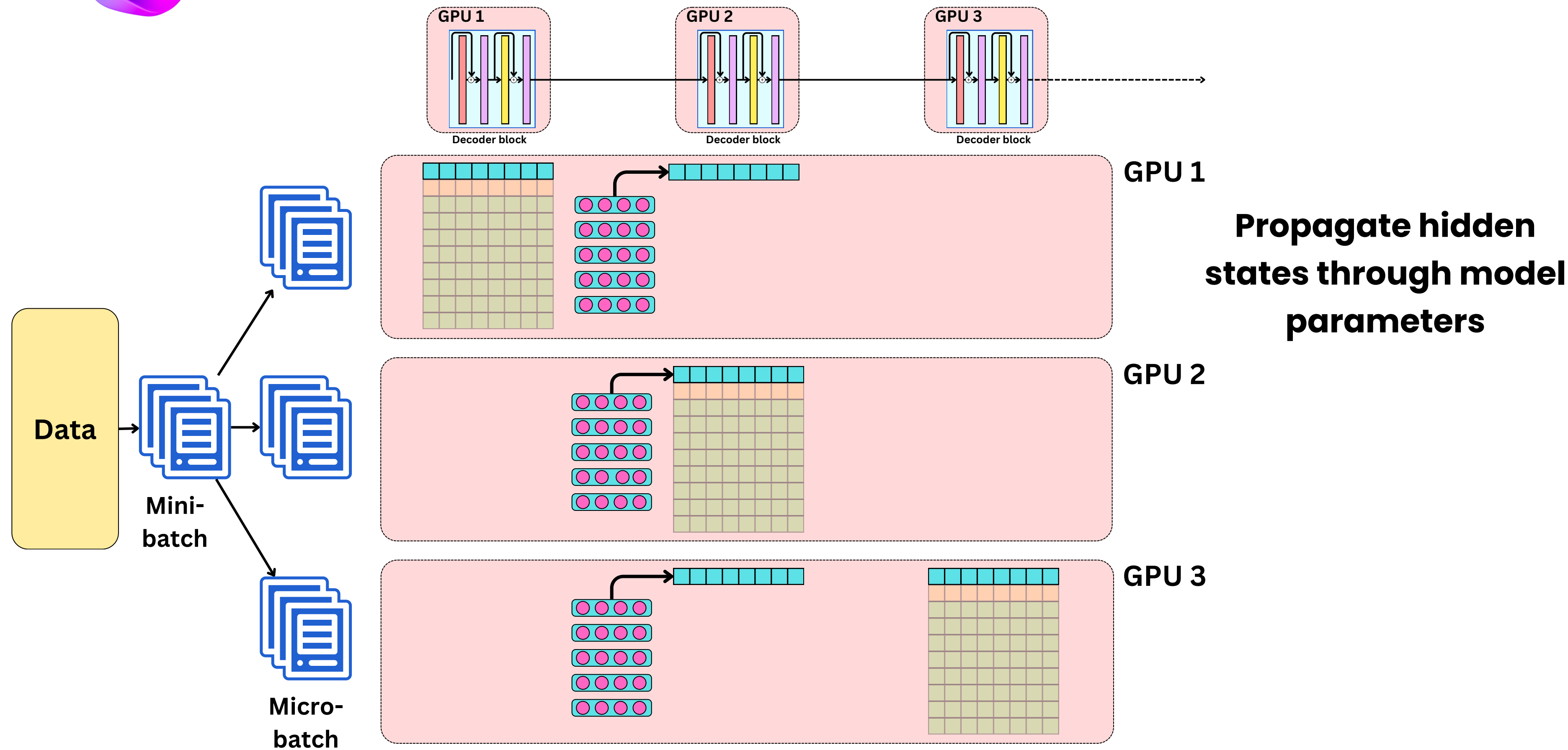


Zero Redundancy Optimizer



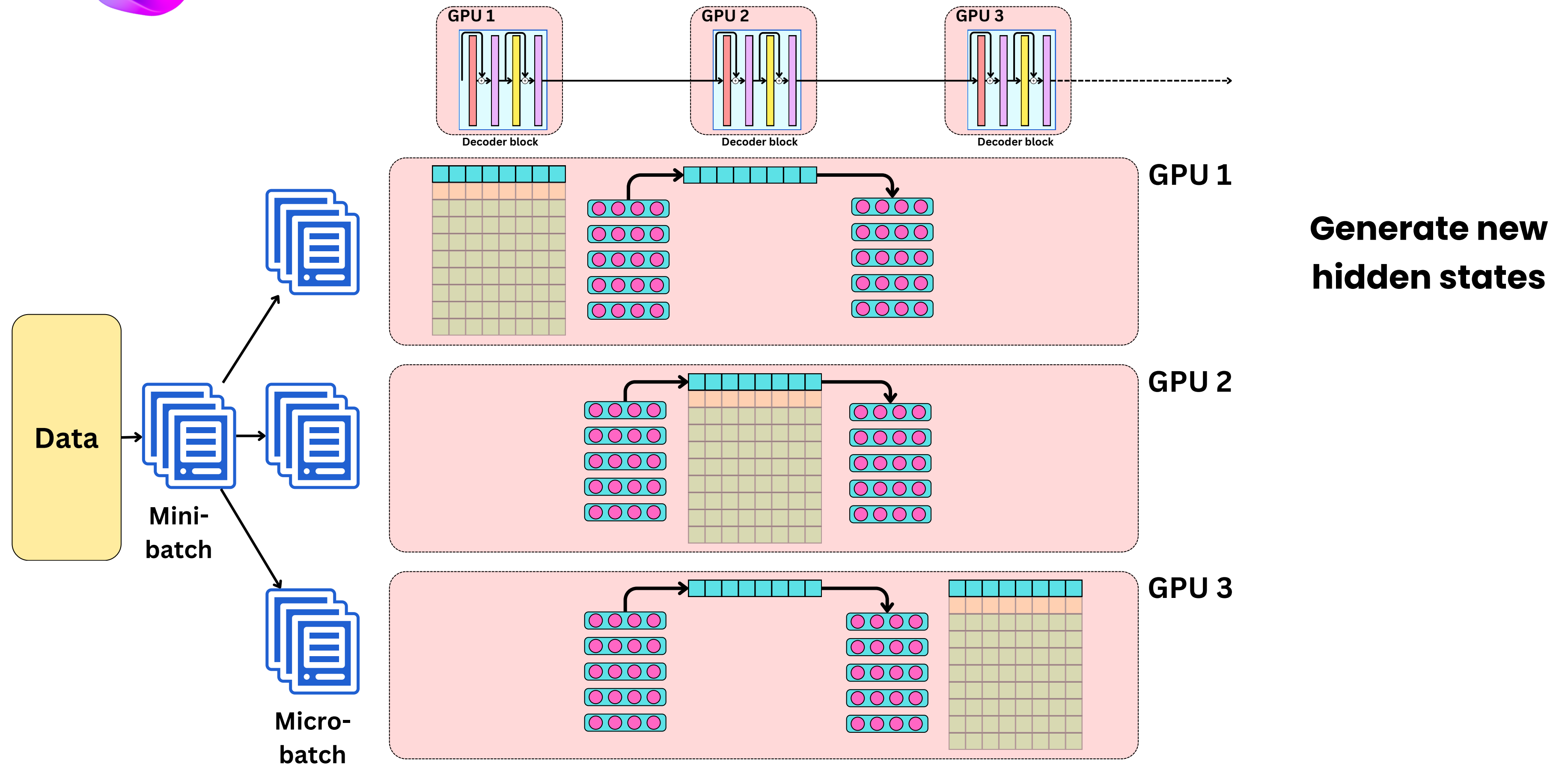


Zero Redundancy Optimizer



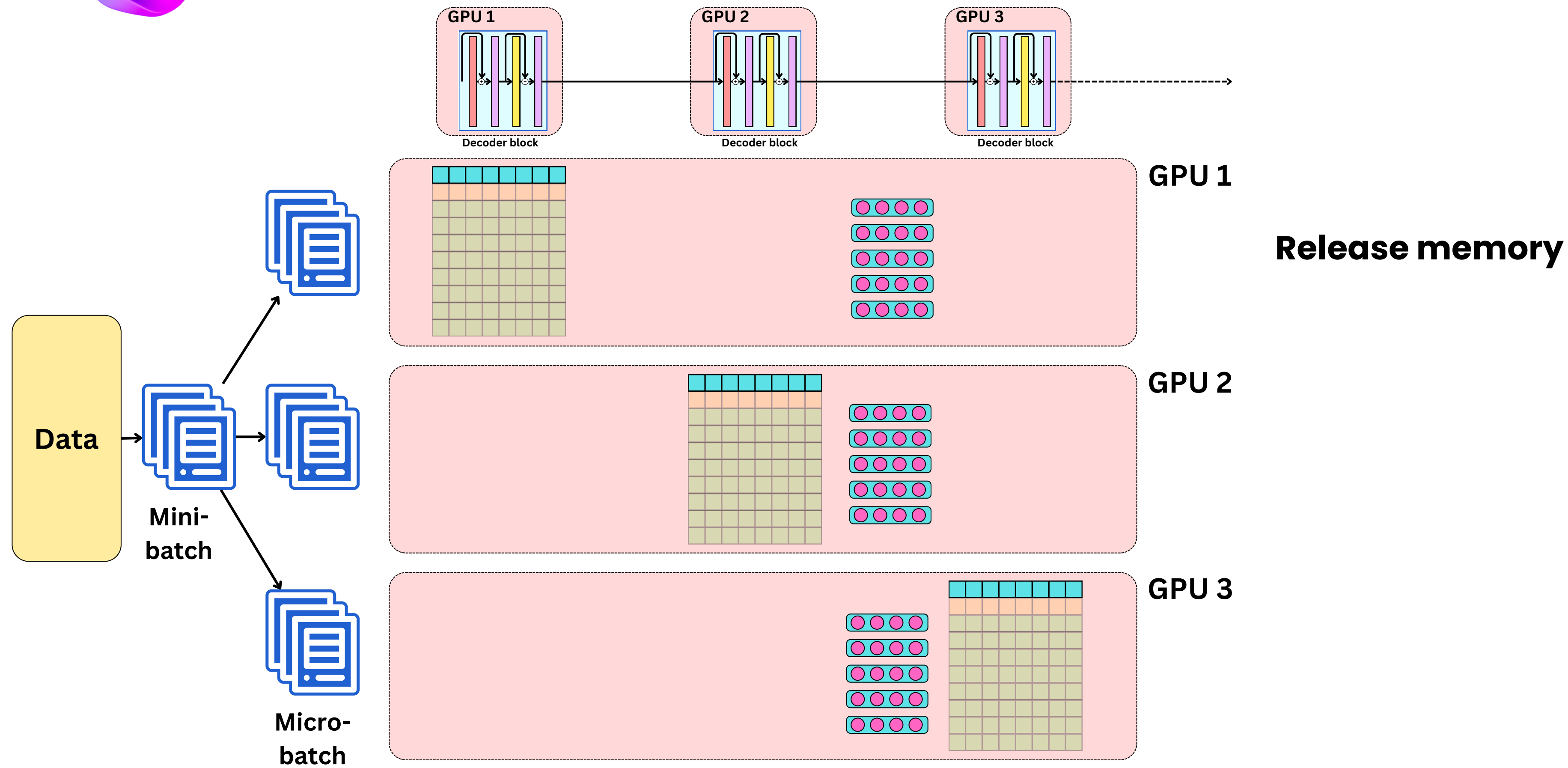


Zero Redundancy Optimizer



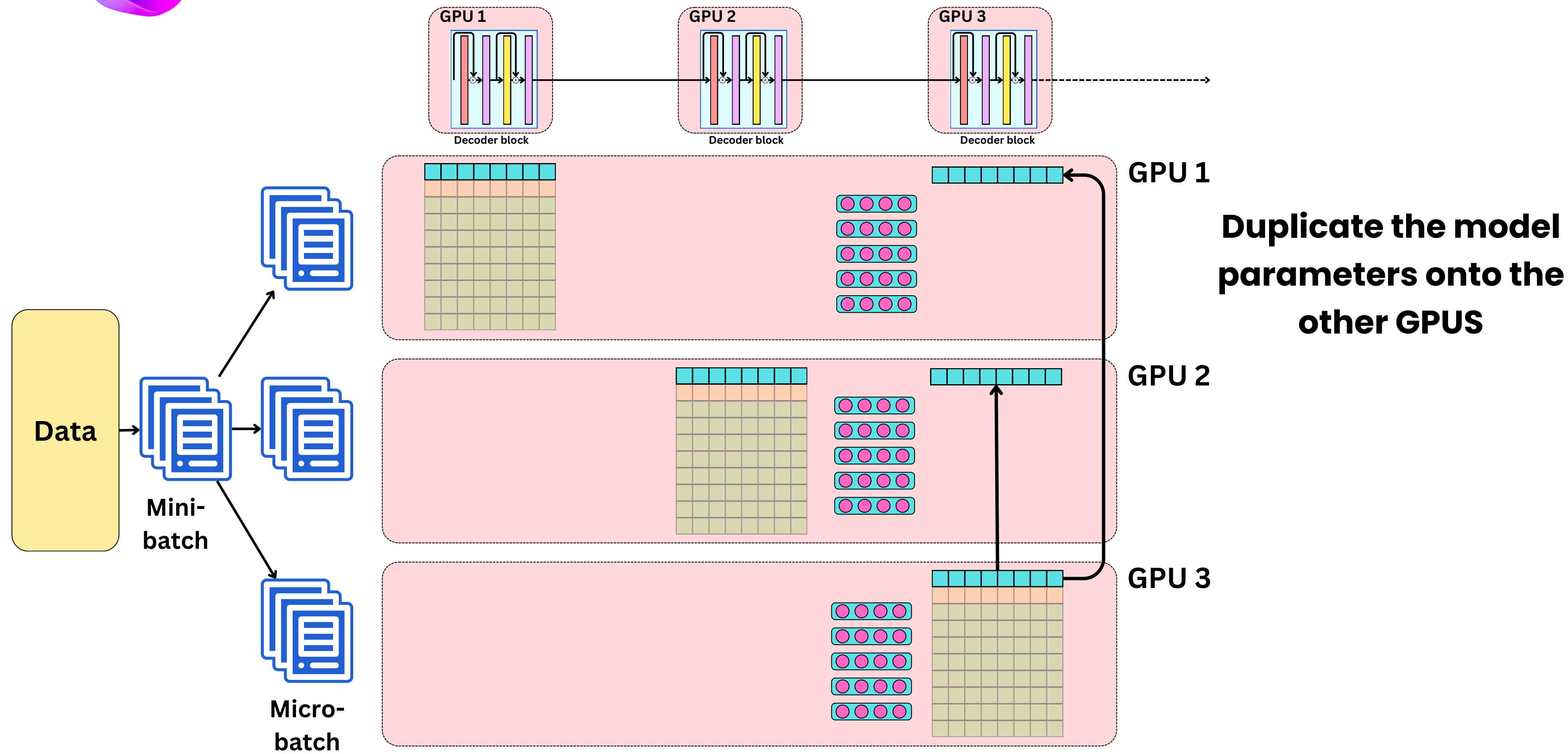


Zero Redundancy Optimizer



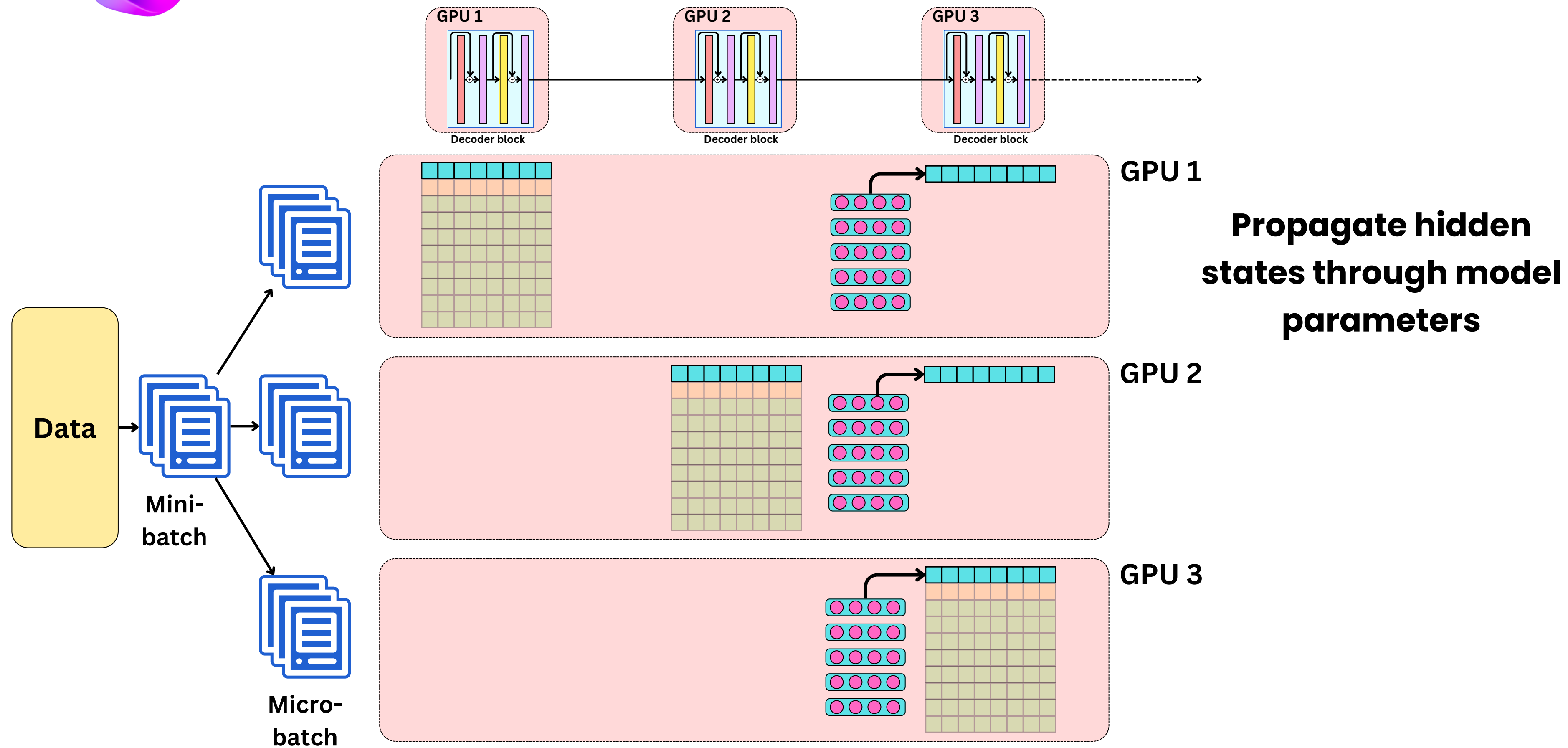


Zero Redundancy Optimizer



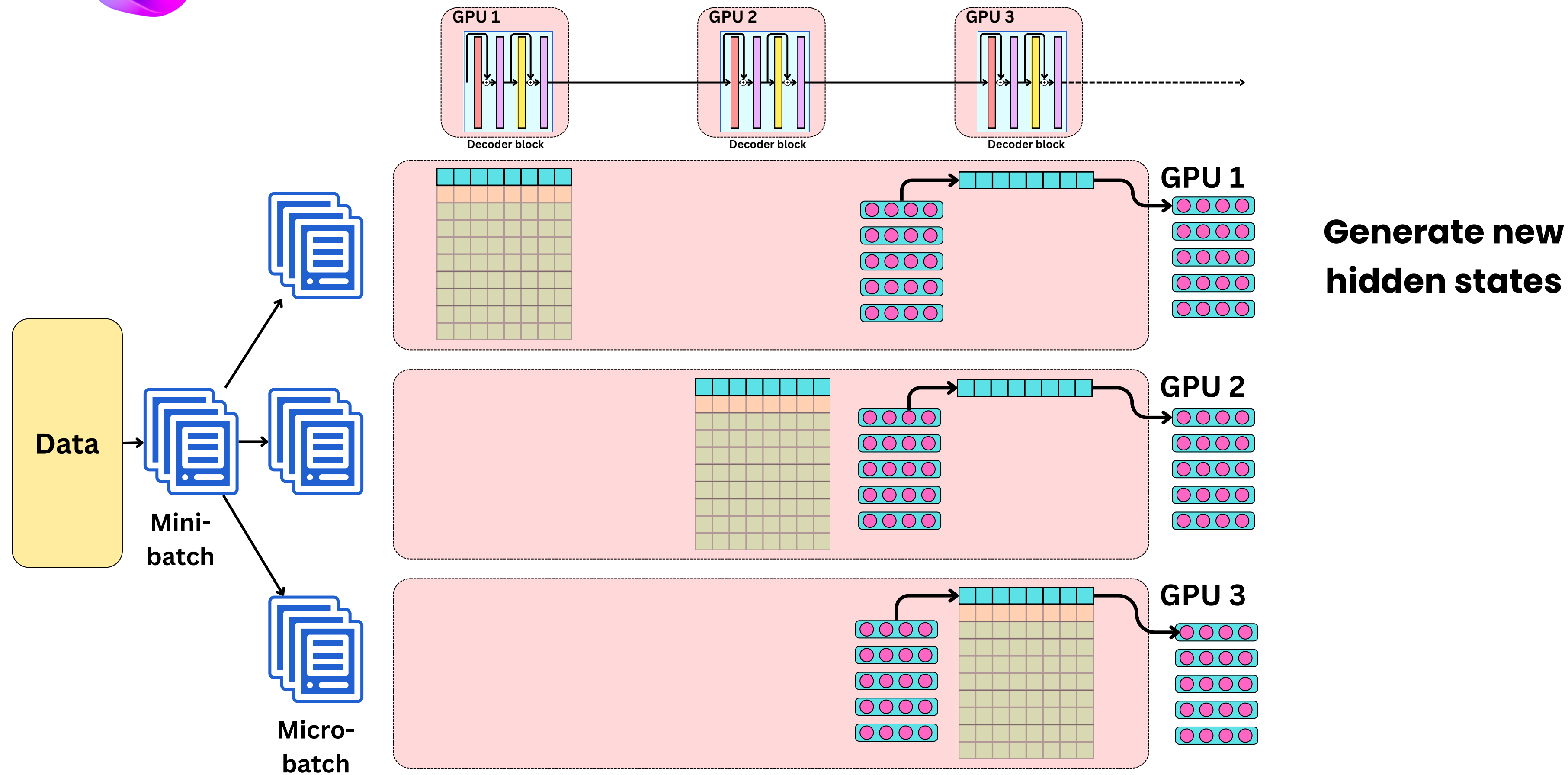


Zero Redundancy Optimizer



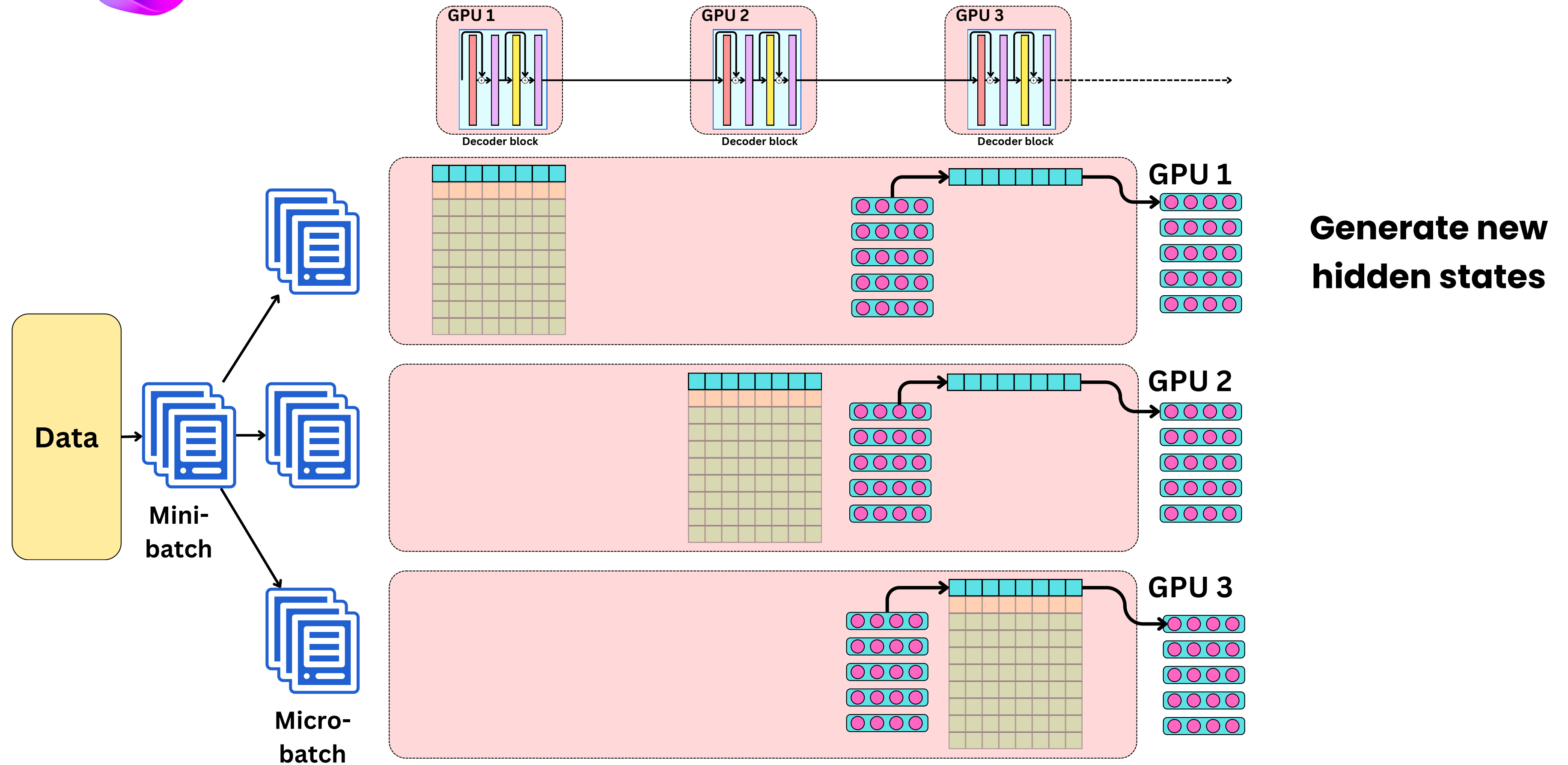


Zero Redundancy Optimizer



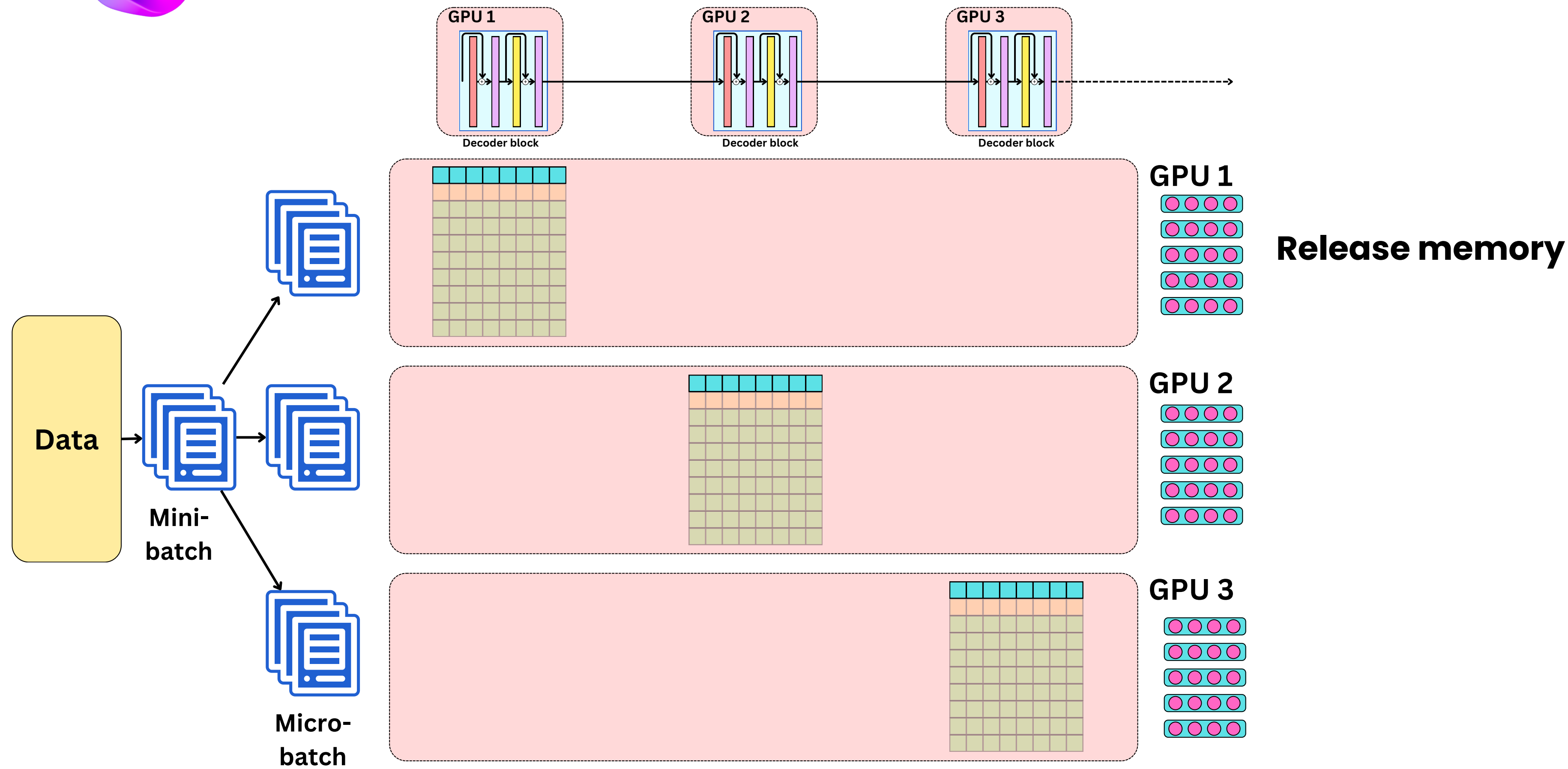


Zero Redundancy Optimizer



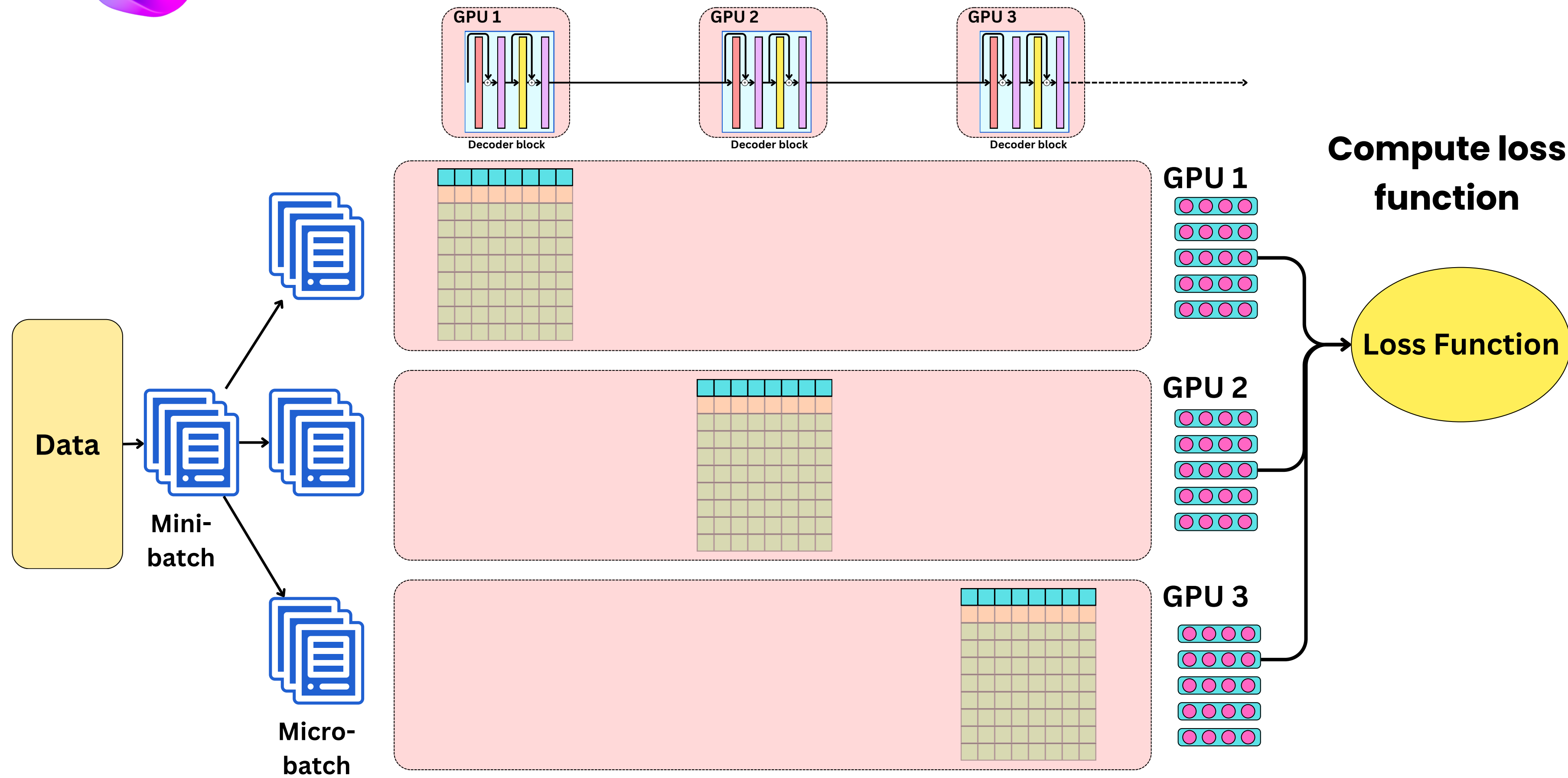


Zero Redundancy Optimizer



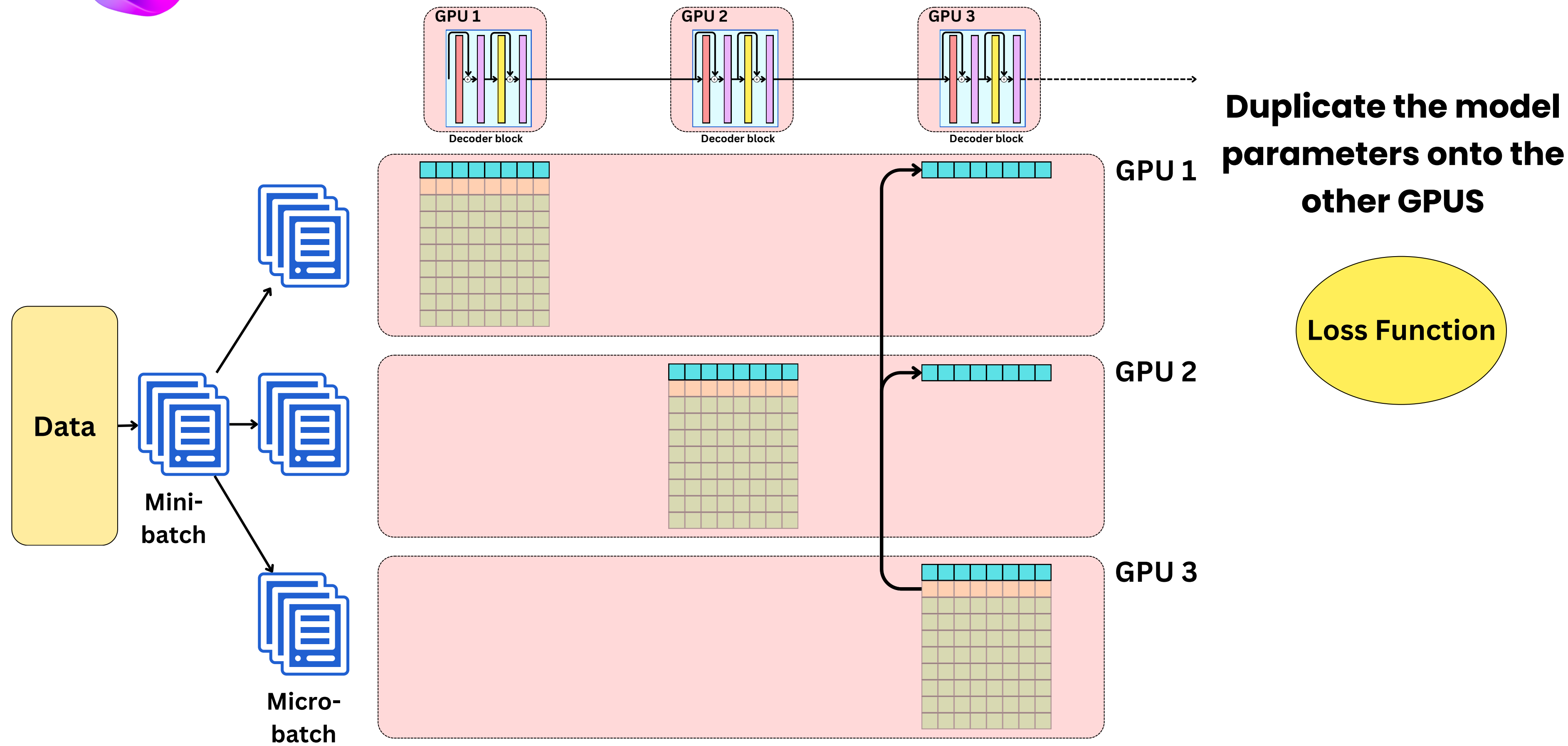


Zero Redundancy Optimizer



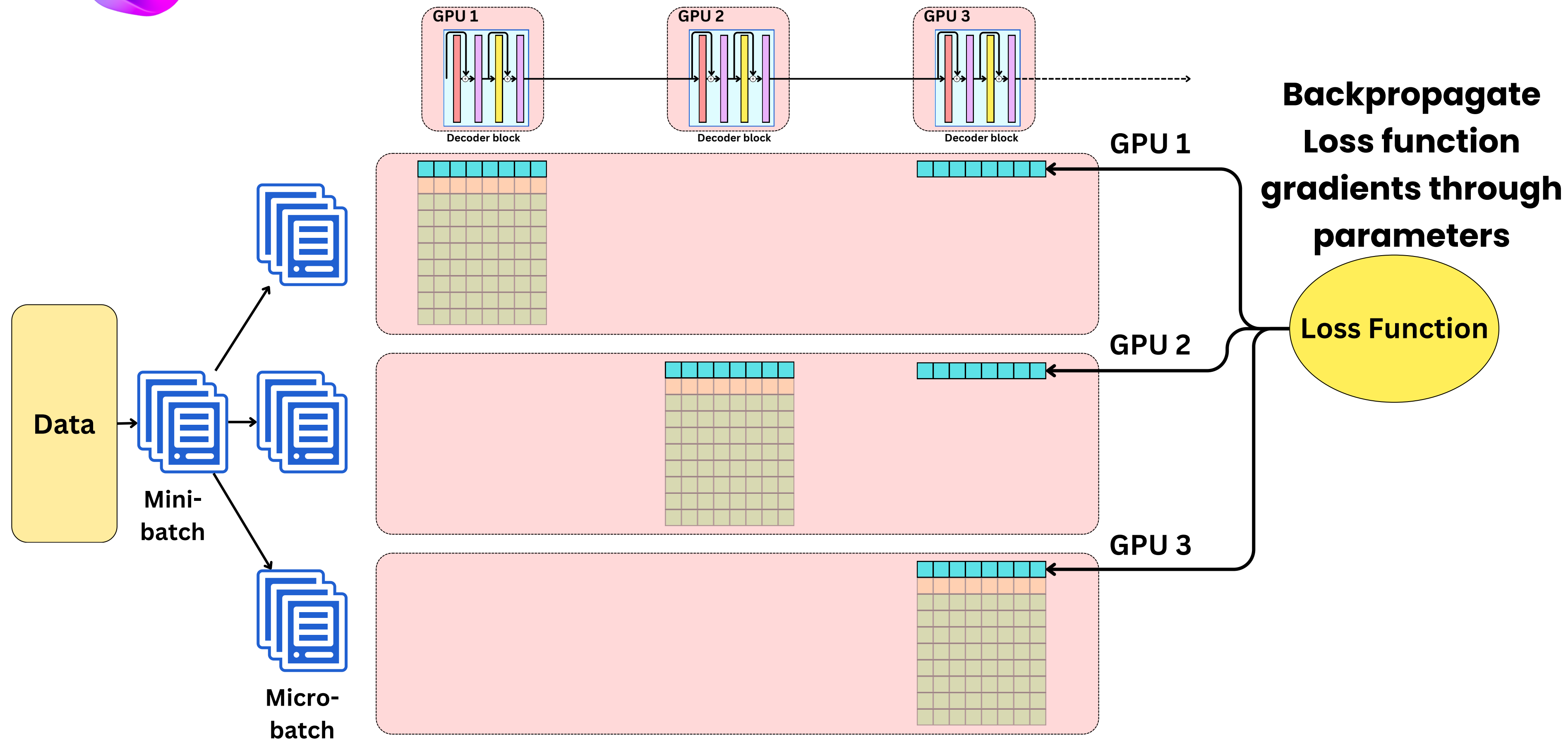


Zero Redundancy Optimizer



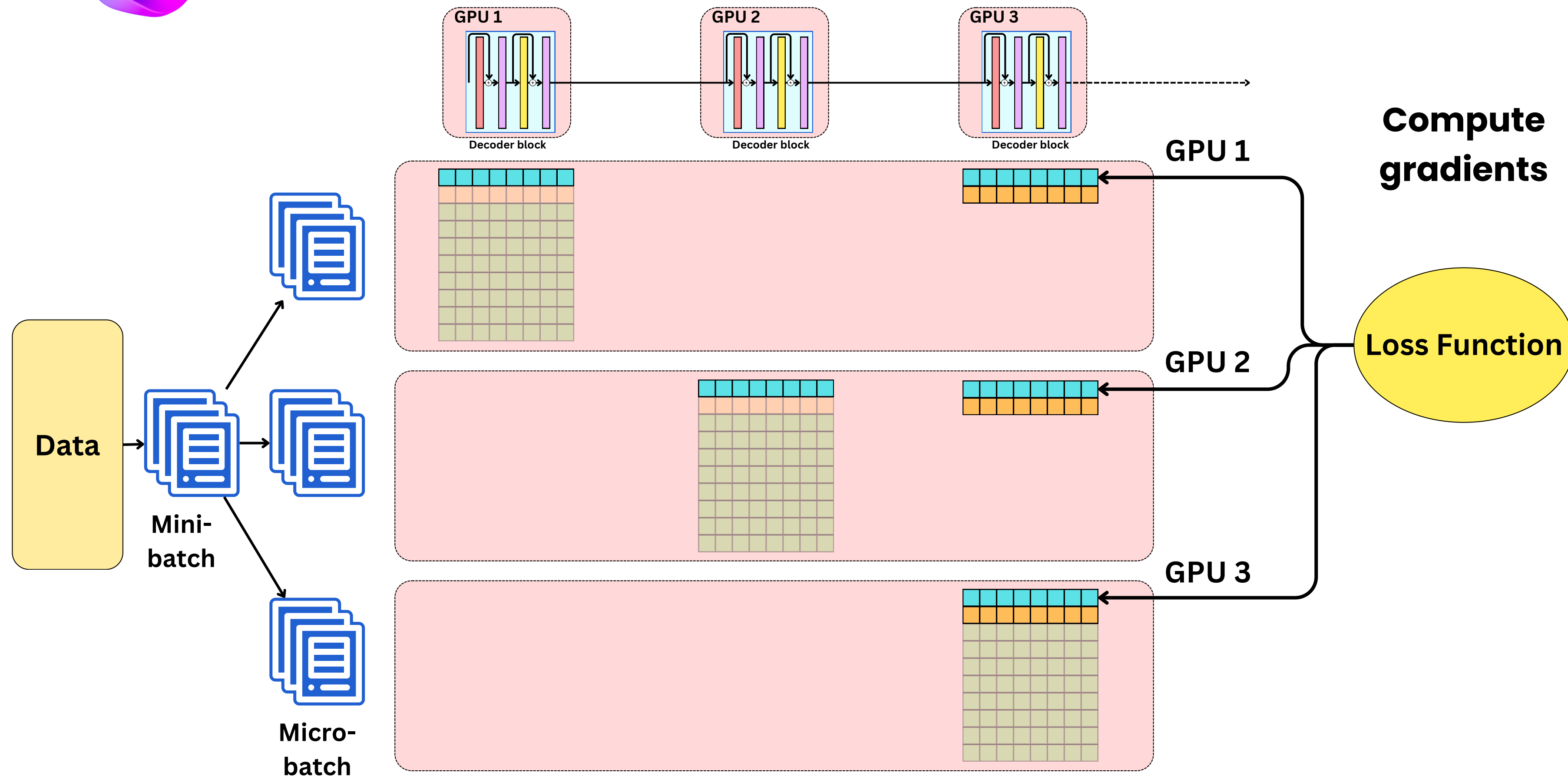


Zero Redundancy Optimizer



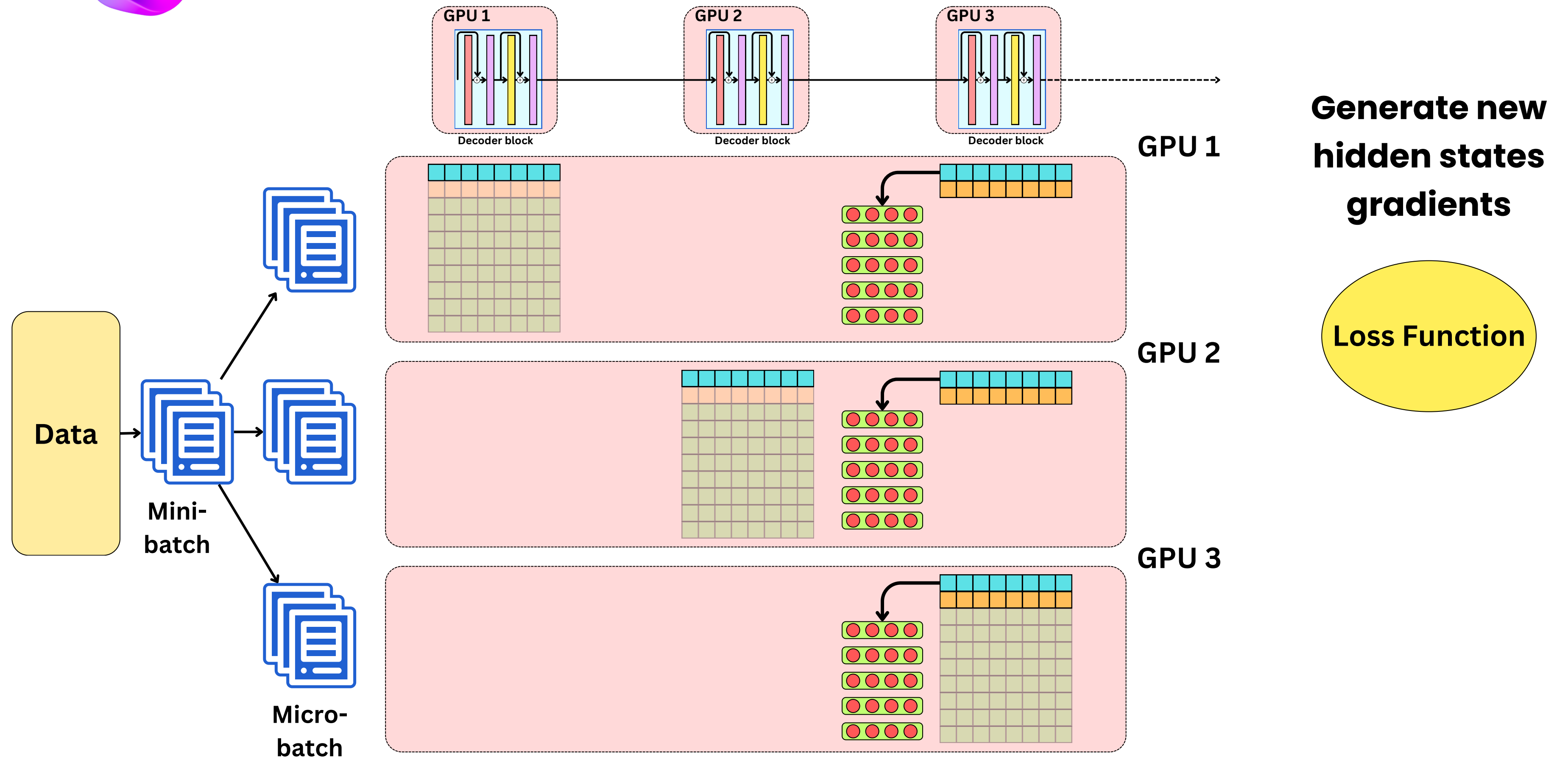


Zero Redundancy Optimizer



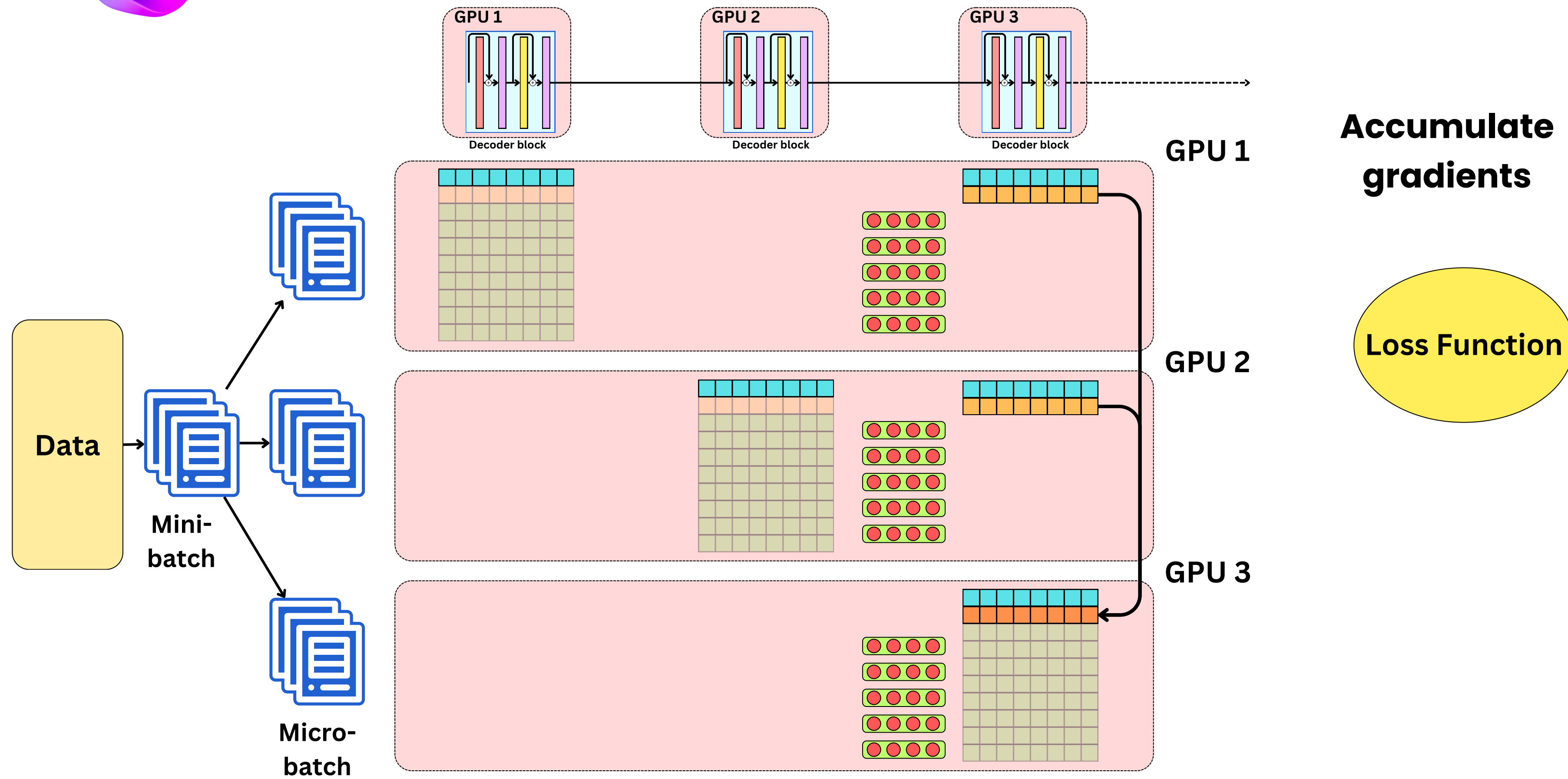


Zero Redundancy Optimizer



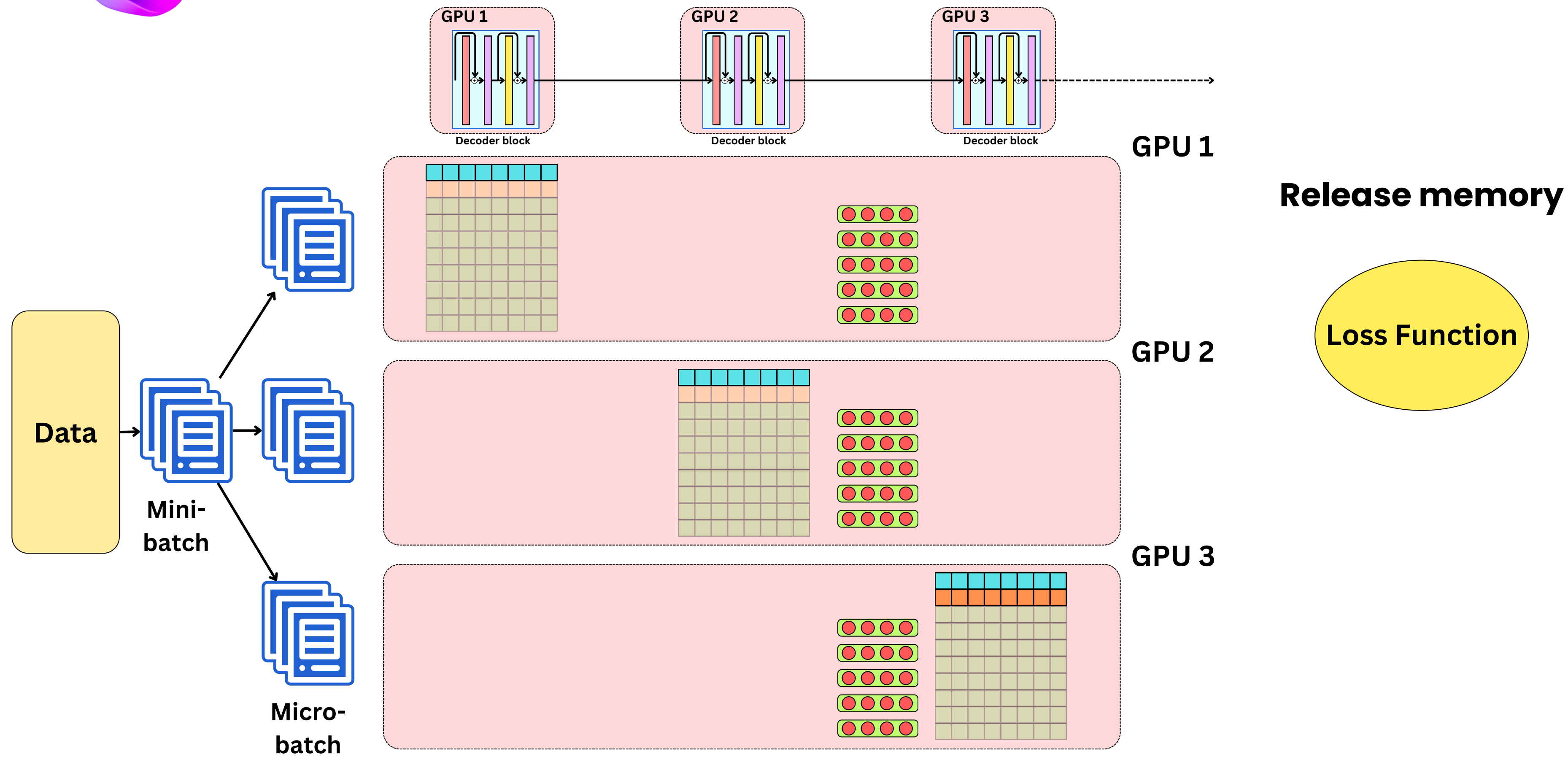


Zero Redundancy Optimizer



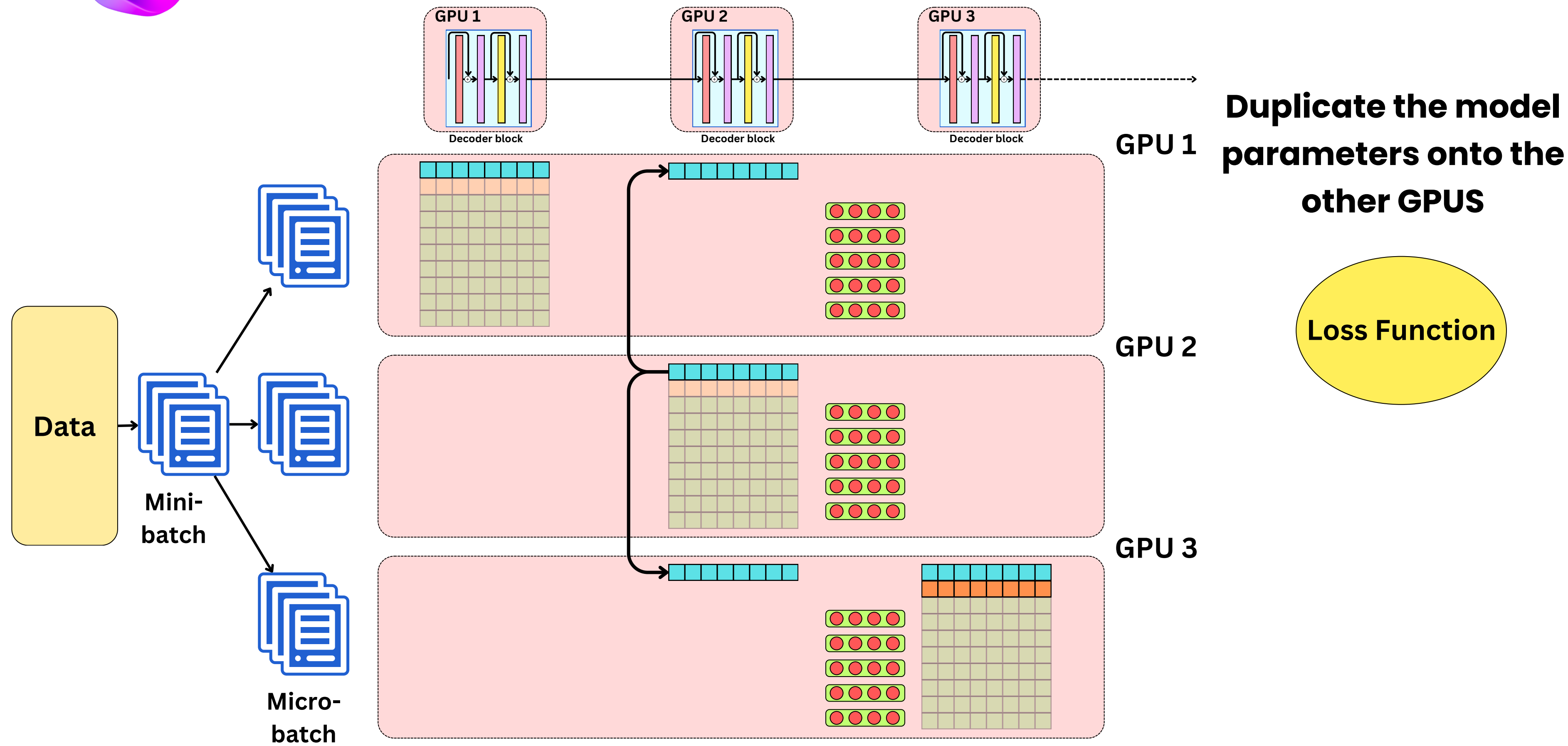


Zero Redundancy Optimizer



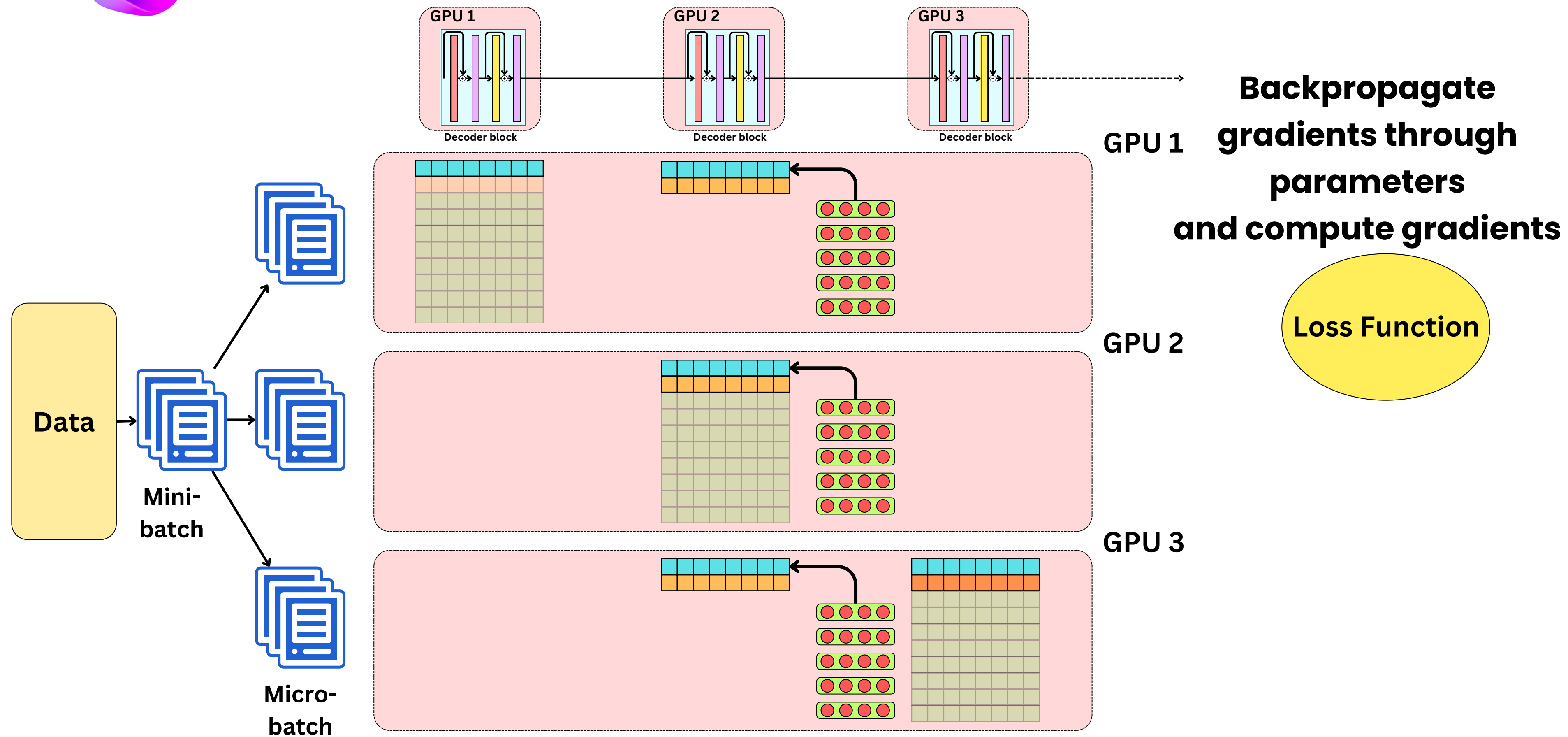


Zero Redundancy Optimizer



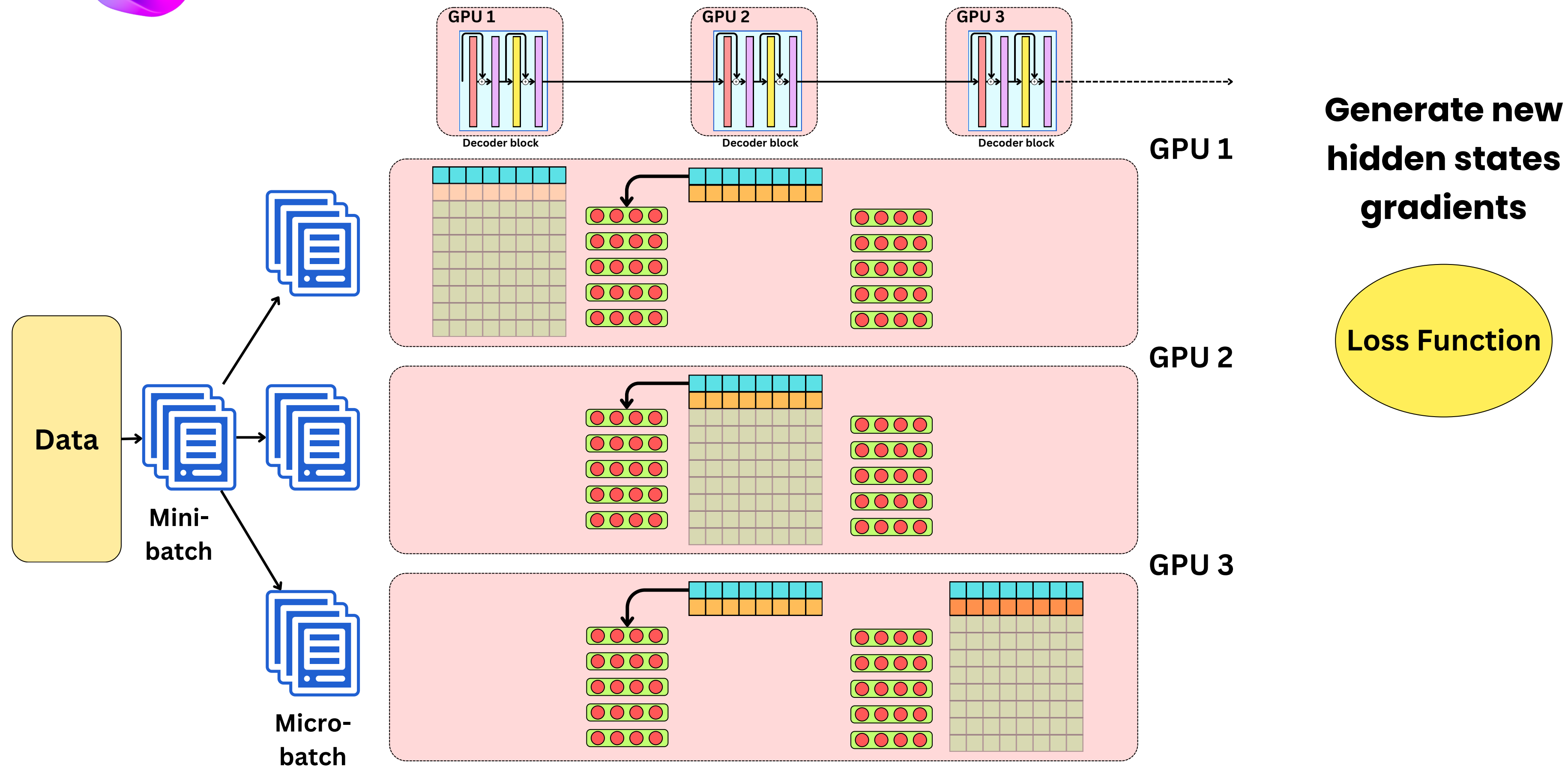


Zero Redundancy Optimizer



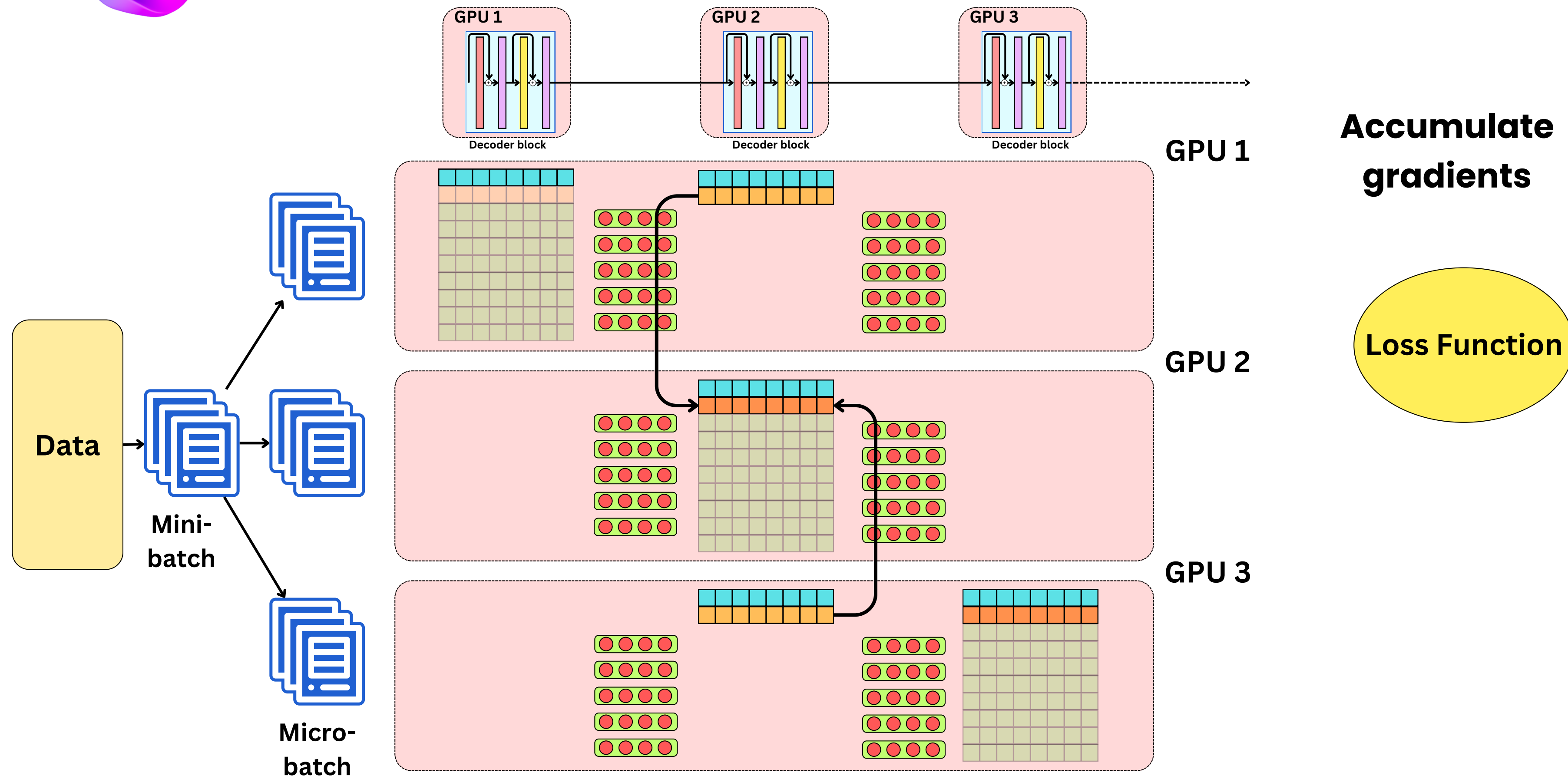


Zero Redundancy Optimizer



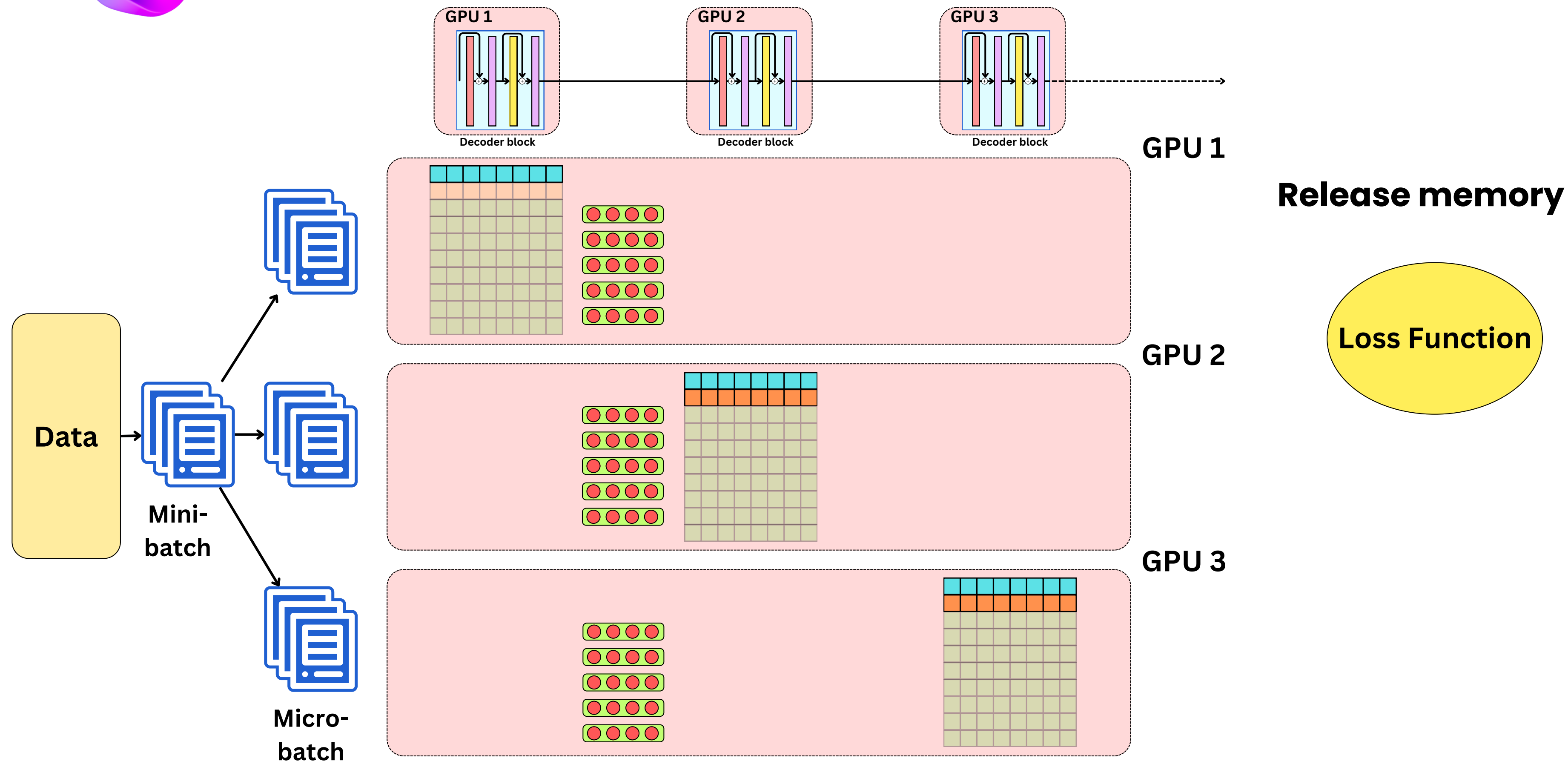


Zero Redundancy Optimizer



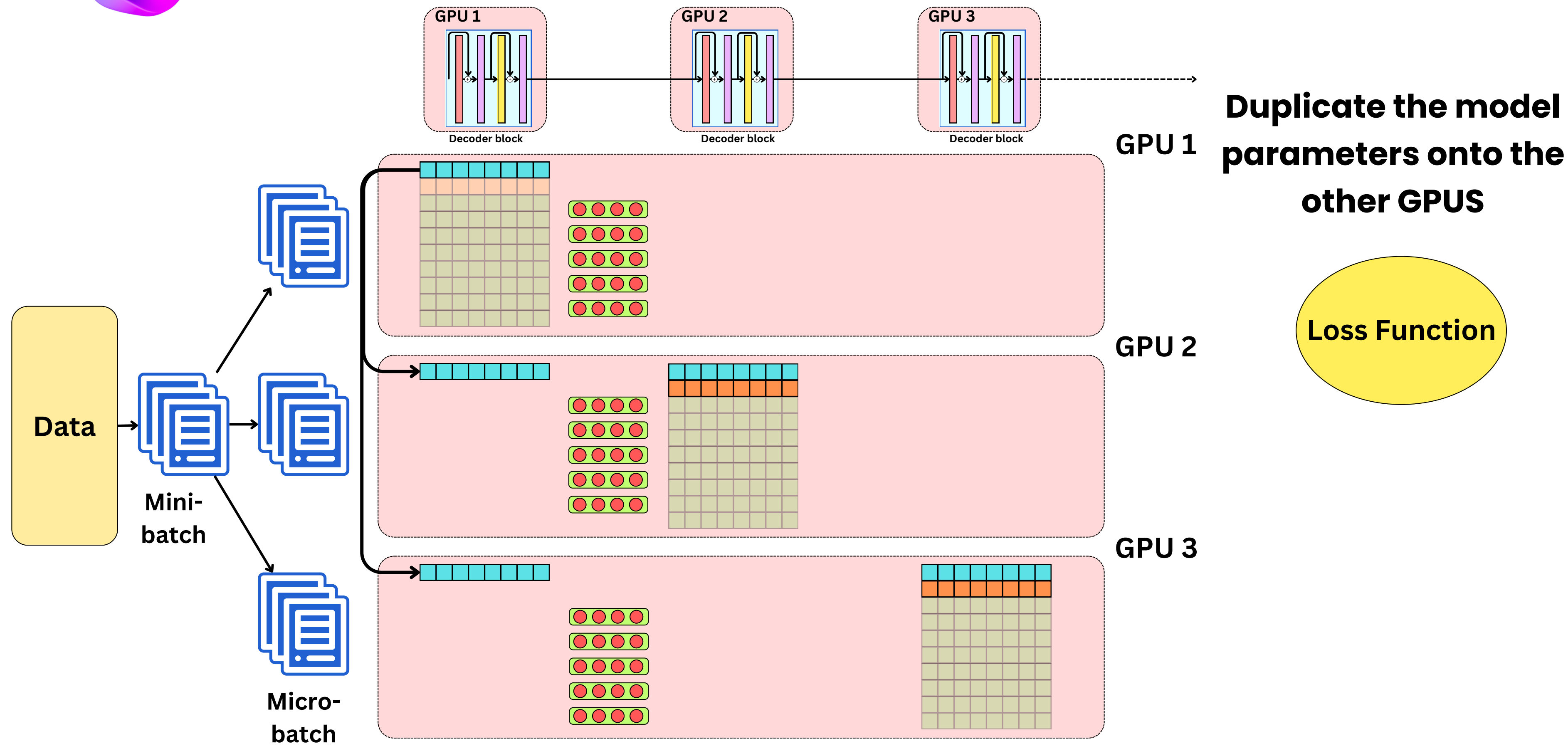


Zero Redundancy Optimizer



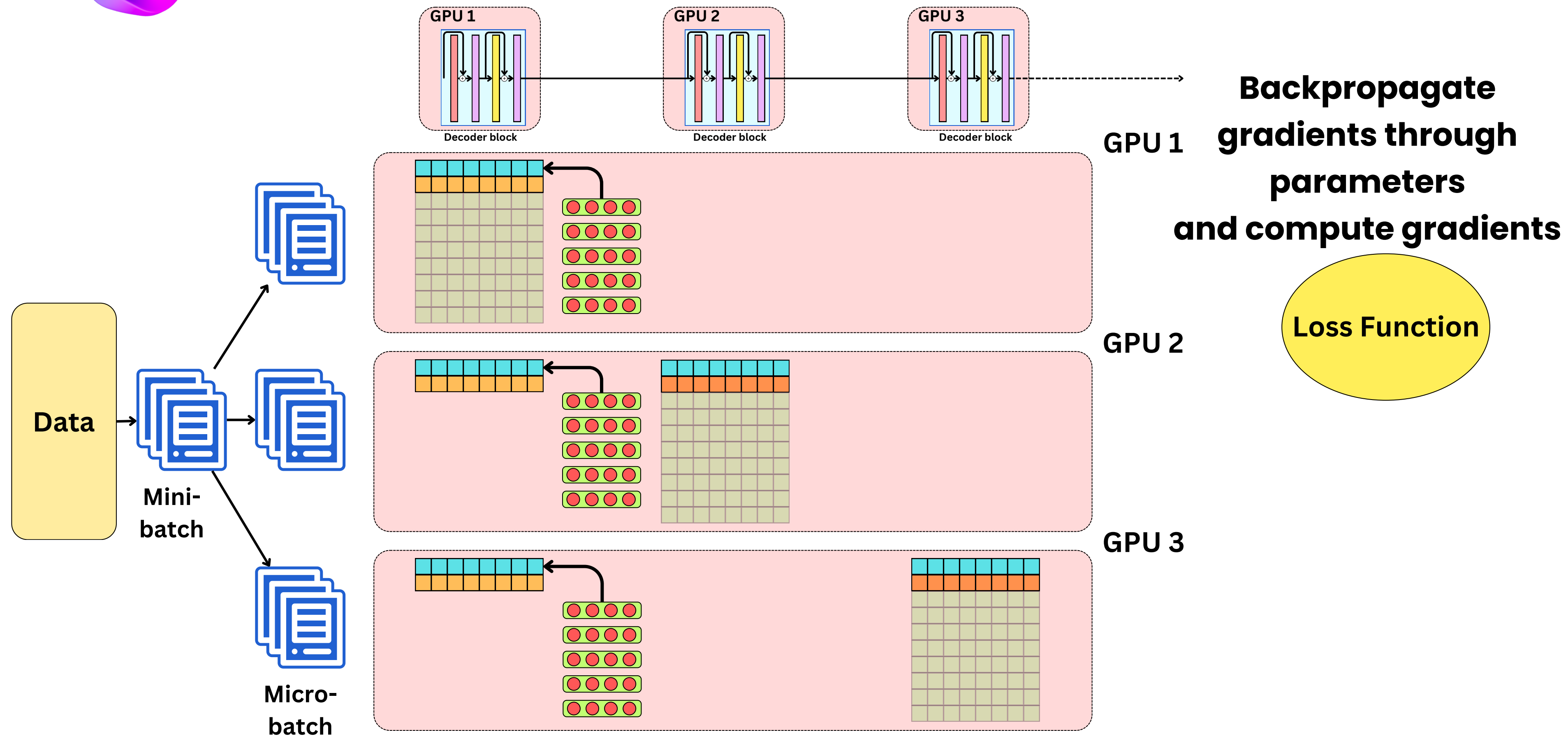


Zero Redundancy Optimizer



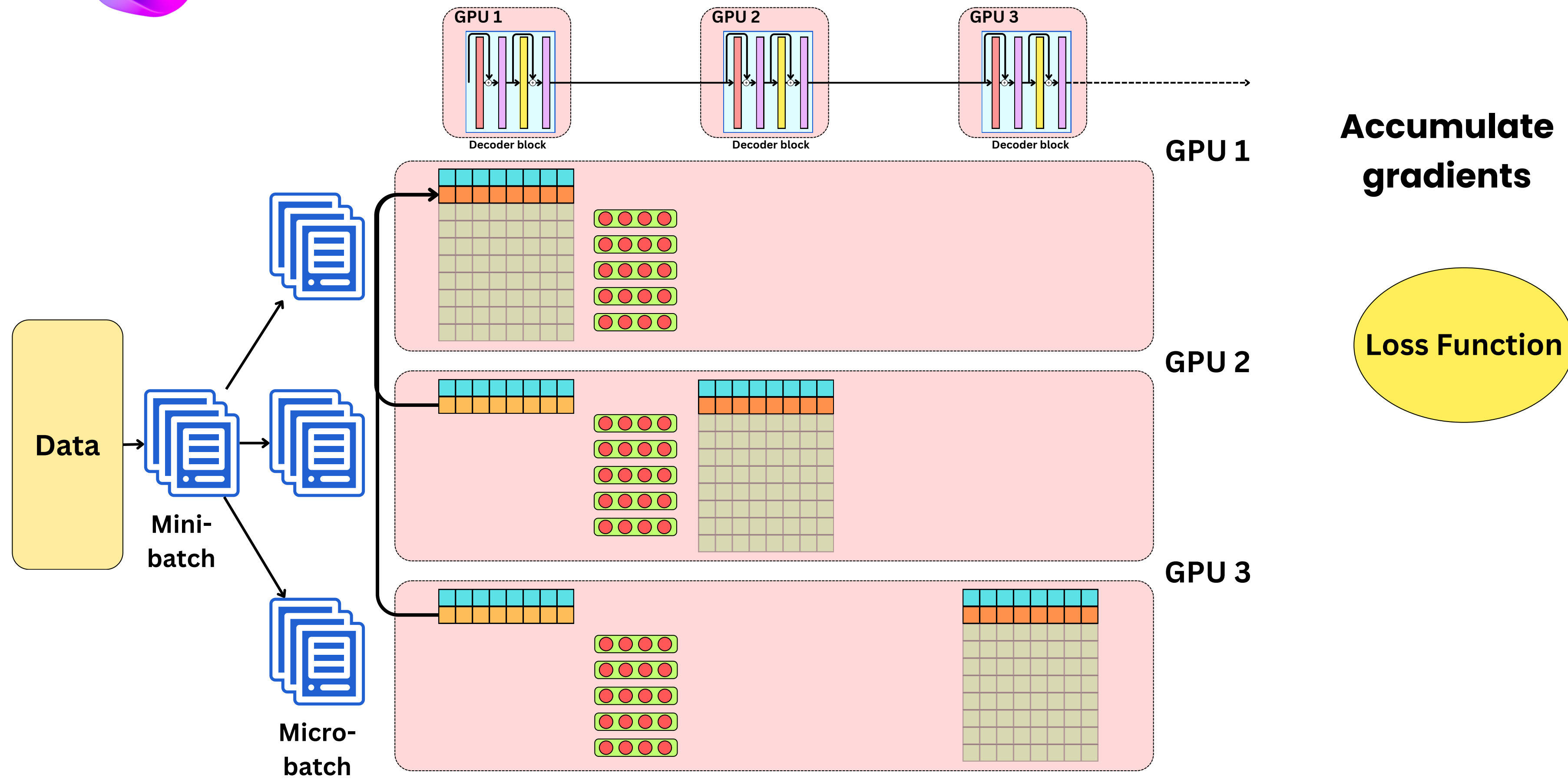


Zero Redundancy Optimizer



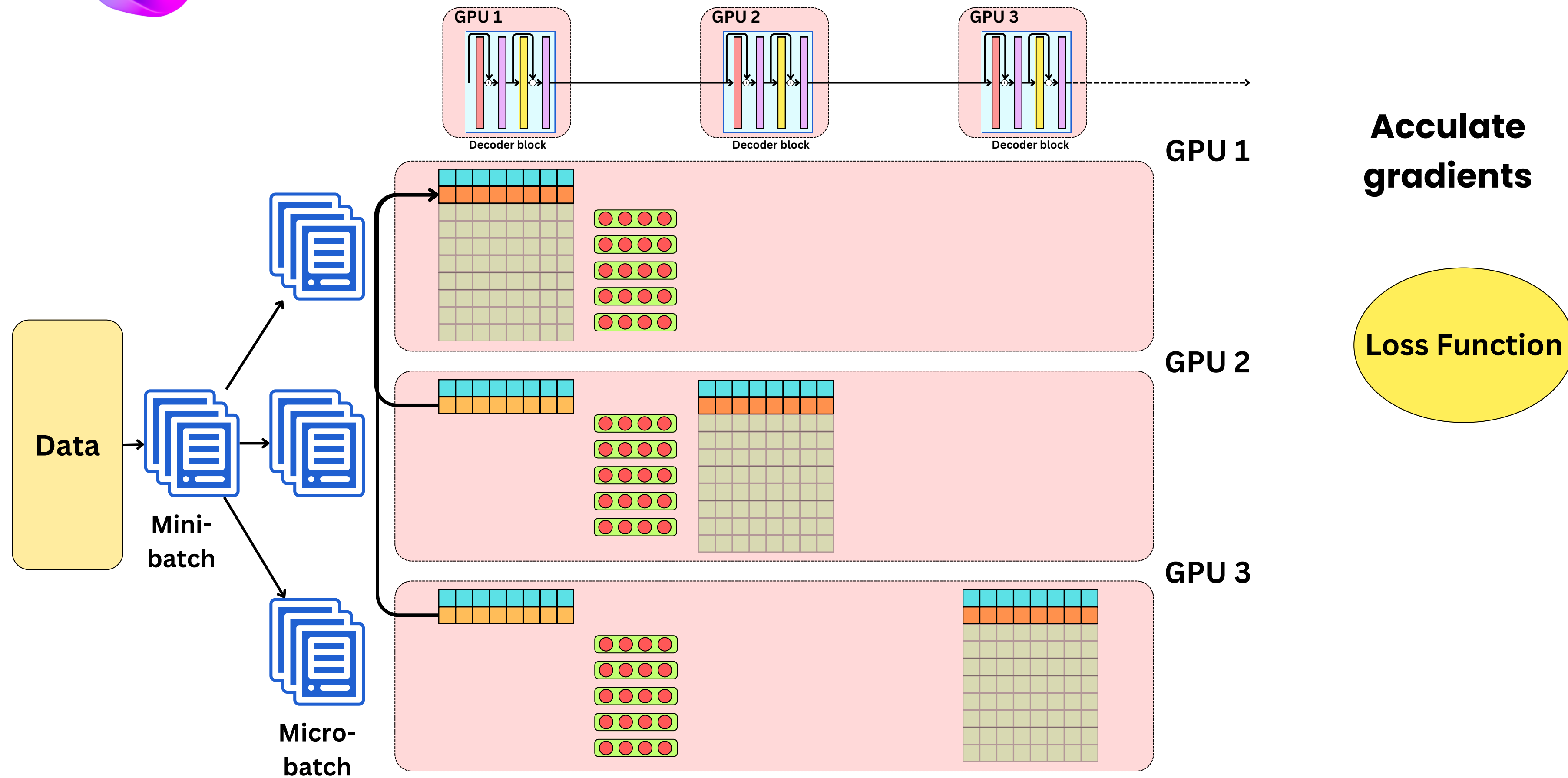


Zero Redundancy Optimizer



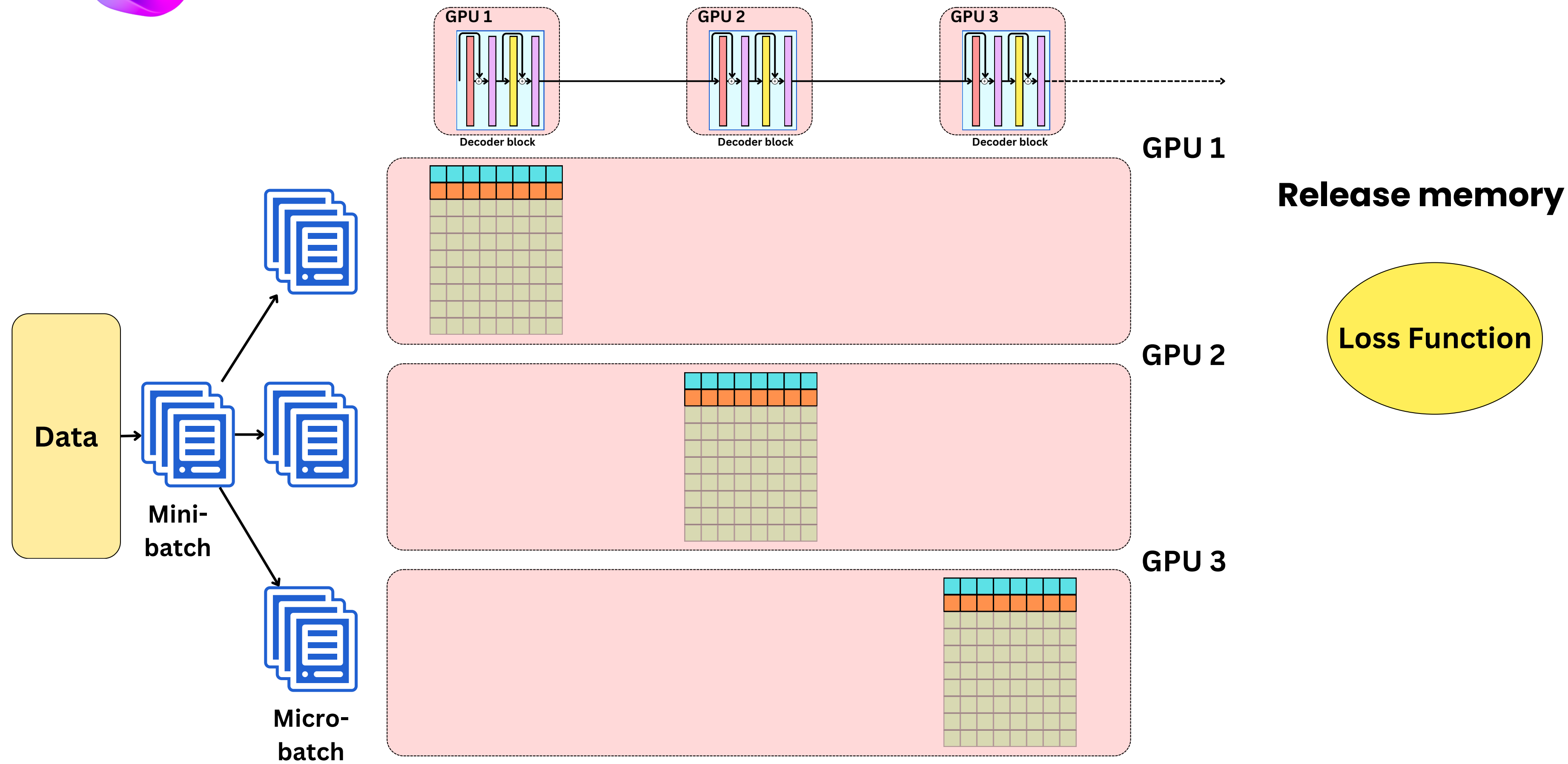


Zero Redundancy Optimizer



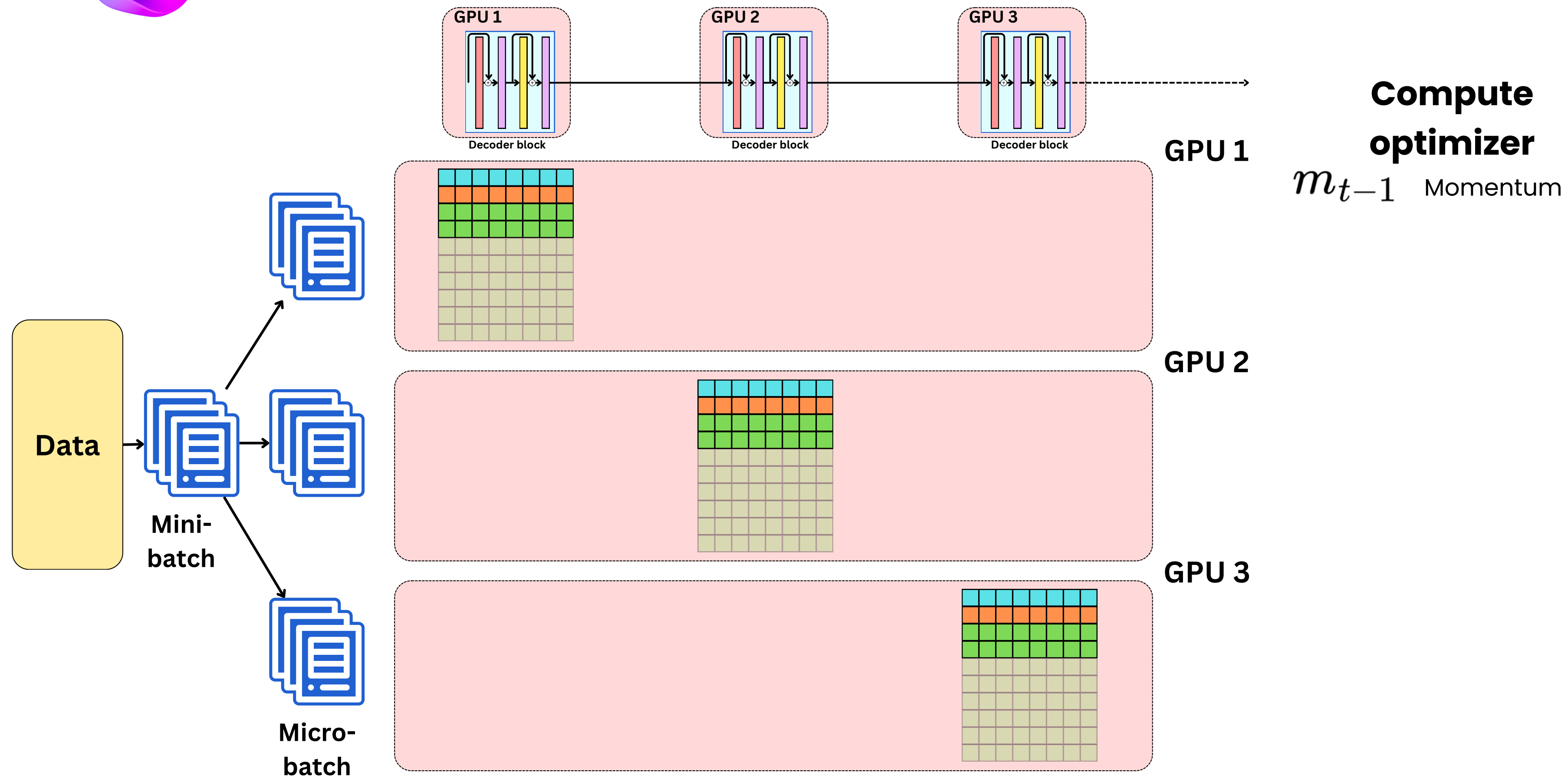


Zero Redundancy Optimizer



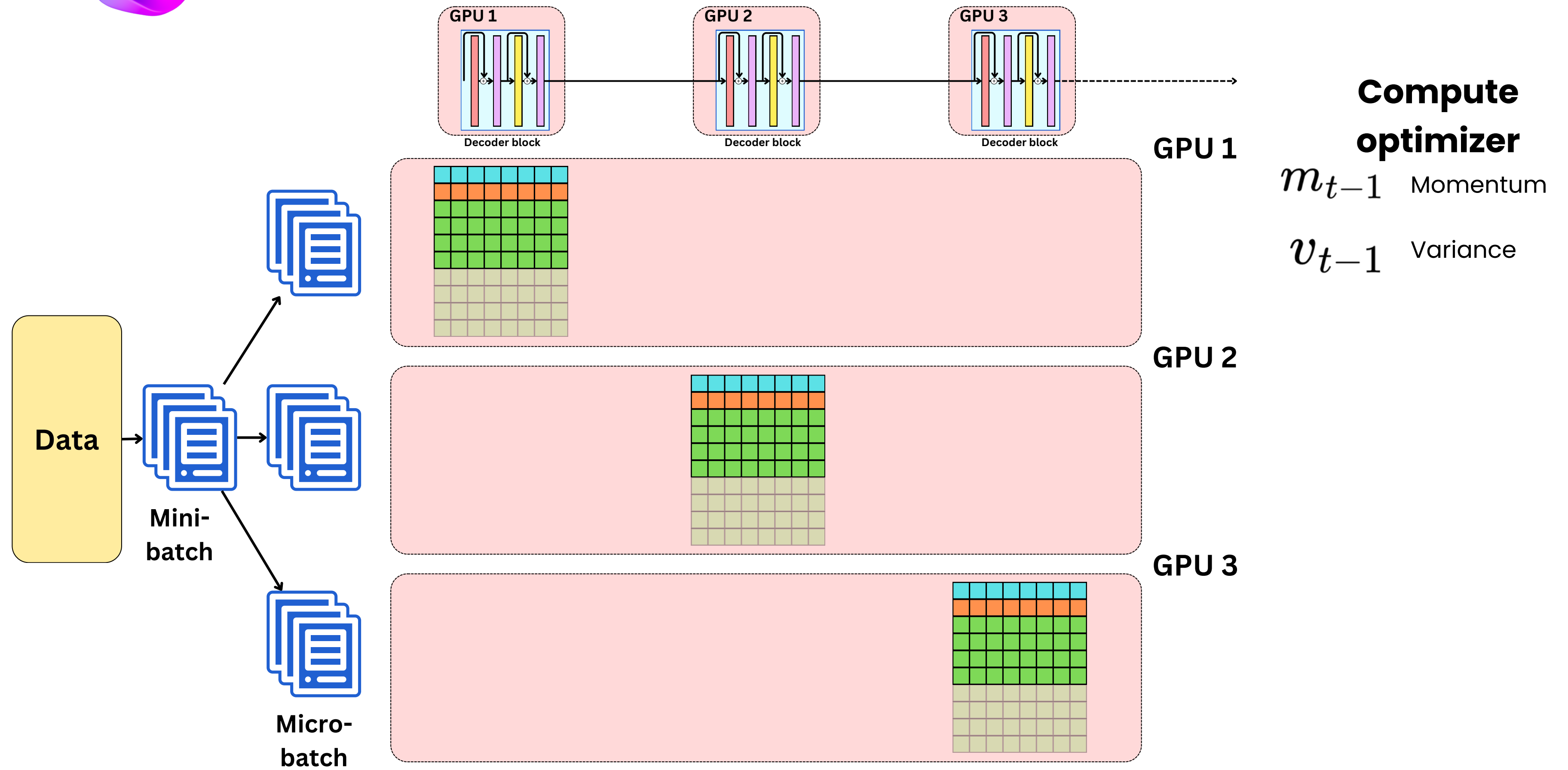


Zero Redundancy Optimizer



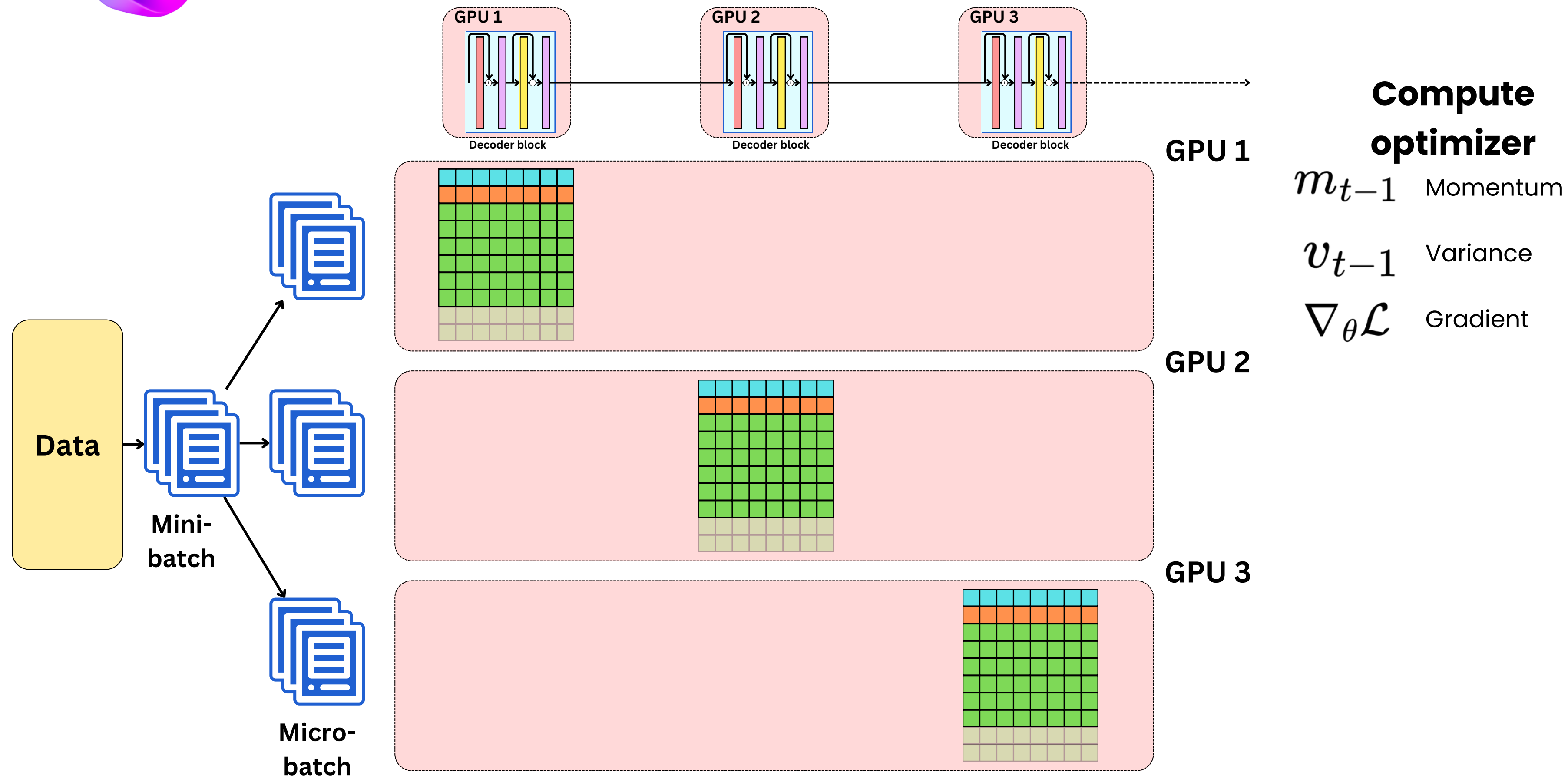


Zero Redundancy Optimizer



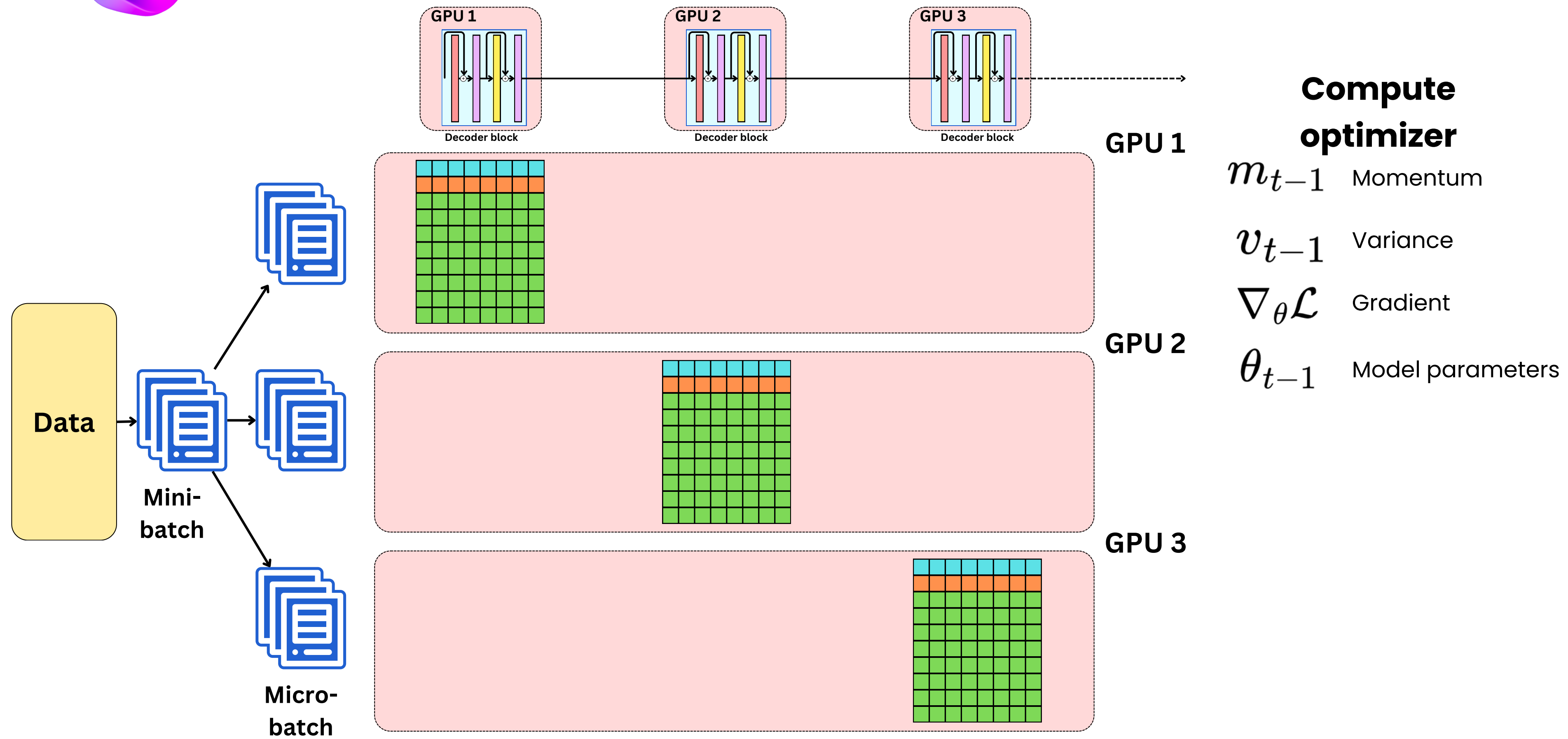


Zero Redundancy Optimizer



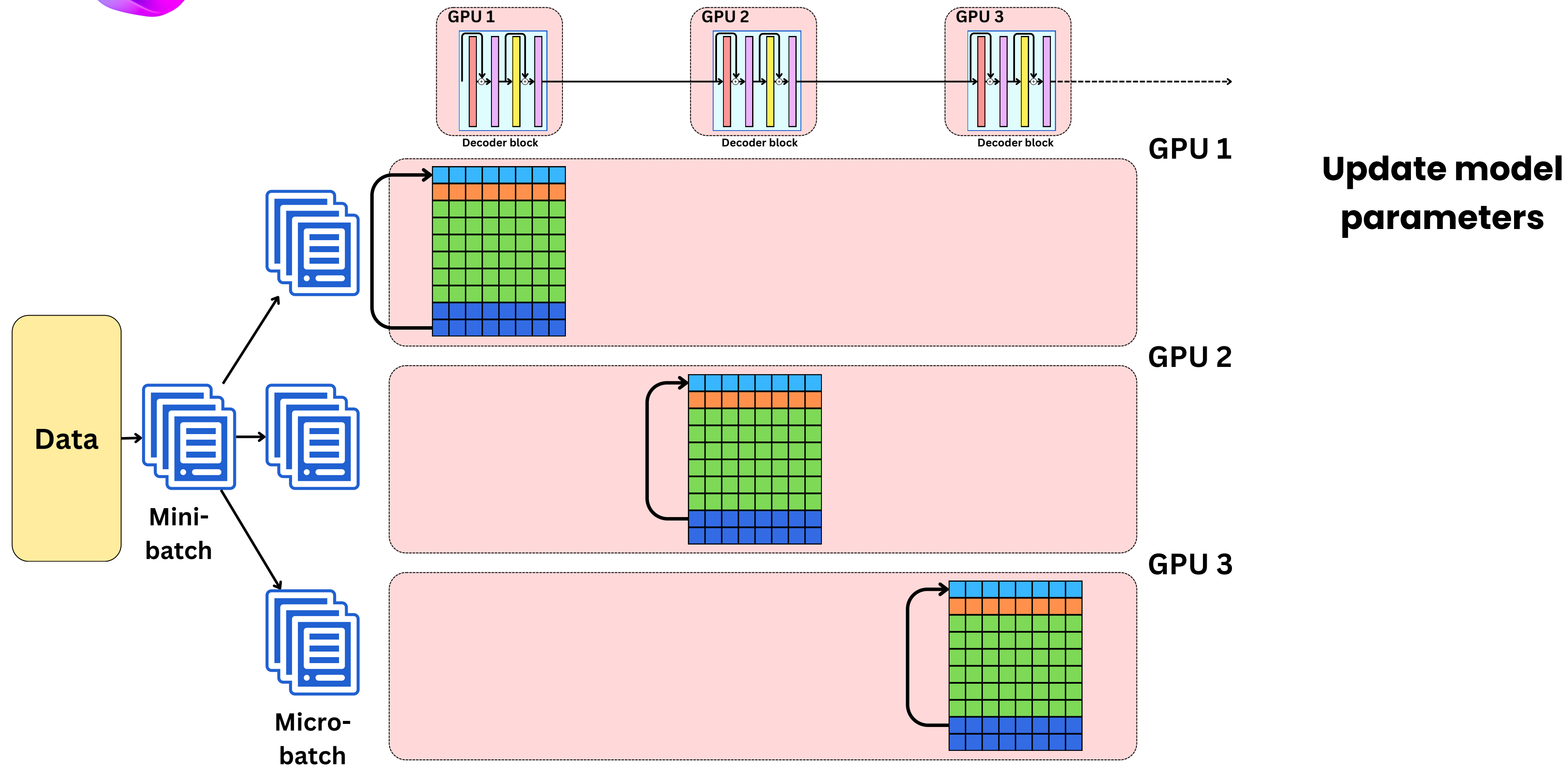


Zero Redundancy Optimizer





Zero Redundancy Optimizer





Zero Redundancy Optimizer

